



Vivekanand Education Society's

Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074.

Department of Information Technology

CERTIFICATE

This is to certify that **Atharva Prabhu** of **D20A** semester **VIII**, have successfully completed necessary experiments in the **ITC801: Blockchain & DLT Lab** under my supervision in **VES Institute of Technology** during the academic year **2025-2026**.

Subject Teacher

Name: Mrs. Kajal Joseph

Signature:

Head of Department

Name: Dr. Mrs. Shalu Chopra

Signature:

Lab Teacher

Name: Kajal Joseph

Signature:



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

Lab Code	Lab Name	Credit
ITC801	Blockchain and DLT Lab	1

Programme Outcomes: The graduate will be able to:

PO1	Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization as specified in WK1 to WK4 respectively to develop to the solution of complex engineering problems.
PO2	Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development. (WK1 to WK4)
PO3	Design/Development of Solutions: Design creative solutions for complex engineering problems and design / develop systems /components / processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required. (WK5)
PO4	PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions. (WK8).
PO5	Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems. (WK2 and WK6)
PO6	The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment. (WK1, WK5, and WK7).
PO7	Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws. (WK9)
PO8	Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.
PO9	Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences
PO10	Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

PO11	Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.
PSO 1	Computing & Emerging Technology: Apply core computing and modern technologies, such as AI, Cloud Computing, IoT, cybersecurity, and data-driven tools, to build efficient and innovative solutions.
PSO 2	Professionalism & Innovation : Demonstrate ethics, teamwork, and an entrepreneurial mindset to deliver sustainable solutions for society.

Course Outcomes: Student will be able	
ITC801.1	Describe the basic concept of Blockchain and Distributed Ledger Technology.
ITC801.2	Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions
ITC801.3	Implement smart contracts in Ethereum using different development frameworks.
ITC801.4	Develop applications in permissioned Hyperledger Fabric network.
ITC801.5	Interpret different Crypto assets and Crypto currencies
ITC801.6	The use of Blockchain with AI, IoT and Cyber Security using case studies.

ITC801 Blockchain and DLT Lab													
CO	PO											PSO	
	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO 10	PO 11	PSO 1	PSO 2
ITC801.1	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.2	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.3	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.4	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.5	2	3	3	3	3	3	3	3	3	2	2	3	3
ITC801.6	2	3	3	3	3	3	3	3	3	2	2	3	3
Avg PO AL	2	3	3	3	3	3	3	3	3	2	2	3	3



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

ITC801 Blockchain and DLT Lab												
CO	Experiment											
	Exp 1	Exp 2	Exp 3	Exp 4	Exp 5	Exp 6	Exp 7	Exp 8	Exp 9	Exp 10	Exp 11	Exp 12
ITC801.1	3	3	2	-	2	2	2	-	-	3	-	3
ITC801.2	2	3	3	-	2	2	3	2	-	3	-	3
ITC801.3	-	-	2	3	3	3	3	3	2	-	-	3
ITC801.4	-	-	-	-	-	-	-	-	-	-	3	3
ITC801.5	2	3	3	-	-	2	2	2	-	3	-	3
ITC801.6	-	-	-	-	-	-	-	-	-	2	-	3

INDEX

Sr. No.	List of Experiments
1	Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash
2	Create a Blockchain using Python
3	Create a Cryptocurrency using Python and perform mining in the Blockchain created.
4	Hands-on Solidity Programming Assignments for creating Smart Contracts
5	Deploying a Voting/Ballot Smart Contract
6	Creating Smart Contracts and performing transactions using Solidity and Remix IDE
7	Implement the embedding wallet (Metamask) and transaction using Solidity
8	To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.
9	To implement a Private Ethereum Blockchain using Geth.
10	Explore the Cryptocurrency Landscape
11	Implementation of Hyperledger Fabric Network and Asset Management using Chaincode
12	Mini Project
12	Assignment-1



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY**
Department of Information Technology
Academic Year: 2025-26

13	Assignment-2
14	Assignment-3
15	Mock Viva

Subject teacher



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment No. 01

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO 01

Name: Atharva Prabhu

Class: D20A

Roll: No: 45

Batch: B

Aim: Write a Python program to understand SHA and Cryptography in Blockchain, Merkle root tree hash.

Theory:

Cryptographic Hash Functions in Blockchain

A cryptographic hash function is a mathematical algorithm that converts input data of any size into a fixed-length output known as a hash value or digest. In blockchain systems, hash functions play a critical role in maintaining data integrity, security, immutability, and trust within decentralized networks.

Widely used hash functions in blockchain include:

- SHA-256 (Bitcoin)
- Keccak-256 / SHA-3 (Ethereum)
- RIPEMD-160

These algorithms ensure that blockchain data remains tamper-proof and verifiable.

Characteristics of Cryptographic Hash Functions:

A secure cryptographic hash function must satisfy the following properties:

1. Deterministic

The same input always produces the same hash output, ensuring consistency across all blockchain nodes.

2. Fixed Output Length

Regardless of input size, the hash output has a constant length.

Example: SHA-256 always generates a 256-bit hash.

3. Pre-image Resistance

Given a hash value, it is computationally infeasible to determine the original input, protecting transaction data from reverse engineering.

4. Second Pre-image Resistance

Given an input and its hash, it is extremely difficult to find another input that produces the same

hash, preventing data replacement attacks.

5. Collision Resistance

It is highly improbable to find two different inputs that generate identical hashes, ensuring uniqueness of transactions.

6. Avalanche Effect

A small change in input produces a drastically different hash output, allowing immediate detection of data tampering.

Properties of SHA-256

SHA-256 (Secure Hash Algorithm – 256-bit) belongs to the SHA-2 family and is one of the most commonly used hashing algorithms in blockchain technology. It converts any input data into a secure 256-bit hash value, regardless of the input's original size.

- **Fixed 256-bit Output:** SHA-256 always produces a hash of exactly 256 bits, usually shown as 64 hexadecimal characters, no matter how large the input is.
- **High Security Strength:** It is widely trusted for modern cryptography and is a standard choice in blockchain networks for strong protection.
- **Collision Resistance:** The chance that two different pieces of data generate the same hash is extremely rare, ensuring reliable data integrity.
- **Avalanche Property:** Even the smallest change in input data results in a completely different hash output, making unauthorized modifications easy to detect.
- **Irreversible (One-Way Function):** It is practically impossible to reverse the hash and recover the original input data.

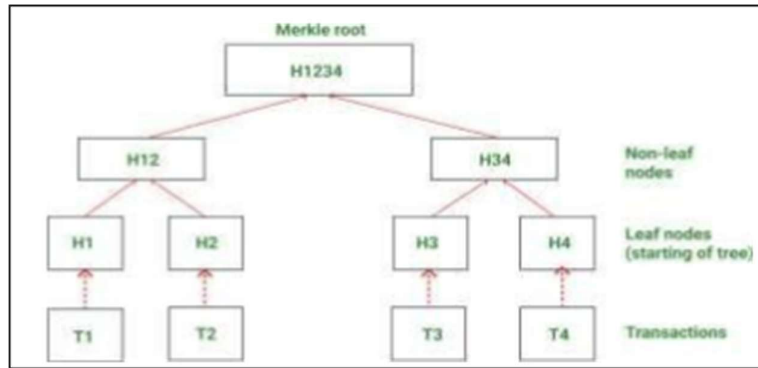
Merkle Tree (Hash Tree)

A Merkle Tree is a specialized binary tree structure used in blockchain to store and verify a large number of transactions efficiently. In this structure, every transaction is first converted into a hash, forming the leaf nodes. These hashes are then combined in pairs, hashed again, and repeated layer by layer until one final hash remains, called the **Merkle Root**.

The Merkle Root represents the complete set of transactions in a block. By storing only this single root hash inside the block header, blockchain systems can confirm that all transactions remain unchanged. If even one transaction is altered, the Merkle Root changes instantly, making fraud or tampering easy to detect.

Structure of a Merkle Tree

A Merkle Tree follows a layered hierarchical arrangement. It begins with individual transaction hashes at the bottom level. These hashes are merged pairwise to form parent nodes, continuing upward through multiple levels until reaching the topmost hash, the **Merkle Root**, which summarizes and secures all transactions within the block.



1. **Leaf Nodes:** These are the hashes of individual transactions in a block. Each transaction is first hashed to form a leaf node.
2. **Intermediate (Parent) Nodes:** Pairs of leaf nodes are combined and hashed again to form parent nodes. This process continues upward, combining child nodes at each level.
3. **Root Node (Merkle Root):** The topmost node of the tree, representing all transactions in the block. If any transaction changes, the Merkle Root changes, making tampering detectable.

2. Merkle Root

The Merkle Root is the **topmost hash** of the Merkle Tree. It acts as a summary of all transactions in a block.

If any transaction is changed, its hash changes, which affects the Merkle Root. Since the Merkle Root is stored in the block header, this makes tampering easy to detect.

3. Working of Merkle Tree

The Merkle Tree works by organizing and summarizing all transactions in a block into a single hash (the Merkle Root) so that data can be verified efficiently and securely.

The process follows these steps:

- **Hashing Transactions:** Each transaction in the block is first hashed using a cryptographic hash function, usually SHA-256. These hashes form the leaf nodes of the Merkle Tree.

Example:

Transactions: T1, T2, T3, T4

Hashes: H(T1), H(T2), H(T3),

H(T4)

- **Pairing and Hashing:** The leaf nodes are grouped in pairs. Each pair of hashes is concatenated (joined together) and then hashed again to create a parent node. If the number of transactions is odd, the last hash is duplicated to form a pair. This ensures that every level of the tree has an even number of nodes.

Example:

- Pair 1: $H(T1) + H(T2) \rightarrow \text{Hash} = H12$
- Pair 2: $H(T3) + H(T4) \rightarrow \text{Hash} = H34$
- **Building the Tree:** The pairing and hashing process continues up the tree, combining parent nodes to create new higher-level nodes. **Example:**
 $H12 + H34 \rightarrow \text{Hash} = H1234$
- **Creating the Merkle Root:** This process repeats until only one hash remains at the top of the tree. This top-level hash is called the Merkle Root, which represents all transactions in the block.
- **Verification:** To verify that a transaction exists in a block, a Merkle Proof is used. Instead of checking every transaction, only a small number of hashes along the path from the transaction leaf to the Merkle Root are needed. This makes verification fast and efficient, even for large blocks of data.

Example:

To verify T1: Use $H(T2)$ and $H34$ along with $H(T1)$ to recalculate $H1234$. If it matches the Merkle Root, T1 is valid.

- **Tamper Detection:** If any transaction is modified, its hash changes. This change propagates up the tree and alters the Merkle Root. Since the Merkle Root is stored in the block header, any tampering becomes immediately obvious. **Example:**
If T3 changes $\rightarrow H(T3)$ changes $\rightarrow H34$ changes $\rightarrow H1234$ changes $\rightarrow$ Merkle Root mismatch.

4. Benefits of Merkle Tree

- **Efficient Verification:** Only a small number of hashes are needed to verify a transaction using a Merkle Proof, so checking data is fast even for large blocks.
- **Data Integrity:** Any change in a transaction alters the Merkle Root, making it easy to detect tampering.
- **Reduced Storage:** Instead of storing all transaction data in the block header, only the Merkle Root is stored, saving space.
- **Scalability:** Merkle Trees allow blockchains to handle a large number of transactions without slowing down verification.
- **Security:** The structure ensures that transactions cannot be altered without being noticed, keeping the blockchain secure.

5. Use of Merkle Tree in Blockchain

- **Efficient Transaction Verification:** Nodes can verify a single transaction without downloading the entire block, using a Merkle Proof.
- **Ensures Data Integrity:** Any change in a transaction immediately changes the Merkle Root, helping detect tampering.

- **Reduces Storage Needs:** Only the Merkle Root is stored in the block header, instead of all transaction data, saving space.
- **Supports Lightweight Nodes:** Simple Payment Verification (SPV) nodes can confirm transactions securely without holding the full blockchain.

Applications of Merkle Tree

Merkle Trees are an important data structure used not only in blockchain but also in many digital security systems. Their main advantage is that they can verify huge amounts of data quickly without needing to store or process everything. Below are some common practical uses:

1. Blockchain Block Transaction Management

In cryptocurrencies such as Bitcoin, thousands of transactions may exist inside a single block. Merkle Trees help compress this information efficiently.

- **Example:** Instead of keeping the entire transaction list in the block header, only one hash value—the **Merkle Root**—is stored.
- If any transaction is modified, the Merkle Root changes instantly.

Why useful: Ensures block integrity and makes transaction verification simpler.

2. Lightweight Wallet Verification (SPV Method)

Many mobile wallets and small devices cannot store complete blockchain data. They use Merkle Trees for fast checking.

- **Example:** A phone wallet can verify a payment by downloading only the transaction proof path, not the entire block.

Benefit: Saves internet bandwidth, storage space, and still maintains trust.

3. Software Development and File Tracking

Merkle Trees are widely used in systems that need reliable tracking of file changes.

- **Example:** In software platforms, every file version can be hashed, and any update changes the root hash, clearly showing that the content was altered.

Advantage: Helps in managing project history and preventing unnoticed modifications.

4. Cloud Storage Validation

Cloud providers must guarantee that stored files are not corrupted over time.

- **Example:** A Merkle Tree can be built from file chunks, allowing users to verify that their uploaded data remains unchanged.

Result: Secure storage and easy corruption detection.

5. Secure Data Sharing in P2P Systems

Peer-to-peer networks often share data in fragments, making verification necessary.

- **Example:** When downloading a large video split into parts, Merkle Trees allow users to confirm each part is authentic before assembling the full file.

Benefit: Prevents fake or tampered file pieces from spreading.

6. Digital Auditing and Record Security

Merkle Trees are also useful in financial audits and secure record-keeping.

- **Example:** Banks or companies can store transaction logs in a Merkle Tree so that auditors can later confirm no records were changed.

Importance: Ensures transparency and long-term trust in stored logs.

Colab Notebook:

[Code Link](#)

Code & Output:

1. Hash Generation using SHA-256: Developed a Python program to compute a SHA-256 hash for any given input string using the hashlib library.

```
import hashlib
def create_hash(string):
    # Create a hash object using SHA-256 algorithm
    hash_object = hashlib.sha256()
    # Convert the string to bytes and update the hash object
    hash_object.update(string.encode('utf-8'))
    # Get the hexadecimal representation of the hash
    hash_string = hash_object.hexdigest()
    # Return the hash string
    return hash_string

# Example usage
input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

2. Target Hash Generation with Nonce: Created a program to generate a hash code by concatenating a user input string and a nonce value to simulate the mining process.

```
import hashlib
def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()
        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce
        nonce += 1

# Get user input
input_string = "A"
num_zeros = 1
# Find the expected nonce
expected_nonce = find_nonce(input_string, num_zeros)
# Print the expected nonce
print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)
```

Enter a string: Atharva

Enter the nonce: 4

Hash Code: c18c0ab360b4b8f433845a7df2c266d5a358929bedb633b424514d1b99ea1b80

3. **Proof-of-Work Puzzle Solving:** Implemented a program to find the nonce that, when combined with a given input string, produces a hash starting with a specified number of leading zeros.

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros
    while True:
        # Concatenate the string and nonce
        hash_string = input_string + str(nonce)
        # Calculate the hash using SHA-256
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()
        # Check if the hash code has the required number of leading zeros
        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
            return nonce
        nonce += 1
    # Get user input
    input_string = "A"
    num_zeros = 1
    # Find the expected nonce
    expected_nonce = find_nonce(input_string, num_zeros)
    # Print the expected nonce
    print("Input String:", input_string)
    print("Leading Zeros:", num_zeros)
    print("Expected Nonce:", expected_nonce)
```

```
Enter the input string: Atharva
Enter the number of leading zeros: 4
Hash: 0000161e30b1ef5493a7a64d6b8f7e4c11423737eb8406e8dec44f9ee7ebd48c
Input String: Atharva
Leading Zeros: 4
Expected Nonce: 97093
```

4. **Merkle Tree Construction:** Built a Merkle Tree from a list of transactions by recursively hashing pairs of transaction hashes, doubling up last nodes if needed, and generated the Merkle Root hash for blockchain transaction integrity.

```
import hashlib

def sha256(data):
    return hashlib.sha256(data.encode()).hexdigest()
```

```

def merkle_root(transactions):
    hashes = [sha256(tx) for tx in transactions]

    while len(hashes) > 1:
        if len(hashes) % 2 != 0:
            hashes.append(hashes[-1])
        )
        new_level = []
        for i in range(0, len(hashes), 2):
            combined_hash = hashes[i] + hashes[i +
            1]
            new_level.append(sha256(combined_hash))
        hashes = new_level
    return hashes[0]

transactions = [
    "User1 sends 12 coins to User7",
    "Miner42 rewards itself 6.25 coins",
    "WalletA transfers 0.8 coins to WalletB",
    "ContractX releases 15 coins to Vault9"
]

print("Merkle Root Hash:", merkle_root(transactions))

```

Merkle Root Hash: 6f3d19cf44fda0e0a0af16ec5c6d00db12d88eeabc4b875b105db2d87b6b4eb

Conclusion:

Cryptographic hash functions like **SHA-256** help keep blockchain data safe, secure, and unchangeable. **Merkle Trees** organize transactions in a way that makes it easy to verify them quickly and detect any tampering. Together, they make blockchain trustworthy, capable of handling large amounts of data without compromising security. These concepts help us understand how blockchain maintains security, efficiency even with large amounts of data.



-----VIVEKANAND EDUCATION
SOCIETY'S INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26

ITC801: Blockchain & DLT Lab
Experiment 02

Aim: Create a Blockchain using Python

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT NO.2

Name: Atharva Prabhu

Class: D20A

Roll No:45

Batch:B

Aim: Create a Blockchain using Python

1. What is Blockchain

Blockchain is revolutionary technology that functions as a shared, immutable digital ledger, often described as a decentralized database or distributed ledger technology. It records transactions across a network of computers in a way that ensures security, transparency, and resistance to tampering. The term "blockchain" derives from its structure: data is organized into blocks, each linked to the previous one through cryptographic hashes, forming a continuous chain. This decentralized nature eliminates the need for intermediaries, allowing multiple parties to perform transactions without third-party intervention.

Key characteristics include:

- **Decentralization:** No single entity controls the network; it's maintained by nodes (computers) that validate and record transactions.
- **Immutability:** Once data is added, it cannot be altered without consensus from the network, making it tamper-proof.
- **Transparency:** All participants can view the ledger, but privacy is maintained through cryptography.
- **Security:** Uses cryptographic techniques to protect data.

Blockchain stores various types of information, but it's most commonly used for transactions in cryptocurrencies like Bitcoin. It operates on principles such as consensus mechanisms to agree on the state of the ledger. Unlike traditional databases that structure data in tables, blockchain uses blocks

chained together, ensuring that changes to one block would require altering all subsequent blocks, which is computationally infeasible

2. What is a Block?

A block is the fundamental building block of a blockchain, acting as a container for data such as transactions. It serves as a record in the digital ledger, similar to a page in a book, where each block holds a complete record of transactions or other information. Blocks are linked together in a chain, with each new block referencing the previous one via a cryptographic hash, creating an unbreakable sequence.

In essence, a block is a node in the blockchain network that stores encrypted data, including a list of transactions, a timestamp, and a unique identifier (hash). The first block in any blockchain is called the Genesis Block, which has no previous block reference and is hardcoded into the protocol. This structure ensures the integrity of the entire chain, as altering any block would invalidate the hashes of all following blocks

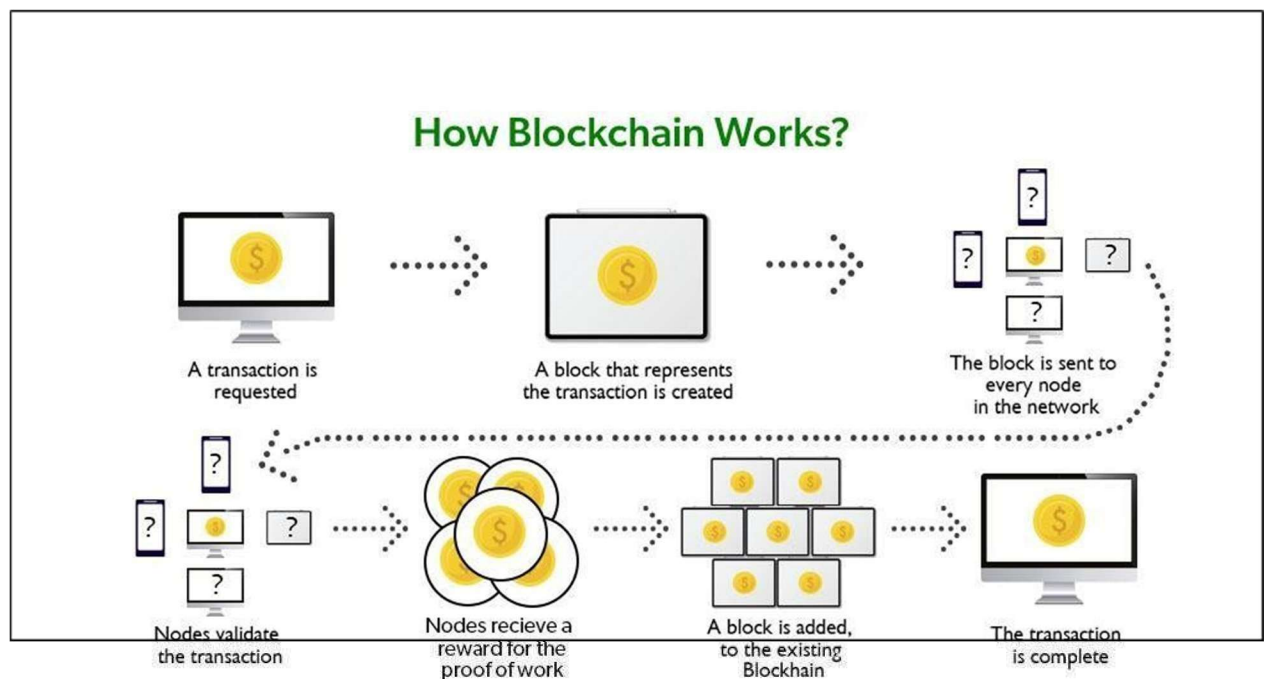
2. Components of a Block

A block in blockchain consists of two main parts: the block header and the block body (containing transactions). The header is crucial for identification and linking, while the body holds the actual data.

Key components include:

Component	Description
Block Header	Identifies the block and includes metadata like version, timestamp, Merkle root, difficulty target, nonce, and previous block hash.
Version	A 4-byte field tracking software or protocol upgrades.
Previous Block Hash	A 32-byte reference to the hash of the prior block, ensuring chain continuity.
Merkle Root	A 32-byte hash of all transactions in the block, derived from a Merkle tree for efficient verification.
Timestamp	A 4-byte record of the block's approximate creation time.
Difficulty Target	A 4-byte value setting the mining difficulty.
Nonce	A 4-byte number used in proof-of-work mining to find a valid hash.
Transactions	The list of verified transactions stored in the block body.

These elements work together to maintain security and enable validation. The header alone is often sufficient for light clients to verify blocks without the full data



3. Process of mining

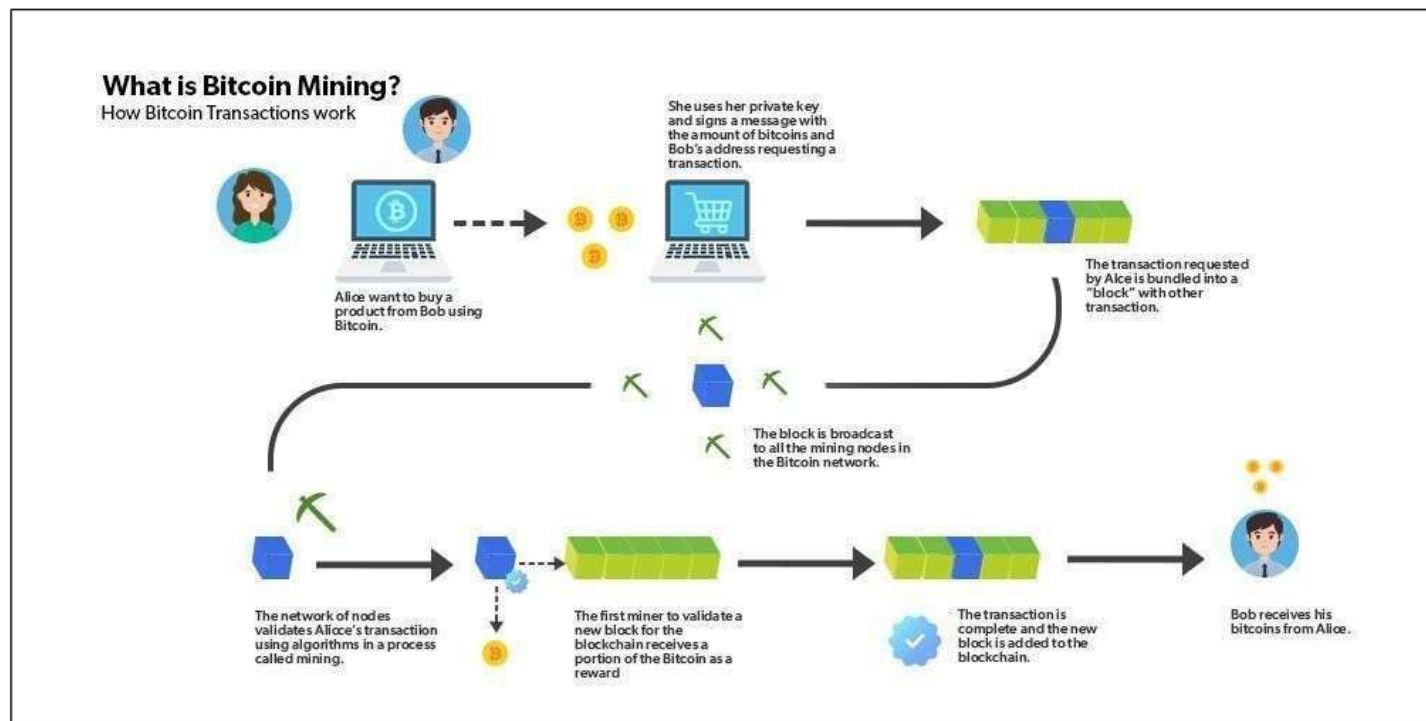
Mining is the process of validating transactions and adding new blocks to the blockchain. It is primarily associated with **Proof-of-Work (PoW)** consensus mechanisms (e.g., Bitcoin). Miners compete to solve a complex cryptographic puzzle, and the winner gets to add the block and receive a reward.

Key steps in the mining process:

1. **Transaction Collection** Unconfirmed transactions are collected from the network and stored in a **mempool** (memory pool). Miners select transactions (usually prioritizing those with higher fees) to include in a candidate block. A special **coinbase transaction** is added, which rewards the miner with new coins + transaction fees.

2. **Block Header Preparation** The miner constructs the block header, which includes:
 - Previous block hash
 - Merkle root (summary hash of all transactions)
 - Timestamp
 - Difficulty target
 - Nonce (a variable number starting from 0)
3. **Solving the Cryptographic Puzzle (Proof-of-Work)** Miners repeatedly hash the block header (using SHA-256 in Bitcoin) while changing the nonce value. The goal is to find a hash that is **less than or equal to the current difficulty target** (a very small number, often requiring the hash to start with many leading zeros). This is computationally intensive and probabilistic — miners try billions/trillions of nonces per second.
4. **Block Found** The first miner to find a valid nonce (producing a qualifying hash) has "solved" the puzzle. They broadcast the completed block to the network.
5. **Network Acceptance & Reward** Other nodes verify the block. If valid, it is added to their copy of the blockchain. The successful miner receives the **block reward** (e.g., currently 3.125 BTC as of recent halvings) + transaction fees. Difficulty adjusts periodically to keep average block time stable (~10 minutes in Bitcoin).

Mining secures the network by making it extremely expensive to alter past blocks (would require re-mining all subsequent blocks) and prevents double-spending.



4. Checking the validity of blocks in blockchain

Once a mined block is broadcast, every full node independently verifies it before accepting it into their blockchain copy. This decentralized validation is what makes blockchain trustless and secure.

Key validity checks:

1. **Block Structure & Syntax**
 - Correct format and size
 - Valid version number
 - Timestamp is reasonable (not too far in future or before previous block)
2. **Previous Block Hash Match** The "previous block hash" field must exactly match the hash of the most recent block already in the node's chain.

3. **Proof-of-Work Validation** Recompute the block hash → check if it is $\leq$ current difficulty target (i.e., has enough leading zeros). This confirms the miner did the required work.
4. **Merkle Root Verification** Rebuild the Merkle tree from all transactions in the block body → verify the computed Merkle root matches the one in the block header.
5. Transaction Validity
 - Each transaction has valid signatures
 - Inputs are unspent (no double-spending)
 - Input amounts $\geq$ output amounts + fees
 - Transactions follow protocol rules
6. **Consensus Rules Compliance** Block follows all network rules (max block size, valid coinbase, etc.).

If **any** check fails, the block is rejected and not added to the chain. Nodes continue waiting for a valid block. The longest valid chain (most cumulative work) is considered the true chain.

Light clients (SPV wallets) use simplified checks — they rely on **Merkle proofs** to confirm a transaction exists in a block without downloading everything.

Implementation:-

Installing Flask

```
PS C:\Users\Student\Downloads\BCEx2> pip install flask
Collecting flask
  Downloading flask-3.1.2-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Downloading jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
```

blockchain.py

import datetime

import hashlib

import json

class Blockchain: def

 __init__(self):

 self.chain = []

 self.difficulty = 4 # 4 leading zeroes

 self.create_genesis_block()

 def create_genesis_block(self):

 genesis_block = {

 'index': 1,

 'timestamp': str(datetime.datetime.now()), 'data':

 'Genesis Block',

 'nonce': 0,

 'previous_hash': '0'

```

    }
    genesis_block['hash'] = self.calculate_hash(genesis_block) self.chain.append(genesis_block)

def calculate_hash(self, block): block_copy
    = block.copy()
    block_copy.pop('hash', None) # Remove hash before recalculating

    encoded = json.dumps(block_copy, sort_keys=True).encode() return
    hashlib.sha256(encoded).hexdigest()

def mine_block(self, data): previous_block =
    self.chain[-1] nonce = 0

    while True:
        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()), 'data':
            data,
            'nonce': nonce,
            'previous_hash': previous_block['hash']
        }
        block_hash = self.calculate_hash(block) #
        Check difficulty condition
        if block_hash.startswith('0' * self.difficulty):
            block['hash'] = block_hash
            self.chain.append(block)
            return block

```

```
nonce += 1
```

```
def is_chain_valid(self):
```

```
    for i in range(1, len(self.chain)):
```

```
        current = self.chain[i]
        previous = self.chain[i - 1]
```

```
        # Check previous hash link
```

```
        if current['previous_hash'] != previous['hash']:
            return False
```

```
        # Recalculate and verify hash
```

```
        recalculated_hash = self.calculate_hash(current)
        if recalculated_hash != current['hash']:
            return False
```

```
        # Check difficulty rule
```

```
        if not current['hash'].startswith('0' * self.difficulty):
            return False
```

```
    return True
```

```
app.py
```

```
from flask import Flask, jsonify
```

```
from blockchain import Blockchain
```

```
app = Flask(__name__)
```

```
blockchain = Blockchain()
```

```
@app.route('/mine_block', methods=['GET'])
```

```
def mine_block():
```

```
block = blockchain.mine_block("Some transaction data") return  
jsonify({  
    'message': '⚡ Block mined successfully!',  
    'block': block  
}), 200
```

```
@app.route('/get_chain', methods=['GET']) def  
get_chain():  
    return jsonify({  
        'chain': blockchain.chain, 'length':  
        len(blockchain.chain)  
    }), 200
```

```
@app.route('/is_valid', methods=['GET']) def  
is_valid():  
    return jsonify({  
        'is_valid': blockchain.is_chain_valid()  
    }), 200
```

```
if __name__ == '__main__':  
    print("⚡ Mining blockchain with 4 leading zeroes...") app.run(debug=True,  
        use_reloader=False)
```

```
← 127.0.0.1:5000/mine_block
Pretty-print ☒
{
  "index": 2,
  "message": "Congratulations, you just mined a block!",
  "miner_name": "Atharva Prabhu",
  "previous_hash": "6939a9751f19b6253dcb5440a04529d79e2b339060bacb14c7a25bdf193cdcba",
  "proof": 533,
  "timestamp": "2026-02-05 20:06:50.232249"
}
```

```
← 127.0.0.1:5000/get_chain
Pretty-print ☒
{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-05 20:06:45.744430"
    },
    {
      "index": 2,
      "previous_hash": "6939a9751f19b6253dcb5440a04529d79e2b339060bacb14c7a25bdf193cdcba",
      "proof": 533,
      "timestamp": "2026-02-05 20:06:50.232249"
    }
  ],
  "length": 2
}
```



```
{
  "is_valid": true
}
```

Conclusion:-

In this experiment, a simple blockchain was successfully built using Python. It clearly illustrated the essential concepts of blockchain, such as block structure, cryptographic hashing, Proof-of-Work mining, and chain validation. Through hash-linked blocks and strict difficulty-based verification rules, the system guarantees data integrity, immutability, and strong resistance to tampering. Even a minor change in any block breaks the entire chain, perfectly showcasing the decentralized and secure foundation of blockchain technology.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 03

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1: Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2: Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3: Implement smart contracts in Ethereum using different development frameworks. CO5: Interpret different Crypto assets and Crypto currencies

EXPERIMENT 3

Name-Atharva Prabhu

Class-D20A

Roll no-45

Aim: Create a Cryptocurrency using Python and perform mining in the Blockchain created.

Theory:

1. Blockchain Overview

A blockchain is a distributed, decentralized digital ledger used to record transactions securely across multiple computers (nodes). Instead of relying on a central authority, every node in the network maintains its own copy of the blockchain.

Each block in the blockchain contains:

- A list of transactions
- A timestamp
- The hash of the previous block
- Its own cryptographic hash

The blocks are linked together using hashes, forming a chain. Any attempt to modify data in a block changes its hash, which breaks the chain and makes tampering easily detectable. This property ensures data integrity, transparency, and immutability.

2. Mining

Mining is the process by which new blocks are added to the blockchain. It involves:

- Collecting pending transactions
- Creating a block header
- Solving a computationally difficult problem known as Proof-of-Work (PoW)

In Proof-of-Work, miners repeatedly change a nonce value to generate a hash that satisfies the network's difficulty requirement (such as leading zeroes). Once a valid hash is found, the block is added to the blockchain and shared with other nodes. As an incentive, miners receive a cryptocurrency reward for their computational effort.

3. Multi-Node Blockchain Network

In this experiment, a multi-node blockchain network is simulated using three nodes running on different ports (5001, 5002, and 5003).

Each node:

- Operates independently
- Maintains its own copy of the blockchain
- Communicates with peer nodes to share newly mined blocks

This setup demonstrates how blockchain achieves decentralization and synchronization without a central server.

4. Consensus Mechanism

To ensure consistency across all nodes, the Longest Chain Rule is used as the consensus mechanism.

When different versions of the blockchain exist, the node accepts the longest valid chain as the correct one. This rule ensures that all nodes eventually agree on a single transaction history, even if temporary differences arise.

5. Transactions and Mining Reward

Transactions in the cryptocurrency system include:

- Sender address
- Receiver address
- Transaction amount

When a block is mined, all pending transactions are added to the block. Additionally, a special transaction is created to reward the miner with newly generated cryptocurrency. This reward mechanism motivates miners to maintain and secure the network.

6. Chain Replacement

The chain replacement process ensures network consistency. When the `/replace_chain` endpoint is triggered:

- A node requests blockchain data from its peers
- Validates the received chains
- Replaces its own chain if a longer and valid chain is found.

This mechanism helps resolve conflicts and keeps all nodes synchronized.

Code:

Module 2 - Create a Cryptocurrency

To be installed:

Flask==0.12.2: pip install Flask==0.12.2

Postman HTTP Client: <https://www.getpostman.com/>

```
# requests==2.18.4: pip install requests==2.18.4
```

```
# Importing the libraries
```

```
import datetime
```

```
import hashlib
```

```
import json
```

```
from flask import Flask, jsonify, request
```

```
import requests
```

```
from uuid import uuid4
```

```
# Generate a unique id that is in
```

```
hex from urllib.parse import urlparse
```

```
# To parse url of the nodes
```

```
# Part 1 - Building a Blockchain
```

```
class Blockchain:
```

```
    def __init__(self): self.chain
```

```
        = []
```

```
        self.transactions = []
```

```
# Adding transactions before they are
```

```
added to a block
```

```
        self.create_block(proof = 1, previous_hash = '0')
```

```
        self.nodes = set()
```

```
# Set is used as there is no order to be
```

```
maintained as the nodes can be from all around the globe
```

```
    def create_block(self, proof, previous_hash):
```

```
        block = {'index': len(self.chain) + 1,
```

```
                  'timestamp': str(datetime.datetime.now()),
```

```
                  'proof': proof,
```

```
                  'previous_hash': previous_hash,
```

```
                  'transactions': self.transactions}
```

```
# Adding transactions to make
```

```
the blockchain a cryptocurrency
```

```
        self.transactions = []
```

```
# The list of transaction should become
```

```
empty after they are added to a block
```

```
        self.chain.append(block)
```

```
        return block
```

```
    def get_previous_block(self):
```

```
        return self.chain[-1]
```

```
    def proof_of_work(self,
```

```
        previous_proof): new_proof = 1
```

```

    check_proof = False
    while check_proof is False:
        hash_operation = hashlib.sha256(str(new_proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] == '0000':
            check_proof = True
        else:
            new_proof += 1
    return new_proof

def hash(self, block):
    encoded_block = json.dumps(block, sort_keys = True).encode()
    return hashlib.sha256(encoded_block).hexdigest()

def is_chain_valid(self, chain):
    previous_block = chain[0]
    block_index = 1
    while block_index < len(chain):
        block = chain[block_index]
        if block['previous_hash'] != self.hash(previous_block):
            return False
        previous_proof = previous_block['proof']
        proof = block['proof']
        hash_operation = hashlib.sha256(str(proof**2 -
previous_proof**2).encode()).hexdigest()
        if hash_operation[:4] != '0000':
            return False
        previous_block = block
        block_index += 1
    return True

# This method will add the transaction to the list of transactions
def add_transaction(self, sender, receiver, amount):
    self.transactions.append({'sender': sender,
                              'receiver': receiver,
                              'amount': amount})
    previous_block = self.get_previous_block()
    return previous_block['index'] + 1
which the transaction should be added
# It will return the block index to

```

```

# This function will add the node containing an address to the set of nodes created in init
function
def add_node(self, address):
    parsed_url = urlparse(address)          # urlparse will parse the url
    from the address
    self.nodes.add(parsed_url.netloc)        # Add is used and not append as
    it's a set. Netloc will only return '127.0.0.1:5000'

# Consensus Protocol. This function will replace all the shorter chain with the longer chain
in all the nodes on the network
def replace_chain(self):
    network = self.nodes                    # network variable is the set of nodes
    all around the globe
    longest_chain = None                    # It will hold the longest chain
    when we scan the network
    max_length = len(self.chain)            # This will hold the length of the
    chain held by the node that runs this function
    for node in network:
        response = requests.get(f'http://{node}/get_chain')    # Use get chain
        method already created to get the length of the chain
        if response.status_code == 200:
            length = response.json()['length']                  # Extract the length of the
            chain from get_chain function
            chain = response.json()['chain']
            if length > max_length and self.is_chain_valid(chain):    # We check if the length
            is bigger and if the chain is valid then
                max_length = length                                    # We update the max length
                longest_chain = chain                                  # We update the longest chain
            if longest_chain:    # If longest_chain is not none that means
            it was replaced
                self.chain = longest_chain                            # Replace the chain of the current
            node with the longest chain
            return True
            return False                                              # Return false if current chain is the
            longest one
# Part 2 - Mining our Blockchain

# Creating a Web App

app = Flask(__name__)

```

```

# Creating an address for the node on Port 5000. We will create some other nodes as well on
different ports
node_address = str(uuid4()).replace('-', '') #

# Creating a Blockchain
blockchain = Blockchain()

# Mining a new block
@app.route('/mine_block', methods = ['GET'])
def mine_block():
    previous_block = blockchain.get_previous_block()
    previous_proof = previous_block['proof']
    proof = blockchain.proof_of_work(previous_proof)
    previous_hash = blockchain.hash(previous_block)
    blockchain.add_transaction(sender = node_address, receiver = 'Abcde', amount = 1) #
    Hadcoins to mine the block (A Reward). So the node gives 1 hadcoin to Abcde for mining the
    block
    block = blockchain.create_block(proof, previous_hash)
    response = {'message': 'Congratulations, you just mined a block!',
                'index': block['index'],
                'timestamp': block['timestamp'],
                'proof': block['proof'],
                'previous_hash':
                    block['previous_hash'], 'transactions':
                    block['transactions']}
    return jsonify(response), 200

# Getting the full Blockchain
@app.route('/get_chain', methods = ['GET'])
def get_chain():
    response = {'chain': blockchain.chain,
                'length': len(blockchain.chain)}
    return jsonify(response), 200

# Checking if the Blockchain is valid
@app.route('/is_valid', methods = ['GET'])
def is_valid():
    is_valid = blockchain.is_chain_valid(blockchain.chain)
    if is_valid:
        response = {'message': 'All good. The Blockchain is valid.'}

```

```

else:
    response = {'message': 'Houston, we have a problem. The Blockchain is not valid.'}
    return jsonify(response), 200

```

Adding a new transaction to the Blockchain

```

@app.route('/add_transaction', methods = ['POST'])
to pass something to get something in return

```

Post method as we have

```

def add_transaction():

```

```

    json = request.get_json()
    postman. In Postman we will create a json file in which we will pass the values for the keys in the json file

```

```

    transaction_keys = ['sender', 'receiver', 'amount']

```

```

    if not all(key in json for key in transaction_keys):
        available in json

```

Checking if all keys are

```

        return 'Some elements of the transaction are missing', 400

```

```

    index = blockchain.add_transaction(json['sender'], json['receiver'], json['amount'])

```

```

    response = {'message': f'This transaction will be added to Block {index}'}

```

```

    return jsonify(response), 201

```

Code 201 for creation

Part 3 - Decentralizing our Blockchain

Connecting new nodes

```

@app.route('/connect_node', methods = ['POST'])

```

POST request to register

the new nodes from the json file

```

def connect_node():

```

```

    json = request.get_json()

```

```

    nodes = json.get('nodes')

```

Get the nodes from json file

```

    if nodes is None:

```

```

        return "No node", 400

```

```

    for node in nodes:

```

```

        blockchain.add_node(node)

```

```

    response = {'message': 'All the nodes are now connected. The Hadcoin Blockchain now contains the following nodes:'}

```

```

        'total_nodes': list(blockchain.nodes)}

```

```

    return jsonify(response), 201

```

Replacing the chain by the longest chain if

needed @app.route('/replace_chain', methods =

```

['GET']) def replace_chain():

```

```

    is_chain_replaced = blockchain.replace_chain()

```

```

    if is_chain_replaced:
        response = {'message': 'The nodes had different chains so the chain was replaced by the
longest one.',
                    'new_chain': blockchain.chain}
    else:
        response = {'message': 'All good. The chain is the largest one.',
                    'actual_chain': blockchain.chain}
    return jsonify(response), 200

# Running the app
app.run(host = '0.0.0.0', port = 5003)

```

Output:

1)/get_chain

This GET request retrieves the complete blockchain from the node, showing all mined blocks and stored transactions.

The screenshot shows a REST client interface with a GET request to `http://127.0.0.1:5001/get_chain`. The response is a JSON object with a status of 200 OK. The JSON body is as follows:

```

{
  "chain": [
    {
      "index": 1,
      "previous_hash": "0",
      "proof": 1,
      "timestamp": "2026-02-11 23:46:01.894114",
      "transactions": []
    }
  ],
  "length": 1
}

```

2)/mine_block

This GET request performs the mining process using Proof-of-Work and adds a new block to the blockchain.

The screenshot shows a web browser interface for an HTTP client. The address bar displays the URL `http://127.0.0.1:5001/mine_block`. Below the address bar, the method is set to `GET` and the same URL is entered in the text field. A `Send` button is visible. The response is displayed in the `Body` tab, showing a JSON object with the following structure:

```
1 {
2   "index": 2,
3   "message": "Congratulations, you just mined a block!",
4   "previous_hash": "3b86012a3e3a9cde4177b3957ba888f4826784a8b160f6c4e948b411980e83d6",
5   "proof": 533,
6   "timestamp": "2026-03-16 19:57:08.923105",
7   "transactions": [
8     {
9       "amount": 5000,
10      "receiver": "Prabhu",
11      "sender": "Atharva"
12    },
13    {
14      "amount": 5000,
15      "receiver": "Danish",
16      "sender": "Samuel"
17    }
18  ]
19 }
```

The response status is `200 OK` with a response time of `28 ms` and a body size of `571 B`. A `Save Response` button is also present.

GET https://api.coingecko.com [CONFLICT] GET https://api.co GET http://127.0.0.1:5001/get_



http://127.0.0.1:5001/get_chain

Save

GET



http://127.0.0.1:5001/get_chain

Send



Params Auth Headers (8) Body • Pre-req. Tests Settings

Cookies

Body



200 OK

16 ms

642 B

Save Response



Pretty

Raw

Preview

Visualize

JSON



```
1 {
2   "chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-03-16 19:45:22.304334",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash":
13        "3b86012a3e3a9cde4177b3957ba888f4826784a8b160f6c4e948b411980e83d6",
14      "proof": 533,
15      "timestamp": "2026-03-16 19:57:08.923105",
16      "transactions": [
17        {
18          "index": 1,
19          "previous_hash": "0",
20          "proof": 1,
21          "timestamp": "2026-03-16 19:57:08.923105",
22          "transactions": []
23        }
24      ]
25    }
26  ]
27 }
```

3)/is_valid

This GET request checks whether the current blockchain is valid and untampered.

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/is_valid` and the method `GET`. The response status is `200 OK` with a response time of 74 ms and a body size of 215 B. The response body is displayed in JSON format:

```
{  "message": "All good. The Blockchain is valid."}
```

4)/add_transaction

This POST request submits a new transaction (sender, receiver, amount) to be added to the next mined block.

The screenshot shows a REST client interface with the URL `http://127.0.0.1:5001/add_transaction` and the method `POST`. The response status is `201 CREATED` with a response time of 39 ms and a body size of 226 B. The response body is displayed in JSON format:

```
{  "message": "This transaction will be added to Block 2"}
```

5)/connect_node

This POST request connects the current node with other peer nodes in the blockchain network.

POST http://127.0.0.1:5001/connect_node

Body

```
1 {
2   "nodes": [
3     "http://127.0.0.1:5000",
4     "http://127.0.0.1:5002"
5   ]
6 }
```

201 CREATED • 121 ms • 326 B •

Body

```
1 {
2   "message": "All the nodes are now connected. The Hadcoin Blockchain now contains the following
3     nodes:",
4   "total_nodes": [
5     "127.0.0.1:5000",
6     "127.0.0.1:5002"
7   ]
8 }
```

6)/replace_chain

This GET request checks all connected peer nodes and replaces the current blockchain with the longest valid chain, ensuring network-wide consistency.

http://127.0.0.1:5001/replace_chain

GET http://127.0.0.1:5001/replace_chain

200 OK • 8.38 s • 582 B •

Body

```
1 {
2   "actual_chain": [
3     {
4       "index": 1,
5       "previous_hash": "0",
6       "proof": 1,
7       "timestamp": "2026-02-11 23:46:01.894114",
8       "transactions": []
9     },
10    {
11      "index": 2,
12      "previous_hash": "9b001b6cb4216e67488cc58675d577e0115096c8616d710a286c411509a7d2a7",
13      "proof": 533,
14    }
15  ]
16 }
```

```
{
  "transactions": [
    {
      "amount": 1,
      "receiver": "Richard",
      "sender": "7f8423ce398f4f5eb2a8ca22514d2b9c"
    }
  ],
  "message": "All good. The chain is the largest one."
}
```

Conclusion:

In this experiment, a basic cryptocurrency system was designed and implemented using Python by applying fundamental blockchain principles such as block generation, cryptographic hashing, Proof-of-Work mining, transaction processing, and decentralized networking. A Flask-based framework was used to enable communication between multiple nodes, with each node maintaining its own copy of the blockchain ledger.

The mining process required solving the Proof-of-Work challenge, after which new blocks were added to the chain and miners received rewards. The system also supported transaction validation and inclusion of verified transactions into newly mined blocks. Through this implementation, the experiment offered hands-on insight into the functioning of blockchain technology, demonstrating how consensus, decentralization, and mining together enable the secure operation of cryptocurrencies in distributed environments.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 04

Aim: Hands-on Solidity Programming Assignments for creating Smart Contracts

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

Experiment No. 4

Name: Atharva Prabhu

Batch: B

Div: D20A

Roll No: 45

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables, Functions – pure, view

In Solidity, primitive data types form the foundation of smart contract development. Commonly used types include:

- **uint / int:** unsigned and signed integers of different sizes (e.g., uint256, int128).
- **bool:** represents logical values (true or false).
- **address:** holds a 20-byte Ethereum account address, often used for storing user accounts or contract addresses.
- **bytes / string:** store binary data or textual data.

Variables in Solidity can be **state variables** (stored on the blockchain permanently), **local variables** (temporary, created during function execution), or **global variables** (special predefined variables such as msg.sender, msg.value, and block.timestamp).

Functions allow execution of contract logic. Special types of functions include:

- **pure:** cannot read or modify blockchain state; they work only with inputs and internal computations.
- **view:** can read state variables but cannot alter them. This classification helps optimize gas usage and enforces function integrity.

2. Inputs and Outputs to Functions

Functions in Solidity can accept input arguments and return one or more output values. Inputs enable users or other contracts to pass data into the contract, while outputs make it possible to return results after computation. For example, a function can accept an amount in Ether and return whether the transfer was successful. Solidity also allows named return variables, which improve readability and debugging.

3. Visibility, Modifiers and Constructors

- **Function Visibility** defines who can access a function:
 - **public:** available both inside and outside the contract.
 - **private:** only accessible within the same contract.

- o internal: accessible within the contract and its child contracts.
- o external: can be called only by external accounts or other contract
- **Modifiers** are reusable code blocks that change the behavior of functions. They are often used for access control, such as restricting sensitive functions to the contract owner (onlyOwner).
- **Constructors** are special functions executed only once during contract deployment. They initialize important values, such as setting the deploying account as the owner of the contract.

3. Control Flow: if-else, loops

Control flow in Solidity is similar to traditional programming languages:

- **if-else** allows conditional decision-making in contract logic, e.g., checking if a balance is sufficient before transferring funds.
- **Loops** (for, while, do-while) enable repeated execution of code. For example, iterating through an array of users. However, loops must be used carefully, as excessive iterations increase gas consumption, potentially making the contract expensive to execute.

5. Data Structures: Arrays, Mappings, Structs, Enums

- **Arrays**: Can be fixed or dynamic and are used to store ordered lists of elements. Example: an array of addresses for registered users.
- **Mappings**: Key-value pairs that allow quick lookups. Example: mapping(address => uint) for storing balances. Unlike arrays, mappings do not support iteration.
- **Structs**: Allow grouping of related properties into a single data type, such as creating a struct Player {string name; uint score;}.
- **Enums**: Used to define a set of predefined constants, making code more readable. Example: enum Status { Pending, Active, Closed }.

6. Data Locations

Solidity uses three primary data locations for storing variables:

- **storage:** Data stored permanently on the blockchain. Examples: state variables.
- **memory:** Temporary data storage that exists only while a function is executing. Used for local variables and function inputs.
- **calldata:** A non-modifiable and non-persistent location used for external function parameters. It is gas-efficient compared to memory. Understanding data locations is essential, as they directly impact gas costs and performance.

7. Transactions: Ether and Wei, Gas and Gas Price, Sending Transactions

- **Ether and Wei:** Ether is the main currency in Ethereum. All values are measured in Wei, the smallest unit (1 Ether = 10^{18} Wei). This ensures high precision in financial transactions.
- **Gas and Gas Price:** Every transaction consumes gas, which represents computational effort. The gas price determines how much Ether is paid per unit of gas. A higher gas price incentivizes miners to prioritize the transaction.
- **Sending Transactions:** Transactions are used for transferring Ether or interacting with contracts. Functions like `transfer()` and `send()` are commonly used, while `call()` provides more flexibility. Each transaction requires gas, making efficiency in contract design very important.

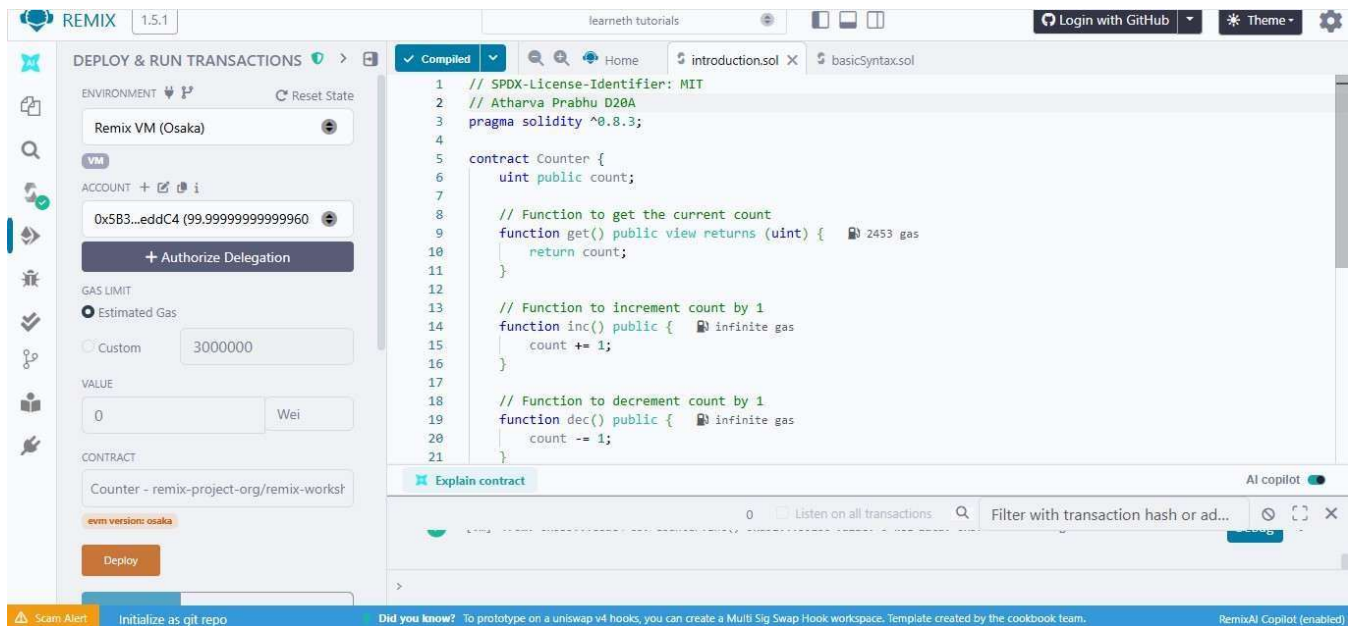
Implementation:

- Tutorial no. 1 – Compile the code

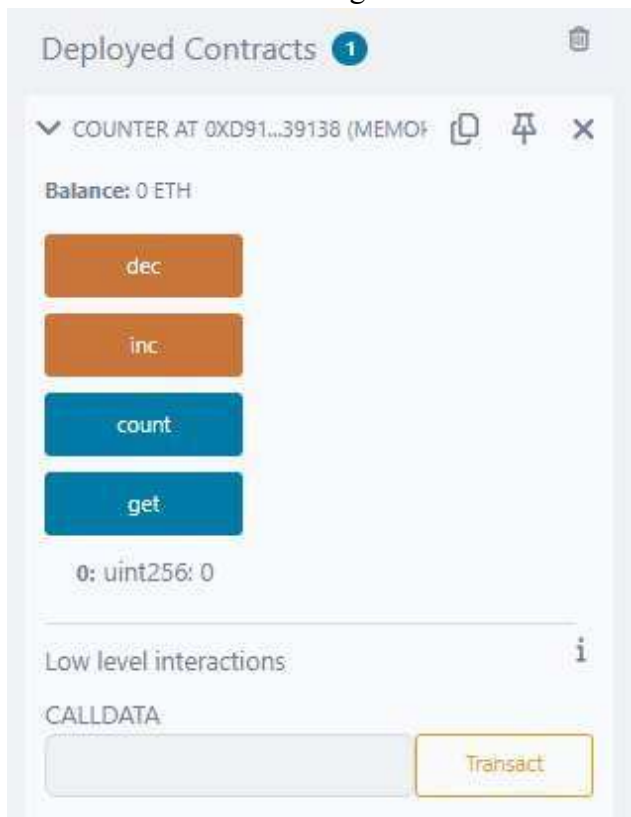
The screenshot displays the Remix IDE interface. On the left, a sidebar contains a 'Tutorials list' with '1. Introduction' selected. The main panel shows the tutorial content, which includes a welcome message and an explanation of the counter contract's functionality. The right panel is a code editor showing the Solidity code for the 'Counter' contract. The code includes a license identifier, author information, a Solidity version pragma, and three functions: 'get', 'inc', and 'dec'. The bottom status bar shows a 'Scam Alert' and a 'Did you know?' message about uniswap v4 hooks.

```
1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Counter {
6     uint public count;
7
8     // Function to get the current count
9     function get() public view returns (uint) { 2453 gas
10         return count;
11     }
12
13     // Function to increment count by 1
14     function inc() public { infinite gas
15         count += 1;
16     }
17
18     // Function to decrement count by 1
19     function dec() public { infinite gas
20         count -= 1;
21     }
22 }
```

- Tutorial no. 1 – Deploy the contract



- Tutorial no. 1 – get



- Tutorial no. 1 – Increment

Deployed Contracts 1

▼ COUNTER AT 0XD91...39138 (MEMO)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 3

Low level interactions i

CALLDATA

Transact

- Tutorial no. 1 – Decrement

Deployed Contracts 1

▼ COUNTER AT 0XD91...39138 (MEMO)

Balance: 0 ETH

dec

inc

count

get

0: uint256: 2

Low level interactions i

CALLDATA

Transact

• Tutorial no. 2

REMUX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH Tutorials list Syllabus

2. Basic Syntax 2 / 19

Don't worry if you didn't understand some concepts like *visibility*, *data types*, or *state variables*. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

Watch a video tutorial on Basic Syntax.

★ Assignment

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 // compiler version must be greater than or equal to 0.8.3 and less than 0.9.0
3 // Atharva Prabhu D20A
4 pragma solidity ^0.8.3;
5
6 contract MyContract {
7     // We declare a public string variable named 'name'
8     // and assign it the value "Alice"
9     string public name = "Alice";
10 }
```

• Tutorial no. 3

REMUX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH Tutorials list Syllabus

3. Primitive Data Types 3 / 19

and Structs

Watch a video tutorial on Primitive Data Types.

★ Assignment

1. Create a new variable `newAddr` that is a `public` `address` and give it a value that is not the same as the available variable `addr`.
2. Create a `public` variable called `neg` that is a negative number, decide upon the type.
3. Create a new variable, `newU` that has the smallest `uint` size type and the smallest `uint` value and is `public`.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Primitives {
6     string public name = "Alice";
7
8     // 1. A public address variable with a unique value
9     address public newAddr = 0xAb843F64d9C6d1EcF9b849Ae677dD3315835cb2;
10
11     // 2. A signed integer to hold a negative number
12     // int8 is sufficient for small negative numbers and saves space
13     int8 public neg = -50;
14
15     // 3. The smallest uint size (uint8) and smallest value (0)
16     uint8 public newU = 0;
17 }
```

• Tutorial no. 4

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

4. Variables 4 / 19

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the [Solidity documentation](#).

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Variable {
6     uint256 public blockNumber;
7     string public name;
8     uint public n;
9
10    uint public blockNumber;
11    function doSomething() public { 22334 gas
12        // Local variable (Exists only during function execution)
13        uint i = 456;
14
15        // 2. Use a Global Variable to get the current block number
16        // 'block.number' is the global variable you're looking for!
17        blockNumber = block.number;
18
19        // Other examples of global variables:
20        uint timestamp = block.timestamp; // Current block timestamp
21        address sender = msg.sender;      // Address of the person calling this function
22    }
23 }

```

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 5

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

5.1 Functions - Reading and Writing to a State Variable 5 / 19

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

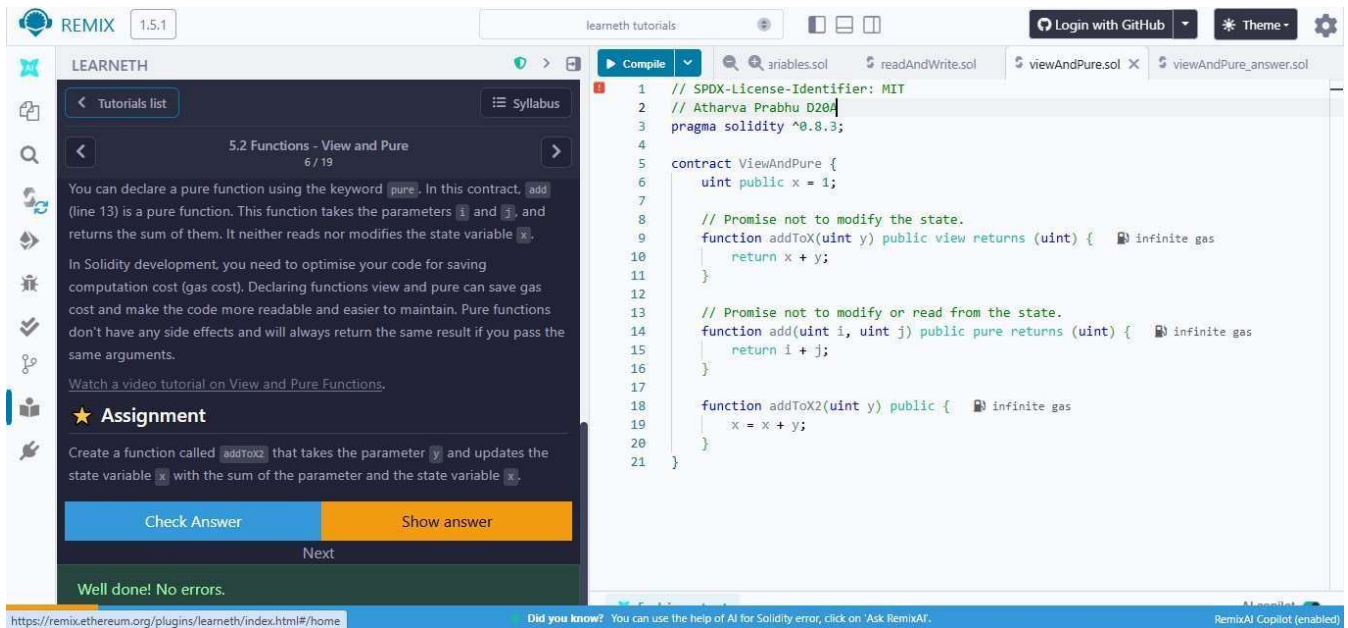
```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract SimpleStorage {
6     // State variable to store a number
7     uint public num;
8
9     // 1. Create a public state variable 'b' initialized to true
10    bool public b = true;
11
12    // Function that changes the state (Writing)
13    function set(uint _num) public { 22536 gas
14        num = _num;
15    }
16
17    // 2. Create a public function 'get_b' that returns the value of 'b'
18    // We use 'view' because we are reading, not changing, the state.
19    function get_b() public view returns (bool) { 2517 gas
20        return b;
21    }
22 }

```

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 6



LEARNETH

5.2 Functions - View and Pure
6 / 19

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

★ **Assignment**

Create a function called `addToX` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

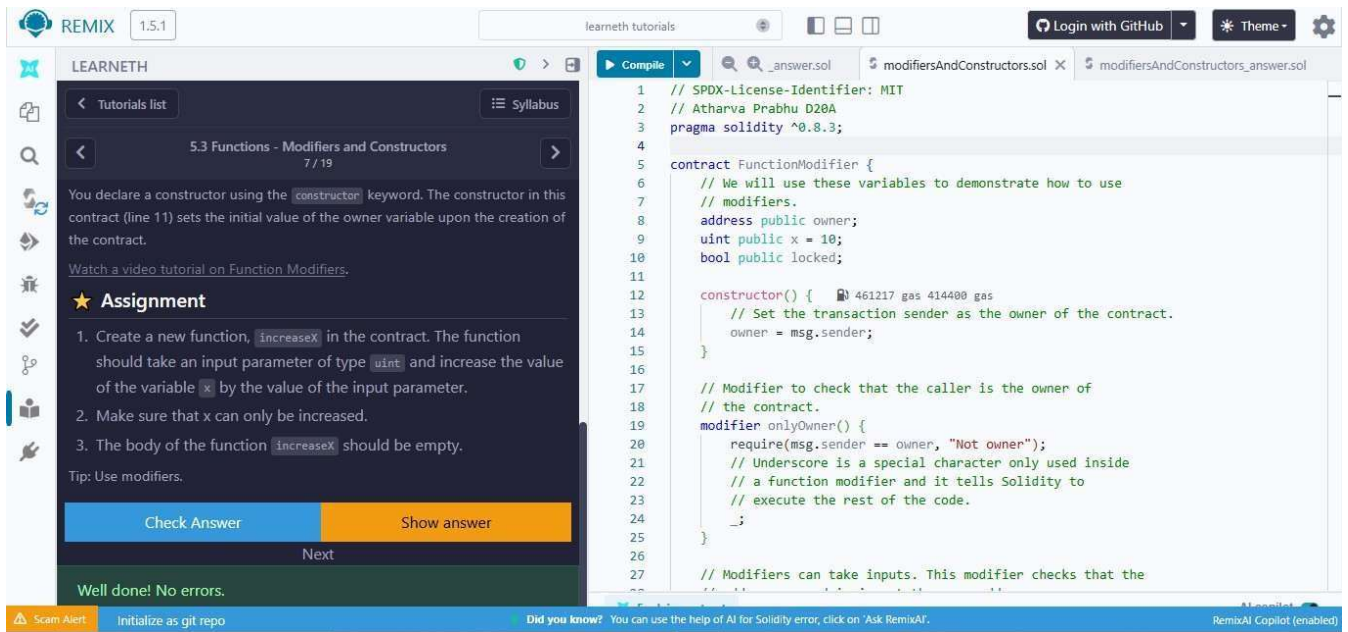
1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D204
3 pragma solidity ^0.8.3;
4
5 contract ViewAndPure {
6     uint public x = 1;
7
8     // Promise not to modify the state.
9     function addToX(uint y) public view returns (uint) { infinite gas
10         return x + y;
11     }
12
13     // Promise not to modify or read from the state.
14     function add(uint i, uint j) public pure returns (uint) { infinite gas
15         return i + j;
16     }
17
18     function addToX2(uint y) public { infinite gas
19         x = x + y;
20     }
21 }

```

Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

RemixAI Copilot (enabled)

• Tutorial no. 7



LEARNETH

5.3 Functions - Modifiers and Constructors
7 / 19

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ **Assignment**

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D204
3 pragma solidity ^0.8.3;
4
5 contract FunctionModifier {
6     // We will use these variables to demonstrate how to use
7     // modifiers.
8     address public owner;
9     uint public x = 10;
10    bool public locked;
11
12    constructor() { 461217 gas 414480 gas
13        // Set the transaction sender as the owner of the contract.
14        owner = msg.sender;
15    }
16
17    // Modifier to check that the caller is the owner of
18    // the contract.
19    modifier onlyOwner() {
20        require(msg.sender == owner, "Not owner");
21        // Underscore is a special character only used inside
22        // a function modifier and it tells Solidity to
23        // execute the rest of the code.
24        _;
25    }
26
27    // Modifiers can take inputs. This modifier checks that the

```

Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'.

RemixAI Copilot (enabled)

• Tutorial no. 8

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

Tutorials list 5.4 Functions - Inputs and Outputs 8 / 19

functions that are publicly visible. From the [Solidity documentation](#).

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

Solidity unit testing cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

★ Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Function {
6     // Functions can return multiple values.
7     function returnMany() infinite gas
8     {
9         public
10        pure
11        returns (
12            uint,
13            bool,
14            uint
15        )
16    {
17        return (1, true, 2);
18    }
19
20    // Return values can be named.
21    function named() infinite gas
22    {
23        public
24        pure
25        returns (
26            uint x,
27            bool b,
28            uint y
29        )
30    }
31 }

```

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

• Tutorial no. 9

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

Tutorials list 6. Visibility 9 / 19

6. Visibility

The `visibility` specifier is used to control who has access to functions and state variables.

There are four types of visibilities: `external`, `public`, `internal`, and `private`.

They regulate if functions and state variables can be called from inside the contract, from contracts that derive from the contract (child contracts), or from other contracts and transactions.

private

- Can be called from inside the contract

internal

- Can be called from inside the contract
- Can be called from a child contract

public

- Can be called from inside the contract

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Base {
6     // Private function can only be called
7     // - inside this contract
8     // Contracts that inherit this contract cannot call this function.
9     function privateFunc() private pure returns (string memory) { infinite gas
10         return "private function called";
11     }
12
13     function testPrivateFunc() public pure returns (string memory) { infinite gas
14         return
15     }
16
17     // Internal
18     // - inside
19     // - inside contracts that inherit this contract
20     function internalFunc() internal pure returns (string memory) { infinite gas
21         return "internal function called";
22     }
23
24     function testInternalFunc() public pure virtual returns (string memory) { infinite
25         return internalFunc();
26     }
27 }

```

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 10

LEARNETH

Tutorials list

7.1 Control Flow - If/Else

10 / 19

watch a video tutorial on the if/else statement.

★ Assignment

Create a new function called `evenCheck` in the `ifElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.0;
4
5 contract IfElse {
6
7     // Original example function for context
8     function foo(uint x) public pure returns (uint) {
9         if (x < 10) {
10             return 0;
11         } else if (x < 20) {
12             return 1;
13         } else {
14             return 2;
15         }
16     }
17
18     // ★ Assignment Solution
19     function evenCheck(uint _number) public pure returns (bool) {
20         // We use the modulo operator (%) to find the remainder.
21         // If _number % 2 is 0, it's even.
22         return _number % 2 == 0 ? true : false;
23     }
24 }

```

• Tutorial no. 11

LEARNETH

Tutorials list

7.2 Control Flow - Loops

11 / 19

present the break statement (you try) using exercises.

break

The `break` statement is used to exit a loop. In this contract, the `break` statement (line 14) will cause the `for` loop to be terminated after the sixth iteration. Watch a video tutorial on Loop statements.

★ Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the `for` loop, increment the `count` variable by 1.
- Try to get the `count` variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Loop {
6     // 1. Create a public uint state variable called count
7     uint public count;
8
9     function loop() public {
10         // To get count to 9, we need the loop to run 9 times.
11         // If there is a 'break' at i == 9, the loop stops before i reaches 10.
12         for (uint i = 0; i < 10; i++) {
13             if (i == 9) {
14                 break; // Stop the loop when i is 9
15             }
16
17             // 2. Increment count by 1 at the end of the for loop logic
18             count++;
19         }
20     }
21 }

```

• Tutorial no. 12

LEARNETH

8.1 Data Structures - Arrays
12 / 19

choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.0;
4
5 contract Array {
6     //
7     // contract Array is Array
8     //
9     uint[] arr3;
10
11     // Note: We use "pure" if we weren't reading state,
12     // but since we are reading arr3, we use "view".
13     function getArr() public view returns (uint[3] memory) {
14         return arr3;
15     }
16 }

```

Scam Alert Initialize as git repo Did you know? To prototype on a uniswap v4 hooks, you can create a Multi Sig Swap Hook workspace. Template created by the cookbook team. RemixAI Copilot (enabled)

• Tutorial no. 13

LEARNETH

8.2 Data Structures - Mappings
13 / 19

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

★ Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping `balances`.
3. Change the function `set` to create a new entry to the `balances` mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Mapping {
6     // Mapping from address to uint
7     mapping(address => uint) public balances;
8
9     function get(address _addr) public view returns (uint) {
10         // Mapping always returns a value.
11         // If the value was never set, it will return the default value.
12         return balances[_addr];
13     }
14
15     function set(address _addr) public {
16         // Update the value at this address
17         balances[_addr] = _addr.balance;
18     }
19
20     function remove(address _addr) public {
21         // Reset the value to the default value.
22         delete balances[_addr];
23     }
24 }
25
26 contract NestedMapping {
27     // Nested mapping (mapping from address to another mapping)

```

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

• Tutorial no. 14

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

Tutorials list **Syllabus**

8.3 Data Structures - Structs 14 / 19

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

[Watch a video tutorial on Structs.](#)

★ **Assignment**

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Todos {
6     struct Todo {
7         string text;
8         bool completed;
9     }
10
11     // An array of 'Todo' structs
12     Todo[] public todos;
13
14     function create(string memory _text) public { infinite gas
15         // 3 ways to initialize a struct
16         // - calling it like a function
17         todos.push(Todo(_text, false));
18
19         // key value mapping
20         todos.push(Todo({text: _text, completed: false}));
21
22         // initialize an empty struct and then update it
23         Todo memory todo;
24         todo.text = _text;
25         // todo.completed initialized to false
26
27         todos.push(todo);
28     }
29 }

```

• Tutorial no. 15

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

Tutorials list **Syllabus**

8.4 Data Structures - Enums 15 / 19

Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

[Watch a video tutorial on Enums.](#)

★ **Assignment**

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Enum {
6     // Enum representing shipping status
7     enum Status {
8         Pending,
9         Shipped,
10        Accepted,
11        Rejected,
12        Canceled
13    }
14
15     enum Size {
16         S,
17         M,
18         L
19     }
20
21     // Default value is the first element listed in
22     // definition of the type, in this case "Pending"
23     Status public status;
24     Size public sizes;
25
26     function get() public view returns (Status) { 2605 gas
27         return status;
28     }
29 }

```

- Tutorial no. 16

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

9. Data Locations 16 / 19

★ **Assignment**

1. Change the value of the `myStruct` member `foo`, inside the function `f`, to 4.
2. Create a new struct `myMemStruct2` with the data location `memory` inside the function `f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location `memory` inside the function `f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract DataLocations {
6     uint[] public arr;
7     mapping(uint => address) map;
8     struct MyStruct {
9         uint foo;
10    }
11    mapping(uint => MyStruct) public myStructs;
12
13    function f() public returns (MyStruct memory, MyStruct memory, MyStruct memory){
14        // call f with state variables
15        _f(arr, map, myStructs[1]);
16        // get a struct from a mapping
17        MyStruct storage myStruct = myStructs[1];
18        myStruct.foo = 4;
19        // create a struct in memory
20        MyStruct memory myMemStruct = MyStruct(0);
21        MyStruct memory myMemStruct2 = myMemStruct;
22        myMemStruct2.foo = 1;
23
24        MyStruct memory myMemStruct3 = myStruct;
25        myMemStruct3.foo = 3;
26        return (myStruct, myMemStruct2, myMemStruct3);
27    }
28 }

```

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

- Tutorial no. 17

LEARNETH 1.5.1 learneth tutorials Login with GitHub Theme

10.1 Transactions - Ether and Wei 17 / 19

Lower numbers without a suffix are treated as `wei` (unit `f`).

One `wei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ **Assignment**

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `wei` and `ether` in the contract.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

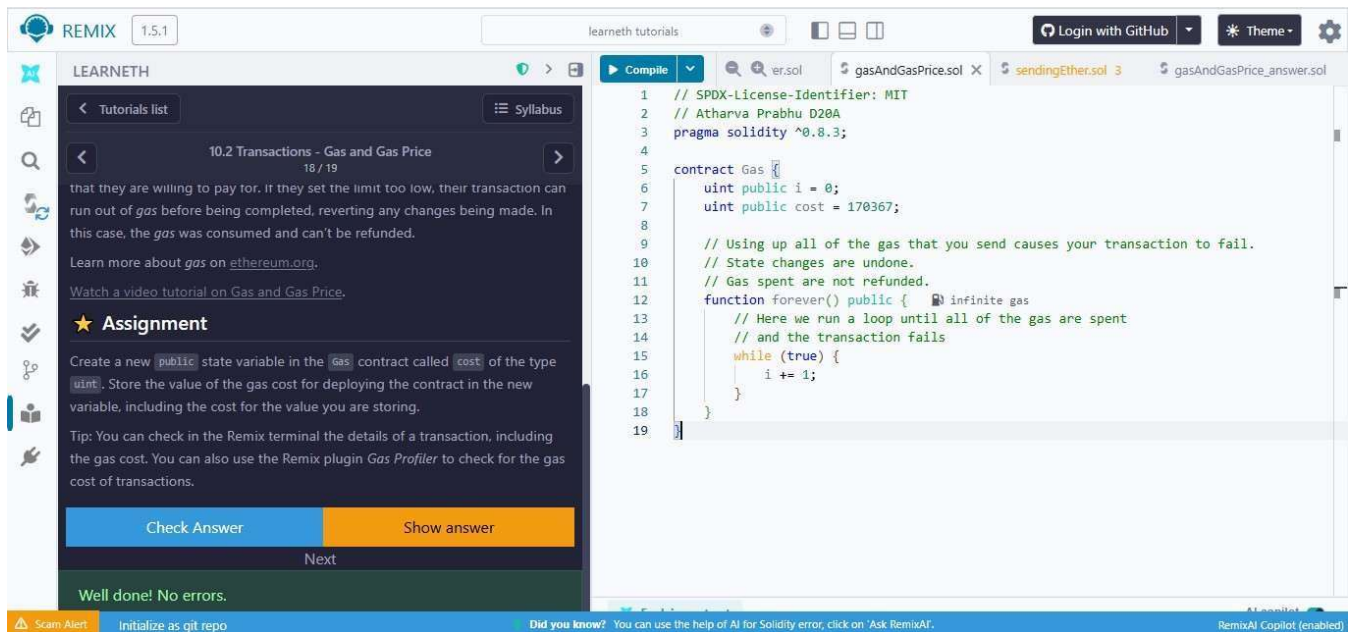
```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract EtherUnits {
6     uint public oneWei = 1 wei;
7     // 1 wei is equal to 1
8     bool public isOneWei = 1 wei == 1;
9
10    uint public oneEther = 1 ether;
11    // 1 ether is equal to 10^18 wei
12    bool public isOneEther = 1 ether == 1e18;
13
14    uint public oneGwei = 1 gwei;
15    // 1 ether is equal to 10^9 wei
16    bool public isOneGwei = 1 gwei == 1e9;
17 }

```

Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

- Tutorial no. 18



REMUX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH Tutorials list Syllabus

10.2 Transactions - Gas and Gas Price 18 / 19

that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

★ Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer Show answer

Next

Well done! No errors.

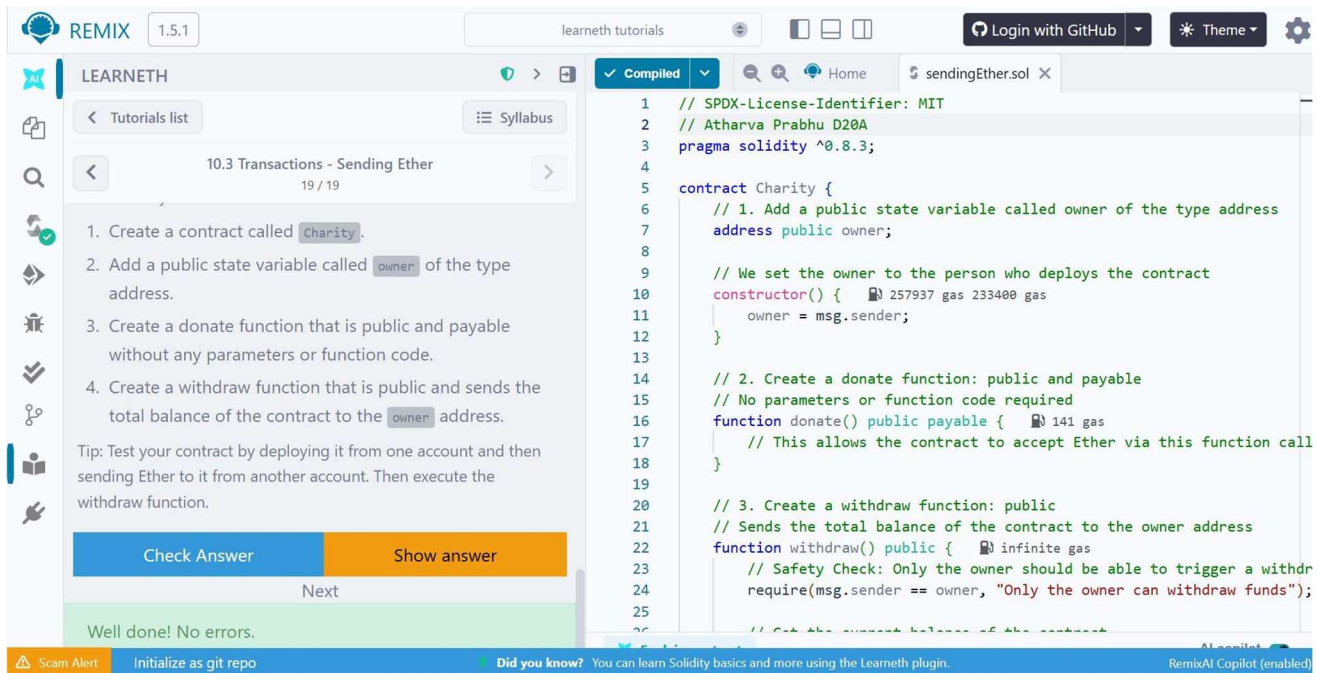
Scam Alert Initialize as git repo Did you know? You can use the help of AI for Solidity error, click on 'Ask RemixAI'. RemixAI Copilot (enabled)

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Gas {
6     uint public i = 0;
7     uint public cost = 170367;
8
9     // Using up all of the gas that you send causes your transaction to fail.
10    // State changes are undone.
11    // Gas spent are not refunded.
12    function forever() public { infinite gas
13        // Here we run a loop until all of the gas are spent
14        // and the transaction fails
15        while (true) {
16            i += 1;
17        }
18    }
19 }

```

- Tutorial no. 19



REMUX 1.5.1 learneth tutorials Login with GitHub Theme

LEARNETH Tutorials list Syllabus

10.3 Transactions - Sending Ether 19 / 19

1. Create a contract called `Charity`.
2. Add a public state variable called `owner` of the type `address`.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the `owner` address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer Show answer

Next

Well done! No errors.

Scam Alert Initialize as git repo Did you know? You can learn Solidity basics and more using the Learneth plugin. RemixAI Copilot (enabled)

```

1 // SPDX-License-Identifier: MIT
2 // Atharva Prabhu D20A
3 pragma solidity ^0.8.3;
4
5 contract Charity {
6     // 1. Add a public state variable called owner of the type address
7     address public owner;
8
9     // We set the owner to the person who deploys the contract
10    constructor() { 257937 gas 233400 gas
11        owner = msg.sender;
12    }
13
14    // 2. Create a donate function: public and payable
15    // No parameters or function code required
16    function donate() public payable { 141 gas
17        // This allows the contract to accept Ether via this function call
18    }
19
20    // 3. Create a withdraw function: public
21    // Sends the total balance of the contract to the owner address
22    function withdraw() public { infinite gas
23        // Safety Check: Only the owner should be able to trigger a withdraw
24        require(msg.sender == owner, "Only the owner can withdraw funds");
25    }
26    // Get the current balance of the contract
27 }

```

Conclusion: Through this experiment, the fundamentals of Solidity programming were explored by completing practical assignments in the Remix IDE. Concepts such as data types, variables, functions, visibility, modifiers, constructors, control flow, data structures, and transactions were implemented and understood. The hands-on practice helped in designing, compiling, and deploying smart contracts on the Remix VM, thereby strengthening the understanding of blockchain concepts. This experiment provided a strong foundation for developing and managing smart contracts efficiently.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 05

Aim: Deploying a Voting/Ballot Smart Contract

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT 5

Name-Atharva Prabhu

Class-D20A

Roll no-45

Aim:Deploying a Voting/Ballot Smart Contract

Theory:

Relevance of require statements in solidity programs

In Solidity, the require statement acts as a guard condition within functions. It ensures that only valid inputs or authorized users can execute certain parts of the code. If the condition inside require is not satisfied, the function execution stops immediately, and all state changes made during that transaction are reverted to their original state. This rollback mechanism ensures that invalid transactions do not corrupt blockchain data.

For example, in a Voting Smart Contract, require can be used to check:

- Whether the person calling the function has the right to vote
(require(voters[msg.sender].weight > 0, "Has no right to vote");).
- Whether a voter has already voted before allowing them to vote again.
- Whether the function caller is the chairperson before granting voting rights.

Thus, require statements enforce security, correctness, and reliability in smart contracts. They prevent unauthorized access, eliminate invalid operations, and maintain the integrity of the voting process. Additionally, developers can attach error messages to require, which improves debugging and makes contract interaction clearer for users.

Keywords: mapping, storage, and memory

Mapping

A mapping is a special data structure in Solidity that links keys to values, similar to a hash table. Its syntax is:

```
mapping(keyType => valueType)
```

For example:

```
mapping(address => Voter) public voters;
```

Here, each address (Ethereum account) is mapped to a Voter structure. Mappings are very useful in contracts like Ballot, where it is necessary to associate each voter with their data (such as whether they have voted and which proposal they selected).

Unlike arrays:

- Mappings do not have a length property.
- They cannot be iterated over directly.
- They are highly gas-efficient for lookups.

Therefore, mappings are ideal for secure and efficient voter data management.

Storage

In Solidity, storage refers to the permanent memory of the contract stored on the blockchain. Variables declared at the contract level are stored in storage by default.

Key characteristics:

- Data is persistent across transactions.
- It remains available unless explicitly modified.
- Writing to storage consumes gas and is relatively expensive.
- For example, a voter's information saved in the voters mapping remains available throughout the contract's lifecycle. Storage is essential for maintaining the state of the voting system, such as voter records and proposal vote counts.

Memory

In contrast, memory is temporary storage used only during the execution of a function call. When the function finishes execution, data stored in memory is discarded.

Key characteristics:

- Temporary lifetime.
- Cheaper than storage.
- Used for local variables, function arguments, and intermediate calculations.
- For example, when handling proposal names passed into a function or performing temporary computations, memory is used instead of storage.
- Thus, a smart contract developer must carefully choose between storage and memory to balance cost efficiency and functionality.

Why bytes32 instead of string?

In earlier implementations of the Ballot contract, bytes32 was used for proposal names instead of string. The reason lies mainly in gas efficiency and simplicity.

bytes32 is a fixed-size data type that always stores exactly 32 bytes. This makes:

- Storage is simpler.
- Comparison operations faster.
- Gas consumption is lower.

However, it limits proposal names to 32 characters, reducing flexibility.

On the other hand, string is a dynamically sized data type. It can store text of variable length, making it more user-friendly and readable. However:

- It requires more complex handling inside the Ethereum Virtual Machine (EVM).
- It consumes more gas.
- String comparisons are more expensive.

Modern Ballot contract implementations often use string for better usability, while earlier versions preferred bytes32 for performance optimization.

In summary:

- bytes32 is preferred when performance and gas efficiency are priorities.
- string is preferred when readability and flexibility are more important.

Code-

```
// SPDX-License-Identifier: GPL-3.0

pragma solidity ^0.8.0;

/**
 * @title Ballot
 * @dev Implements voting process along with vote delegation
 */

contract Ballot {

    struct Voter {
        uint weight;
        bool voted;
        address delegate;
        uint vote;
    }

    struct Proposal {
```

```

    string name;

    uint voteCount;
}

address public chairperson;

mapping(address => Voter) public voters;

Proposal[] public proposals;

constructor(string[] memory proposalNames) {
    chairperson = msg.sender;
    voters[chairperson].weight = 1;

    for (uint i = 0; i < proposalNames.length; i++) {
        proposals.push(Proposal({
            name: proposalNames[i],
            voteCount: 0
        }));
    }
}

```

```
function giveRightToVote(address voter) external {  
    require(msg.sender == chairperson,  
        "Only chairperson can give right to vote.");  
    require(!voters[voter].voted,  
        "The voter already voted.");  
    require(voters[voter].weight == 0,  
        "Voter already has right to vote.");  
  
    voters[voter].weight = 1;  
}
```

```
function delegate(address to) external {  
    Voter storage sender = voters[msg.sender];  
  
    require(sender.weight != 0, "No right to vote");  
    require(!sender.voted, "Already voted");  
    require(to != msg.sender, "Self-delegation not allowed");  
  
    while (voters[to].delegate != address(0)) {  
        to = voters[to].delegate;  
        require(to != msg.sender, "Delegation loop detected");  
    }
```

```

Voter storage delegate_ = voters[to];

require(delegate_.weight >= 1, "Delegate has no right");

sender.voted = true;

sender.delegate = to;

if (delegate_.voted) {
    proposals[delegate_.vote].voteCount += sender.weight;
} else {
    delegate_.weight += sender.weight;
}
}

function vote(uint proposal) external {
    require(proposal < proposals.length,
        "Invalid proposal index");

    Voter storage sender = voters[msg.sender];

    require(sender.weight != 0,
        "No right to vote");

```

```

require(!sender.voted,
    "Already voted");

sender.voted = true;

sender.vote = proposal;

proposals[proposal].voteCount += sender.weight;
}

function winningProposal() public view returns (uint winningProposal_) {
    uint winningVoteCount = 0;

    for (uint p = 0; p < proposals.length; p++) {
        if (proposals[p].voteCount > winningVoteCount) {
            winningVoteCount = proposals[p].voteCount;
            winningProposal_ = p;
        }
    }
}

function winnerName() external view returns (string memory winnerName_) {
    winnerName_ = proposals[winningProposal()].name;
}

```

```
}
```

```
// Spandan Deb D20A
```

```
function getAllProposals() external view returns (string[] memory names, uint[]  
memory voteCounts) {
```

```
    uint length = proposals.length;
```

```
    names = new string[](length);
```

```
    voteCounts = new uint[](length);
```

```
    for (uint i = 0; i < length; i++) {
```

```
        names[i] = proposals[i].name;
```

```
        voteCounts[i] = proposals[i].voteCount;
```

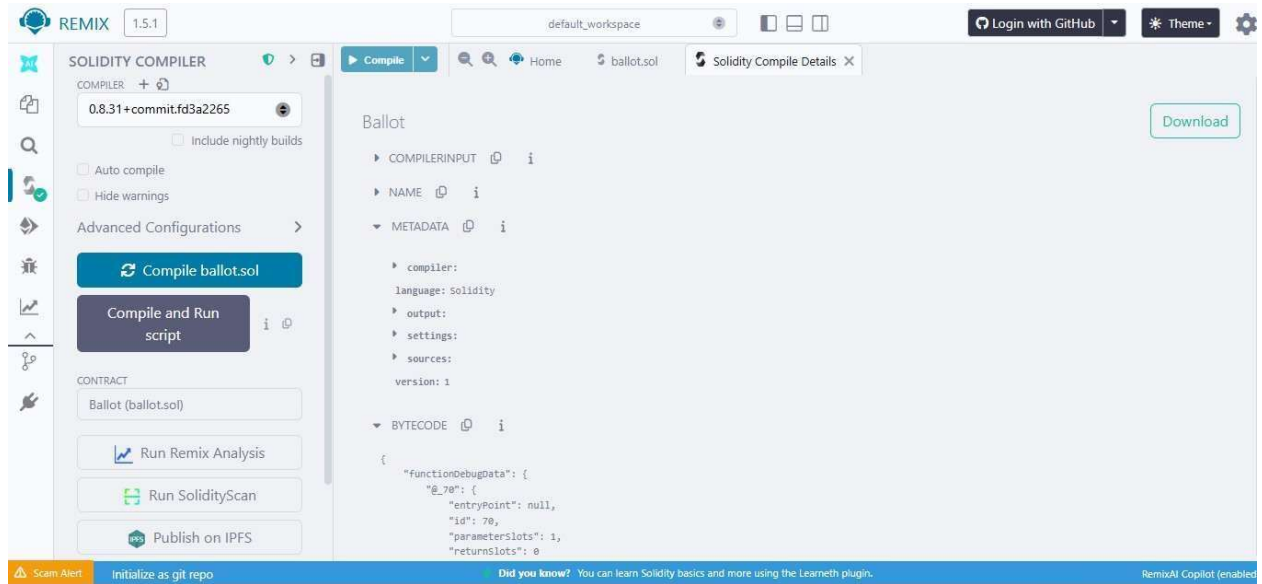
```
    }
```

```
}
```

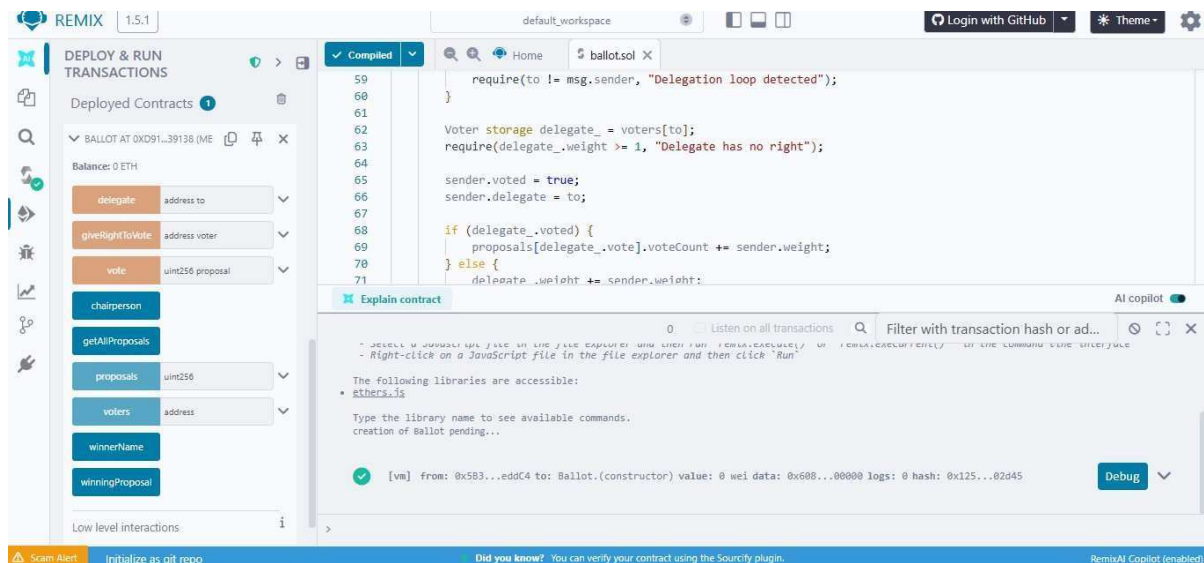
```
}
```

Output-

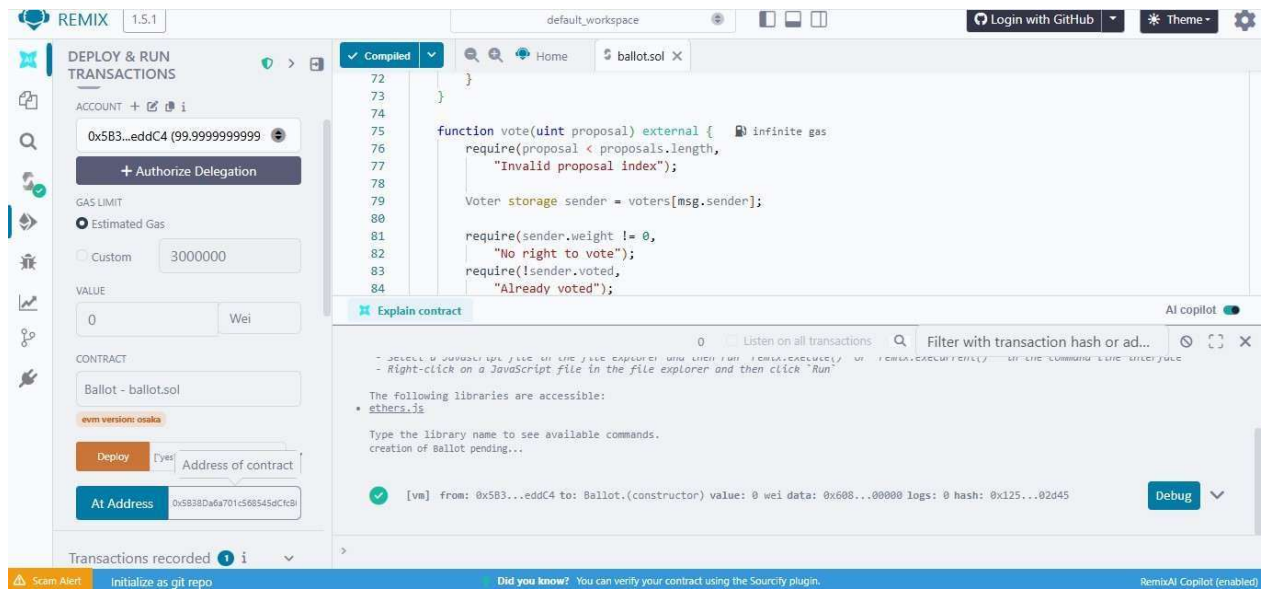
1) Compiled Ballot.sol contract



2) Deploying and running of contract



3) Loading the Proposal Candidate's Names (string)



4) Viewing the details of the proposal candidate

DEPLOY & RUN
TRANSACTIONS

Deployed Contracts 1

▼

BALLOT AT 0XD91...39138 (ME)

×

Balances: 0 ETH

delegate

address to

▼

giveRightToVote

address voter

▼

vote

uint256 proposal

▼

chairperson

0: address: 0x5838Da6a701c568545
dCfcB03FcB875f56beddC4

getAllProposals

0: string[]: names yes,no,lit

1: uint256[]: voteCounts: 0,1,0

proposals

uint256

▼

voters

address

▼

winnerName

winningProposal

5) Giving right to vote other than the chairperson

The screenshot shows the Remix IDE with the following components:

- Left Panel (DEPLOY & RUN TRANSACTIONS):** Displays the contract state. The "vote" button is highlighted with a value of 2. The "chairperson" button is also visible. The "getABProposals" button shows a list of proposals with names "yes", "no", and "lit", and vote counts of 0, 1, and 1 respectively.
- Top Panel (Code Editor):** Shows the Solidity code for the contract. The `vote` function is highlighted, which includes a `require` statement to ensure the proposal index is valid and a `require` statement to ensure the sender has the right to vote.
- Bottom Panel (Transaction Log):** Shows the execution of the `vote` function. The log indicates that the function was called from the address `0xAb843F64d9C6d1ECF9B849Ae677D03315835cb2` and that the data was `0xcce...b68f5`. The log also shows the state of the contract after the transaction, including the `chairperson` and `voters` arrays.

6) Selecting the account other than the chairperson to vote and then writing the proposal candidate's index to vote.

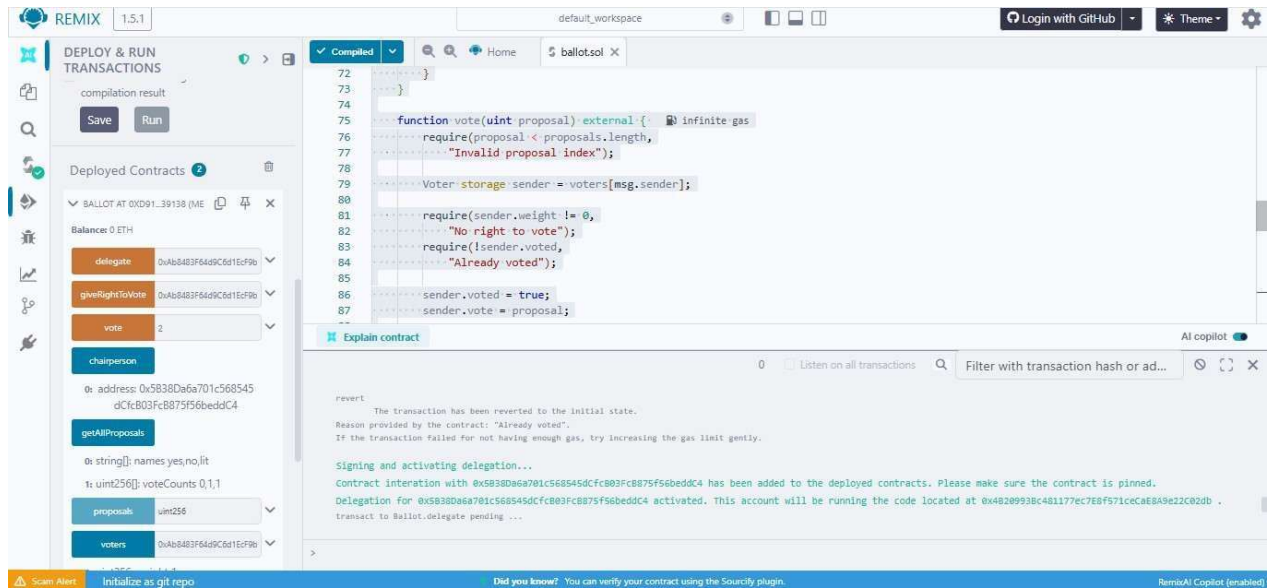
The screenshot shows the Remix IDE with the following components:

- Left Panel (DEPLOY & RUN TRANSACTIONS):** The same as the previous screenshot, but the "vote" button is now highlighted with a value of 1.
- Top Panel (Code Editor):** The same Solidity code is shown, but the `require` statement for the sender's weight is now highlighted.
- Bottom Panel (Transaction Log):** The log shows the execution of the `vote` function, but it ends with an error: `call to Ballot.voters errored: TypeError: Invalid address (argument="address", value="", code=INVALID_ARGUMENT, version=6.14.8) (argument="", value="", code=INVALID_ARGUMENT, version=6.14.8)`. This indicates that the sender's address is invalid.

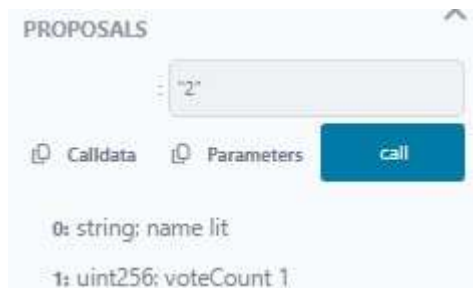
7) We can view the Voter's information



8) Selecting the account of the delegator and then writing the address of the voter to be delegated.



9) Proposal candidates information before the delegates vote



10) In this final screenshot we can see that after the delegation's vote the weight of the vote which was originally 1 (default) is now 2.



Conclusion:-

In this experiment, a Voting/Ballot smart contract was deployed using Solidity on the Remix IDE. The concepts of require statements, mapping, and data location specifiers like storage and memory were explored to understand their role in ensuring security, efficiency, and correctness in smart contracts. The difference between using bytes32 and string for proposal names was also studied, highlighting the trade-off between gas efficiency and readability. Overall, the experiment provided practical insights into the design and deployment of voting contracts on the blockchain.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 06

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT 6

Name-Atharva Prabhu

Class-D20A

Roll no-45

Aim: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

Theory:

What is a smart contract?

A smart contract is a self-executing digital agreement stored on a blockchain that automatically enforces its terms when predefined conditions are met, eliminating the need for intermediaries.

Significance of smart Contracts in Ethereum Blockchain

1. Programmability – Ethereum smart contracts make the blockchain programmable, enabling complex logic beyond simple payments. Decentralized Applications (dApps) – They power dApps like Uniswap, Aave, and OpenSea, running without central servers.
2. Trustless Automation – Transactions execute automatically without intermediaries, ensuring fairness and transparency.
3. Foundation for DeFi & NFTs – Smart contracts enable decentralized finance, NFT ownership transfers, and DAO governance.
4. Global Accessibility – Anyone can interact with Ethereum smart contracts worldwide, creating a borderless digital economy.

Code:

GenesisToken.sol

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/ERC20.sol";

import "@openzeppelin/contracts/access/Ownable.sol";

```

/**
 * @title GenesisToken
 * @notice Simple ERC20 used as the monitored token in Genesis demos.
 *      Owner can mint tokens for testing transfers, whale movements, etc.
 */
contract GenesisToken is ERC20, Ownable {
    uint8 private _decimals;

    constructor(
        string memory name_,
        string memory symbol_,
        uint8 decimals_,
        uint256 initialSupply
    ) ERC20(name_, symbol_) Ownable(msg.sender) {
        _decimals = decimals_;
        _mint(msg.sender, initialSupply * (10 ** decimals_));
    }

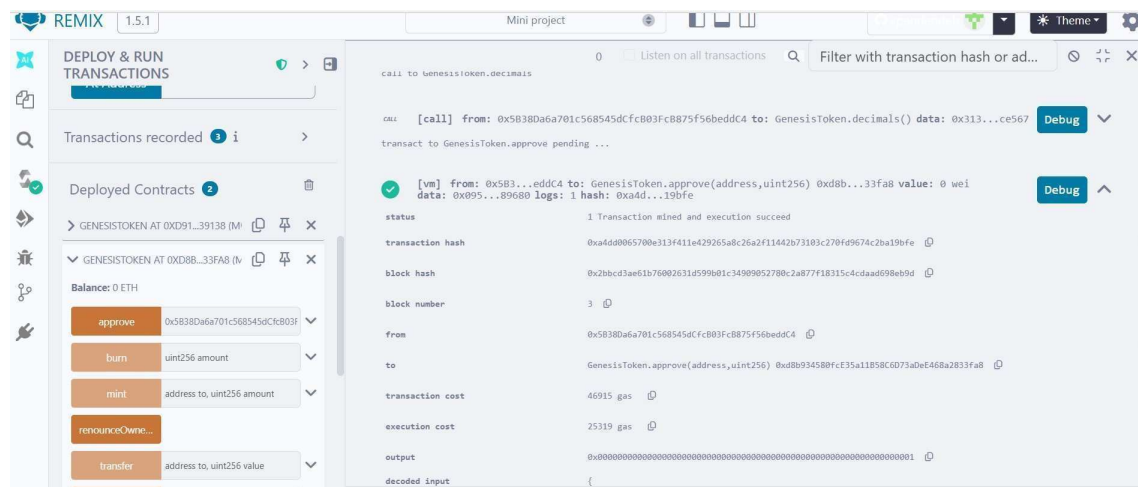
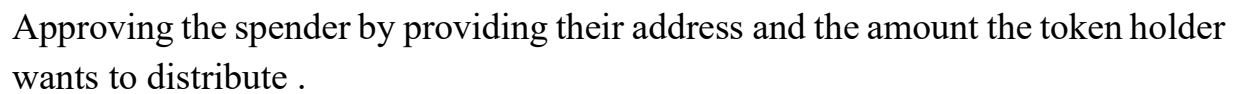
    function decimals() public view override returns (uint8) {
        return _decimals;
    }

    function mint(address to, uint256 amount) external onlyOwner {
        _mint(to, amount);
    }

    function burn(uint256 amount) external {
        _burn(msg.sender, amount);
    }
}

```

Compiled and deployed the contract by providing token name,token symbol,decimals and initial supply.



Burning and minting tokens

The screenshot shows the GenesisToken contract interface. On the left, there's a sidebar with a balance of 0 ETH and buttons for approve, burn, mint, renounceOwnership, transfer, transferFrom, and transferOwnership. The main area displays two transactions: a burn transaction and a mint transaction. The burn transaction shows a value of 1000000 being burned. The mint transaction shows a value of 0 being minted. Both transactions are pending.

Transferring tokens from your own balance using transfer and transfer tokens from someone else's balance to another address only if the address is approved using transfer from.

The screenshot shows the GenesisToken contract interface. On the left, there's a sidebar with buttons for transfer, transferFrom, and transferOwnership. The main area displays two transactions: a transfer transaction and a transferFrom transaction. The transfer transaction shows a value of 0 being transferred to address 0xa90...89680. The transferFrom transaction shows a value of 0 being transferred from address 0x58380a6a701c568545dcfc803f to address 0x23b...f4240. Both transactions are pending.

Checking the allowance of a wallet address after it has been approved and checking the balance of the owner.

The screenshot shows the GenesisToken contract interface. On the left, there's a sidebar with buttons for allowance, balanceOf, and decimals. The main area displays three transactions: an allowance transaction, a balanceOf transaction, and a call to GenesisToken.balanceOf. The allowance transaction shows a value of 0 being approved for address 0x58380a6a701c568545dcfc803f. The balanceOf transaction shows a value of 1000088000000 being checked for address 0x58380a6a701c568545dcfc803f. The call to GenesisToken.balanceOf shows a value of 0 being checked for address 0x70a...eddc4. All transactions are pending.

Code-

GenesisVault.sol

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

```

import "@openzeppelin/contracts/access/Ownable.sol";
import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

/**
 * @title GenesisVault
 * @notice On-chain vault that emits rich events for every deposit, withdrawal,
 *         and internal transfer. Genesis monitors these events in real time.
 *
 * KEY BLOCKCHAIN FEATURES SHOWCASED:
 *   1. Event-driven architecture (Solidity `event`)
 *   2. Inbound/outbound tracking on-chain
 *   3. Reentrancy protection
 *   4. Per-user balance accounting
 *   5. Emergency pause (circuit-breaker pattern)
 */
contract GenesisVault is Ownable, ReentrancyGuard {
    using SafeERC20 for IERC20;

    // ----- State -----
    IERC20 public immutable token;

    mapping(address => uint256) public balances;
    uint256 public totalDeposits;
    uint256 public totalWithdrawals;
    bool public paused;

    // ----- Events (the core of Genesis monitoring) -----
    /// @notice Emitted on every deposit — Genesis indexes this immediately.
    event Deposit(
        address indexed user,
        uint256 amount,
        uint256 newBalance,
        uint256 timestamp
    );

```

/// @notice Emitted on every withdrawal — potential "outbound whale" signal.

```
event Withdrawal(  
    address indexed user,  
    uint256 amount,  
    uint256 remainingBalance,  
    uint256 timestamp  
);
```

/// @notice Vault-internal transfer between two users.

```
event InternalTransfer(  
    address indexed from,  
    address indexed to,  
    uint256 amount,  
    uint256 timestamp  
);
```

/// @notice Large movement detected on-chain — threshold exceeded.

```
event LargeMovement(  
    address indexed user,  
    string movementType, // "deposit" | "withdrawal"  
    uint256 amount,  
    uint256 thresholdUsed,  
    uint256 timestamp  
);
```

/// @notice Emergency pause toggled.

```
event EmergencyAction(  
    string action, // "paused" | "unpaused"  
    address indexed triggeredBy,  
    uint256 timestamp  
);
```

// _____ Modifiers _____

```
modifier whenNotPaused() {
```

```

        require(!paused, "Vault: paused");
        _;
    }

// ----- Constructor -----
constructor(address tokenAddress) Ownable(msg.sender) {
    token = IERC20(tokenAddress);
}

// ----- Core functions -----

/**
 * @notice Deposit tokens into the vault.
 * @param amount Number of token units (with decimals).
 */
function deposit(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");

    token.safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender] += amount;
    totalDeposits += amount;

    emit Deposit(msg.sender, amount, balances[msg.sender], block.timestamp);

    // On-chain large-movement detection (threshold hard-coded for demo;
    // the ThresholdEngine contract provides user-customisable thresholds).
    if (amount >= 100_000 * 1e6) {
        emit LargeMovement(msg.sender, "deposit", amount, 100_000 * 1e6,
block.timestamp);
    }
}

/**
 * @notice Withdraw tokens from the vault.
 */

```

```

function withdraw(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");
    require(balances[msg.sender] >= amount, "Vault: insufficient balance");

    balances[msg.sender] -= amount;
    totalWithdrawals += amount;
    token.safeTransfer(msg.sender, amount);

    emit Withdrawal(msg.sender, amount, balances[msg.sender],
block.timestamp);

    if (amount >= 100_000 * 1e6) {
        emit LargeMovement(msg.sender, "withdrawal", amount, 100_000 * 1e6,
block.timestamp);
    }
}

/**
 * @notice Transfer balance to another vault user without touching the token
contract.
 *      Demonstrates internal accounting — useful for Layer-2 patterns.
 */
function internalTransfer(address to, uint256 amount) external nonReentrant
whenNotPaused {
    require(to != address(0) && to != msg.sender, "Vault: invalid recipient");
    require(balances[msg.sender] >= amount, "Vault: insufficient balance");

    balances[msg.sender] -= amount;
    balances[to] += amount;

    emit InternalTransfer(msg.sender, to, amount, block.timestamp);
}

// _____ Admin / Emergency _____

```

```
function pause() external onlyOwner {  
    paused = true;  
    emit EmergencyAction("paused", msg.sender, block.timestamp);  
}
```

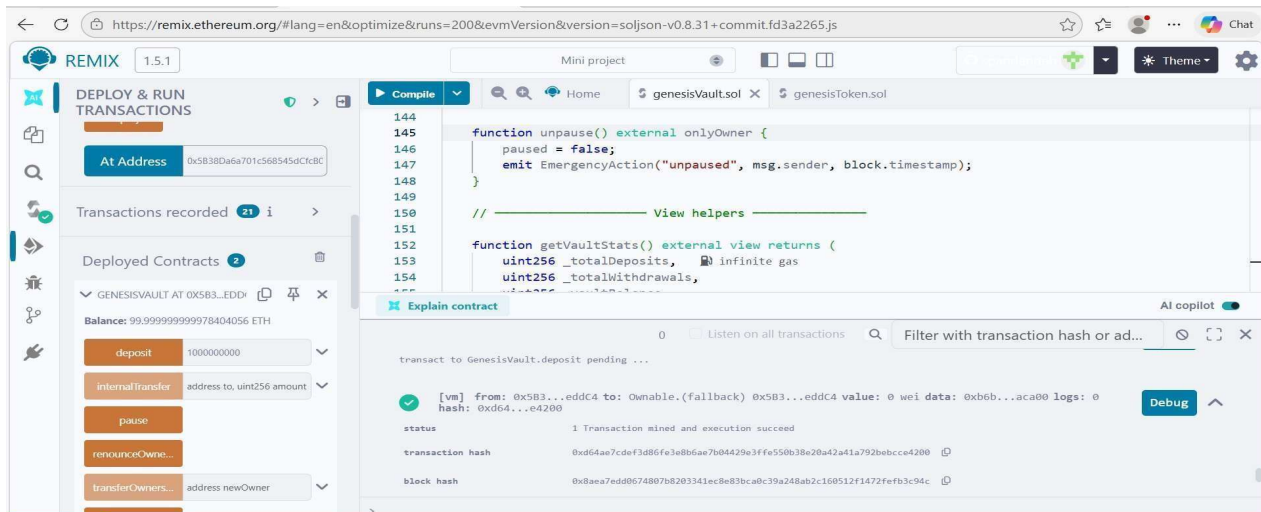
```
function unpause() external onlyOwner {  
    paused = false;  
    emit EmergencyAction("unpaused", msg.sender, block.timestamp);  
}
```

```
// ----- View helpers -----
```

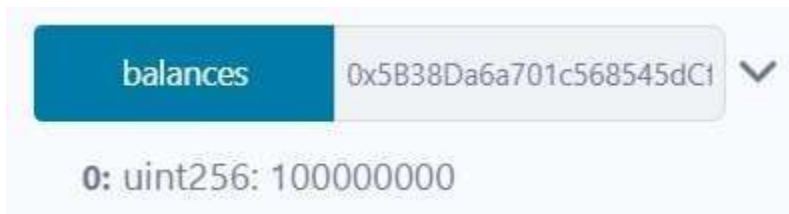
```
function getVaultStats() external view returns (  
    uint256 _totalDeposits,  
    uint256 _totalWithdrawals,  
    uint256 _vaultBalance,  
    bool _paused  
) {  
    return (totalDeposits, totalWithdrawals, token.balanceOf(address(this)),  
    paused);  
}  
}
```

Output-

After minting new tokens, and approving we are depositing an amount to the vault



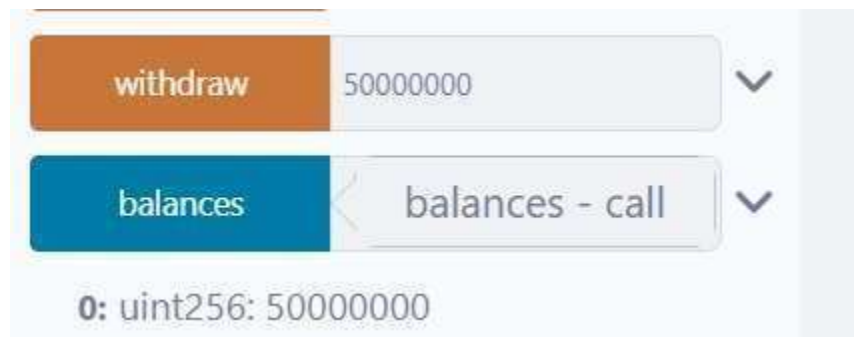
Balance is the same as the amount deposited.



The stats of the Vault



Withdrew 50 tokens and checked balance to confirm .



Transferred 20 tokens to a different account via internal transfer and checked the balance which reduced from 50 tokens to 30 tokens.



Checked the balance of the account to which the tokens were transferred



Conclusion-

In this experiment, we successfully deployed and executed the GenesisToken and GenesisVault smart contracts using Remix IDE. We learned how to transfer tokens securely between contracts and performed deposit, withdrawal, and internal transfer operations.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab

Experiment 07

Aim: Implement the embedding wallet (Metamask) and transaction using Solidity

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT 7

Name-Atharva Prabhu

Class-D20A

Roll no-45

Aim- Implement the embedding wallet (Metamask) and transaction using Solidity.

Theory-

What is MetaMask?

MetaMask is a cryptocurrency wallet and browser extension that allows users to interact with blockchain networks like Ethereum.

- It stores private keys securely
- Enables sending/receiving crypto (like ETH)
- Allows interaction with decentralized applications (DApps)
- Works as a bridge between your browser and blockchain

What is Testnet?

A test net (test network) is a blockchain environment used for testing smart contracts and transactions without using real money.

- Uses free “test cryptocurrency” (no real value)
- Helps developers debug and experiment safely
- Examples:
 - a. Sepolia Testnet
 - b. Goerli Testnet

List the steps to connect MetaMask with RemixIDE for transactions.

Step 1: Install MetaMask

- Add MetaMask extension to browser
- Create or import a wallet

Step 2: Select a Network

- Open MetaMask
- Choose a test network (e.g., Sepolia)

Step 3: Get Test ETH

- Use a faucet to receive free test ETH

Step 4: Open Remix IDE

- Go to Remix IDE in browser

Step 5: Write/Compile Smart Contract

- Create a Solidity file
- Compile the contract

Step 6: Connect MetaMask to Remix

- Go to Deploy & Run Transactions tab
- In Environment dropdown → select Injected Provider - MetaMask

Step 7: Authorize Connection

- MetaMask will pop up → click Connect

Step 8: Deploy Contract

- Click Deploy in Remix
- Confirm transaction in MetaMask

Step 9: Perform Transactions

- Call contract functions
- Each transaction will require MetaMask confirmation

Implementation-

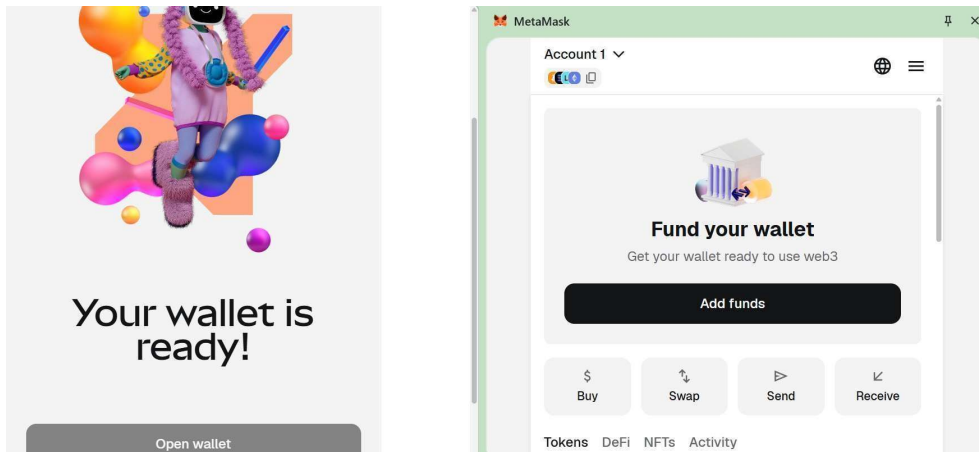
Install metamask



Create a new metamask wallet and give it a password

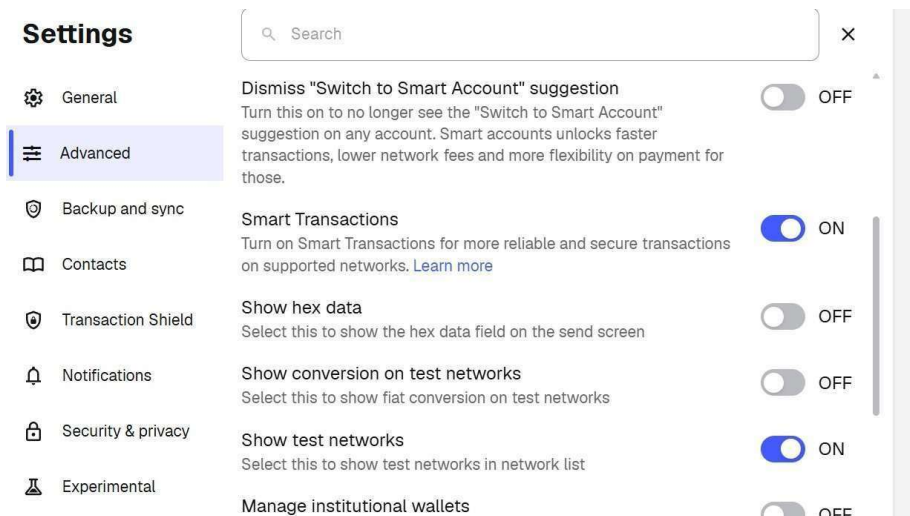
A screenshot of the MetaMask password creation screen. It features a back arrow at the top left. The title 'MetaMask password' is prominently displayed. Below the title, a warning message states: 'Losing this password means losing wallet access on all devices, MetaMask can't reset it.' There are two input fields: 'Create new password' and 'Confirm password'. Each field has a toggle icon (an eye) to the right, likely for showing or hiding the password. A note below the first field says 'Must be at least 8 characters'.

Account is successfully created

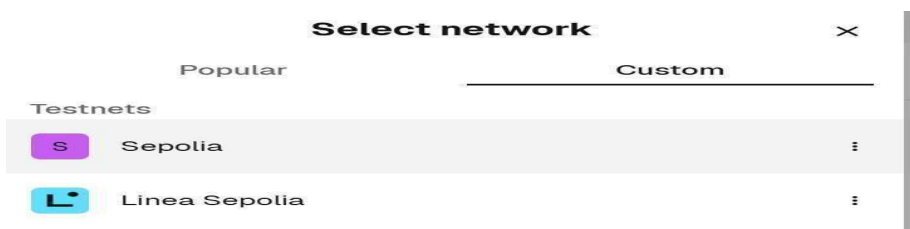


Steps to get funds from Sepolia Testnet

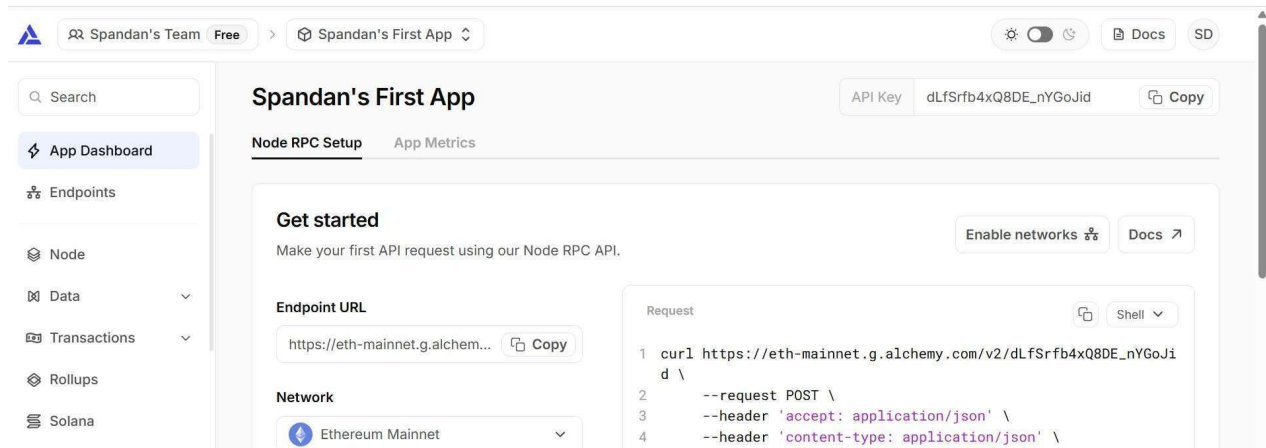
In settings switch on to show test networks



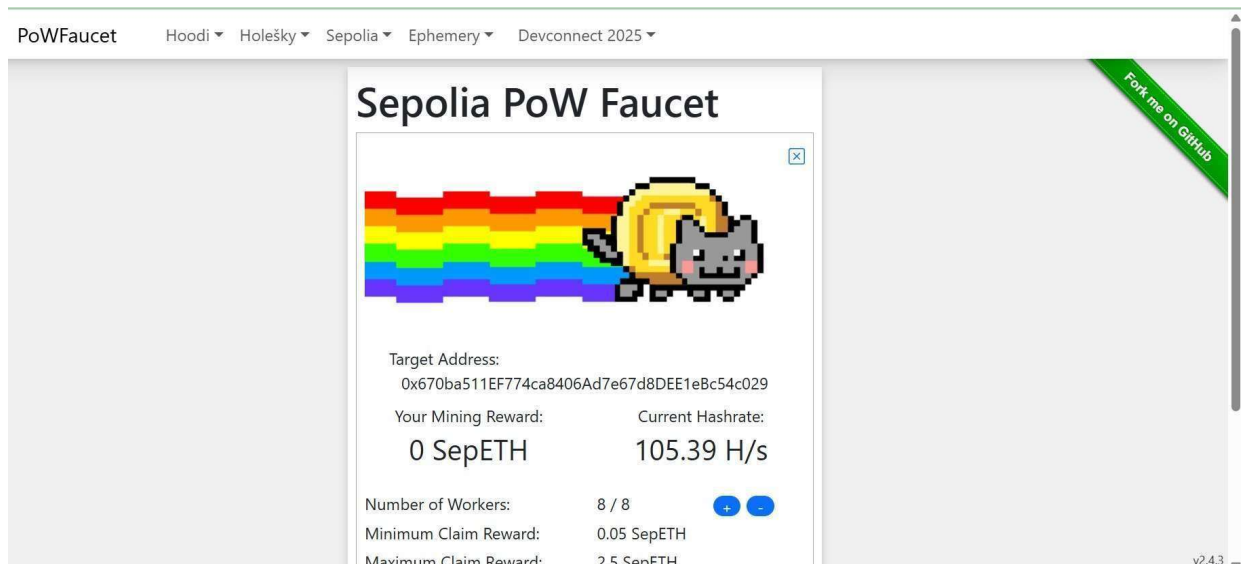
Select sepolia testnet



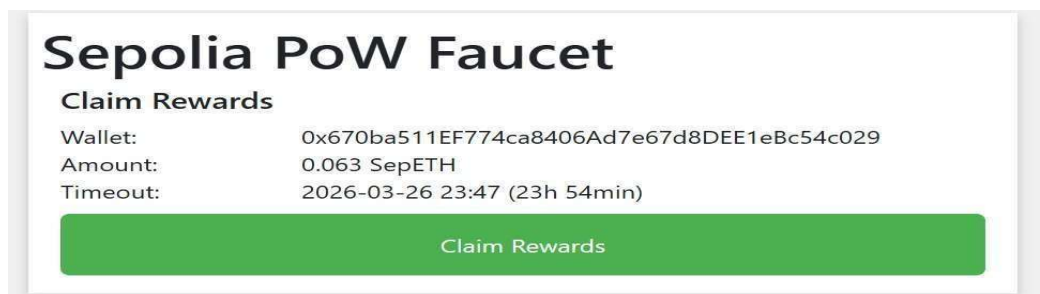
Create a free account in alchemy



Alchemy now requires this at least 0.001 ETH on Ethereum Mainnet, Shifting to PoW faucet.



Click on Claim rewards



Transaction is confirmed

Sepolia PoW Faucet

Claim Rewards

Wallet: 0x670ba511EF774ca8406Ad7e67d8DEE1eBc54c029
Amount: 0.063 SepETH
Timeout: -

Claim Transaction has been confirmed in block #10520426!

TX: [0x9552849a510aa5196ccdabc90780008599cb9a33341743959781682f84879baa](#)

Did you like the faucet? Give that project a  Star 5,456

Or support this faucet by sharing your result with a  Tweet  Post

[Return to startpage](#)

Account Balance



Account 1 





0.0626 SepoliaETH

+\$0.00 (+0.00%) [Discover](#) 

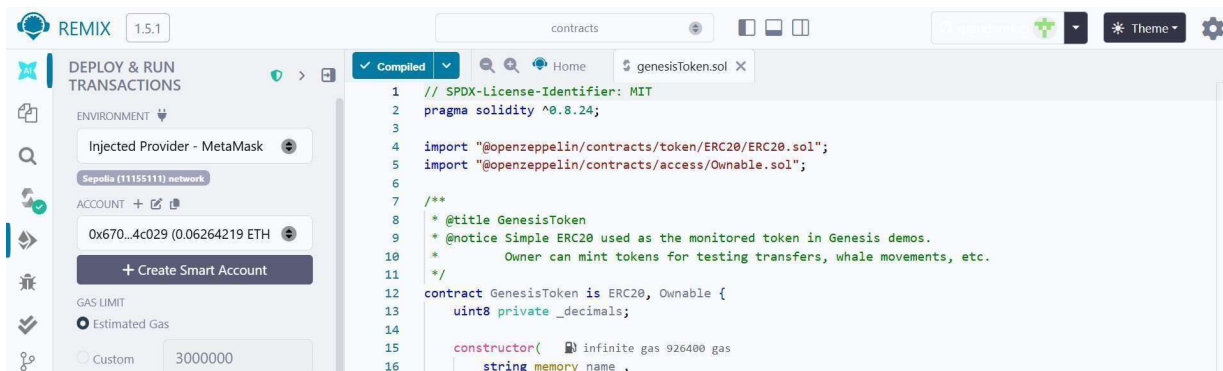
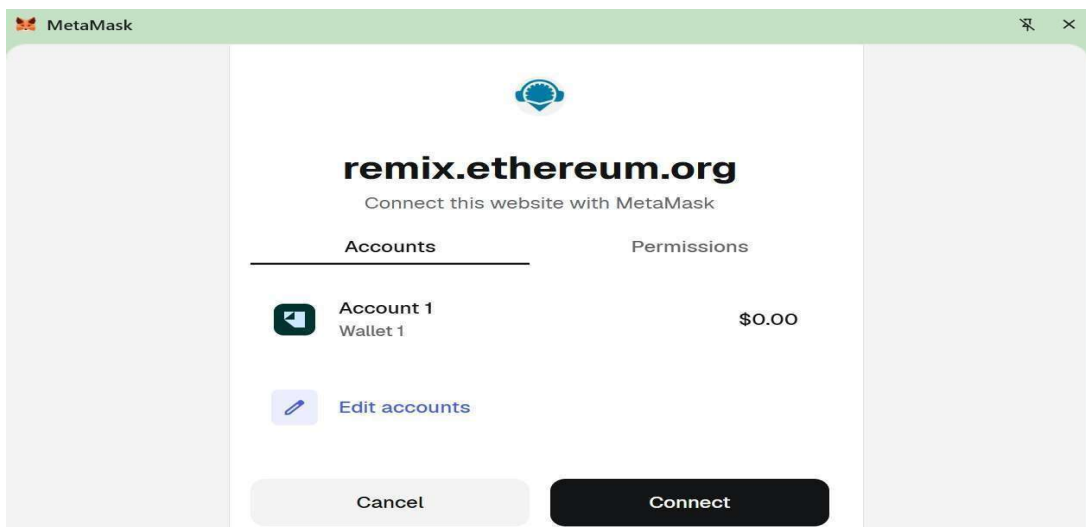
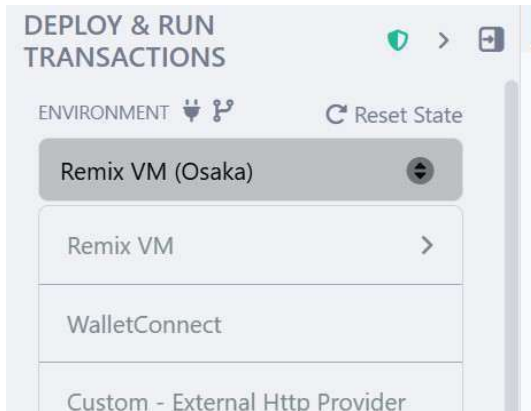
 Buy

 Swap

 Send

 Receive

Go to Remix IDE ,compile the smart contract in deploy under environment select wallet connect



GenesisToken.sol

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.24;
```

```
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

```
import "@openzeppelin/contracts/access/Ownable.sol";
```

```
/**
```

```
 * @title GenesisToken
```

```
 * @notice Simple ERC20 used as the monitored token in Genesis demos.
```

```
 *      Owner can mint tokens for testing transfers, whale movements, etc.
```

```
 */
```

```
contract GenesisToken is ERC20, Ownable {
```

```
    uint8 private _decimals;
```

```
    constructor(
```

```
        string memory name_,
```

```
        string memory symbol_,
```

```
        uint8 decimals_,
```

```
        uint256 initialSupply
```

```
    ) ERC20(name_, symbol_) Ownable(msg.sender) {
```

```
        _decimals = decimals_;
```

```
        _mint(msg.sender, initialSupply * (10 ** decimals_));
```

```
    }
```

```

function decimals() public view override returns (uint8) {

    return _decimals;

}

function mint(address to, uint256 amount) external onlyOwner {

    _mint(to, amount);

}

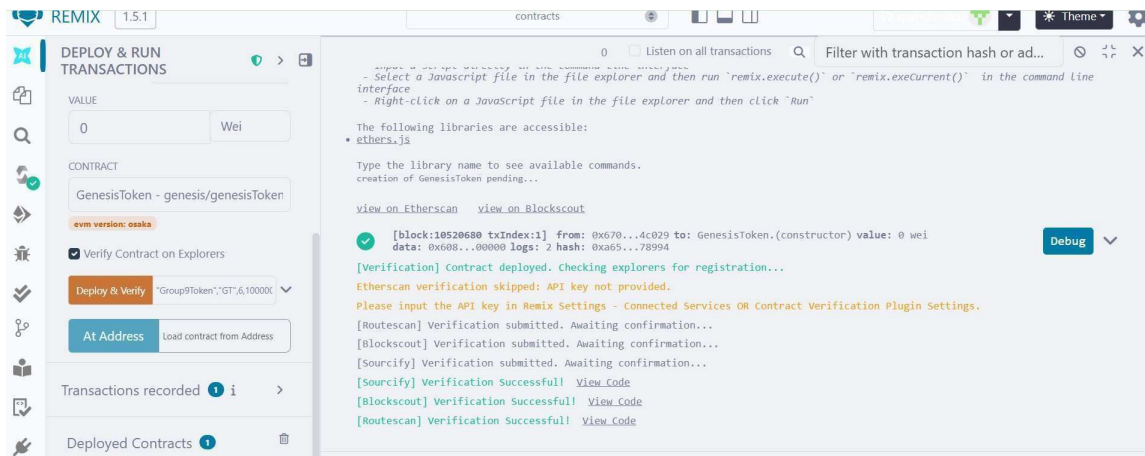
function burn(uint256 amount) external {

    _burn(msg.sender, amount);

}
}

```

Contract Deployed



Transaction Request

MetaMask

Account 1

Deploy a contract

This site wants you to deploy a contract

Estimated changes ?

No changes

Network

s Sepolia

Request from ?

remix.ethereum.org

Network fee ?

0.0005

s SepoliaETH

Speed

Market ~12 sec

Cancel

Confirm

Confirmation on Metamask

Contract deployment

Status

Confirmed

[View on block explorer](#)

[Copy transaction ID](#)

From

0x670ba...4c029

New contract

To

Transaction

Nonce

Amount

Gas Limit (Units)

Gas Used (Units)

Base fee (GWEI)

Priority fee (GWEI)

Total gas fee

0

-0 SepoliaETH

1230881

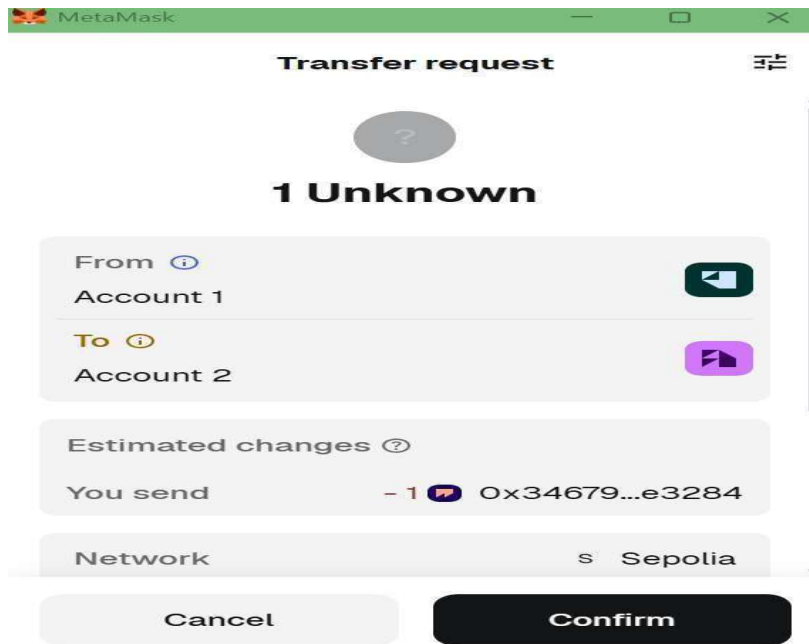
1220031

0.0000000018

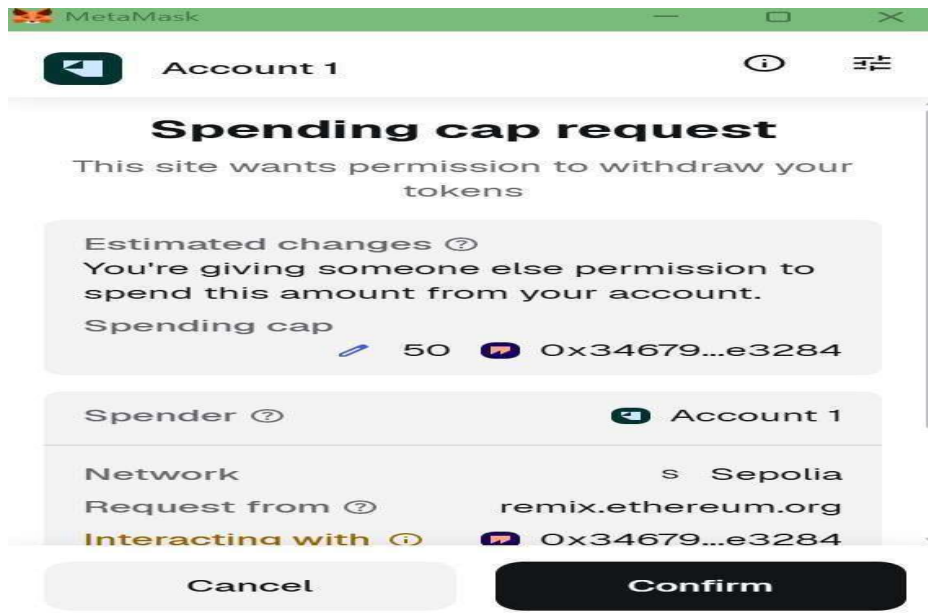
1.5

0.00183 SepoliaETH

Transferred 1 token to another account via metamask



Approved the spending of tokens



✓

[block:10520811 txIndex:10] from: 0x670...4c029 to: GenesisToken.approve(address,uint256) 0x346...e3284 value: 0 wei data: 0x095...af080 logs: 1 hash: 0x810...6a6be

Debug

▼

Contract details on metamask

TokensDeFiNFTsActivity

Sepolia

Mar 26, 2026

✓

Approve GT spending cap

Confirmed

↗

Sent Token

Confirmed

-0 SepoliaETH

🔗

Contract deployment

Confirmed

-0 SepoliaETH
-0 SepoliaETH

🔗

Contract deployment

Confirmed

-0 SepoliaETH
-0 SepoliaETH

Conclusion-

The embedding wallet MetaMask was successfully integrated with Remix IDE to interact with smart contracts written in Solidity. Transactions such as deployment and token transfer were executed on the Sepolia Testnet. This experiment demonstrated secure wallet connectivity, transaction signing, and practical implementation of blockchain-based operations.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 08

Aim: To implement the Blockchain platform Ganache and deploy a simple Solidity smart contract using Metamask and Remix IDE.

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO5:Interpret different Crypto assets and Crypto currencies

EXPERIMENT 8

Name-Atharva Prabhu

Class-D20A

Roll no-45

Aim-Implement the Blockchain platform Ganache.

Theory-

What is Ganache?

Ganache is a local blockchain simulator used for developing and testing Ethereum smart contracts.

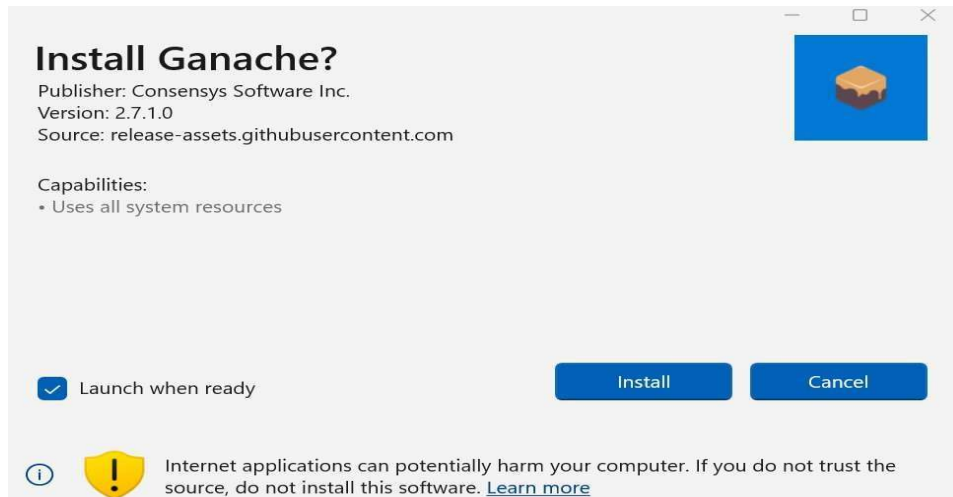
- It creates a personal blockchain on your computer
- Provides pre-funded accounts with test ETH
- Allows instant transactions (no real gas cost)
- Used for testing before deploying on real networks

List the steps involved in connecting Ganache environment with a Metamask and Remix IDE.

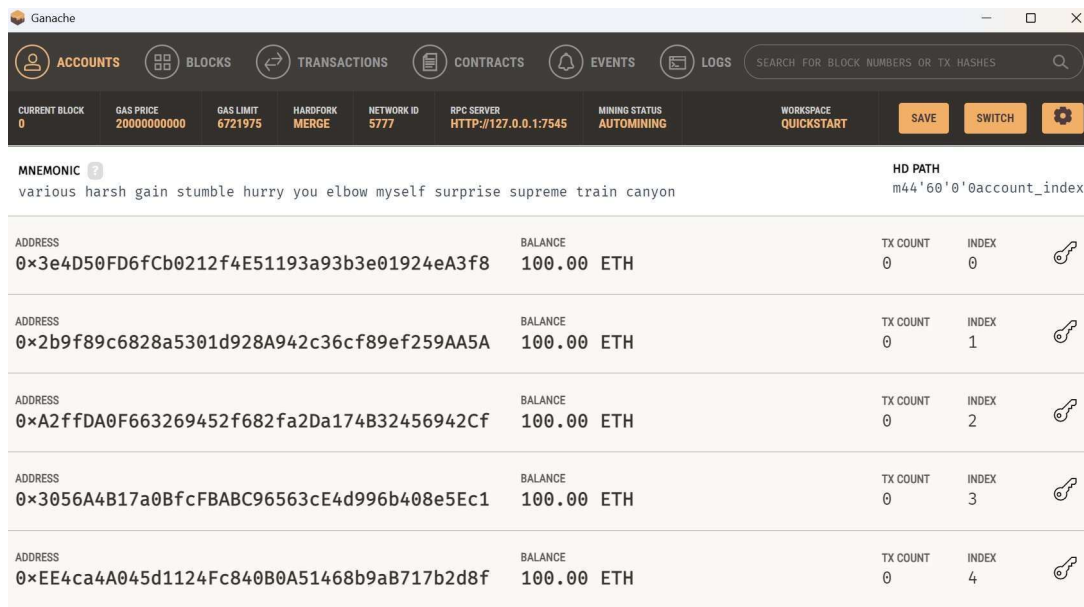
1. Start Ganache and note RPC URL and Chain ID.
2. Open MetaMask.
3. Add a new network in MetaMask using Ganache RPC URL and Chain ID.
4. Copy a private key from Ganache.
5. Import the account into MetaMask using the private key.
6. Open Remix IDE.
7. Go to “Deploy & Run Transactions”.
8. Select environment as Injected Provider - MetaMask.
9. Connect MetaMask to Remix.
10. Compile the Solidity contract.
11. Deploy the contract and confirm the transaction in MetaMask.
12. Interact with the contract functions to perform transactions.

Implementation-

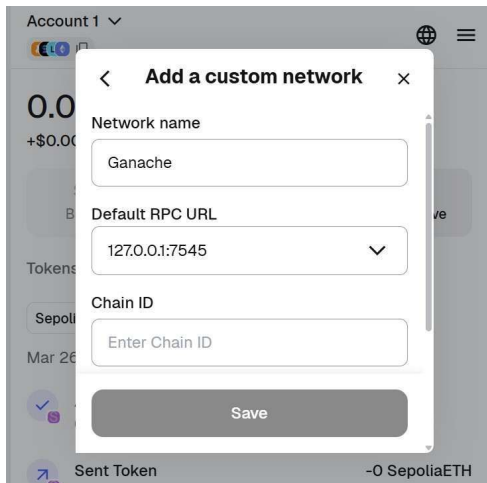
Installing Ganache



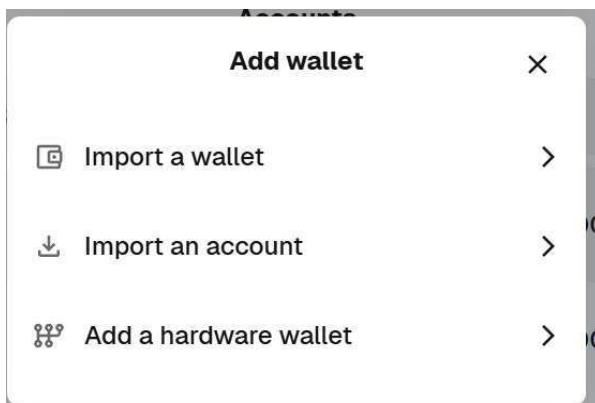
After installing click on quickstart ethereum to get 10 accounts worth 100 ETH for free.



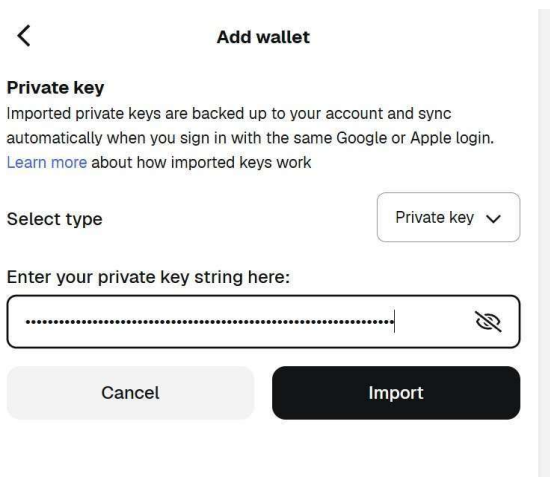
Add ganache network in metamask



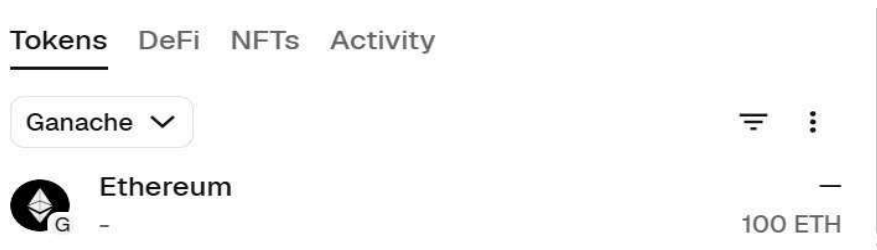
Select import an account in add wallet



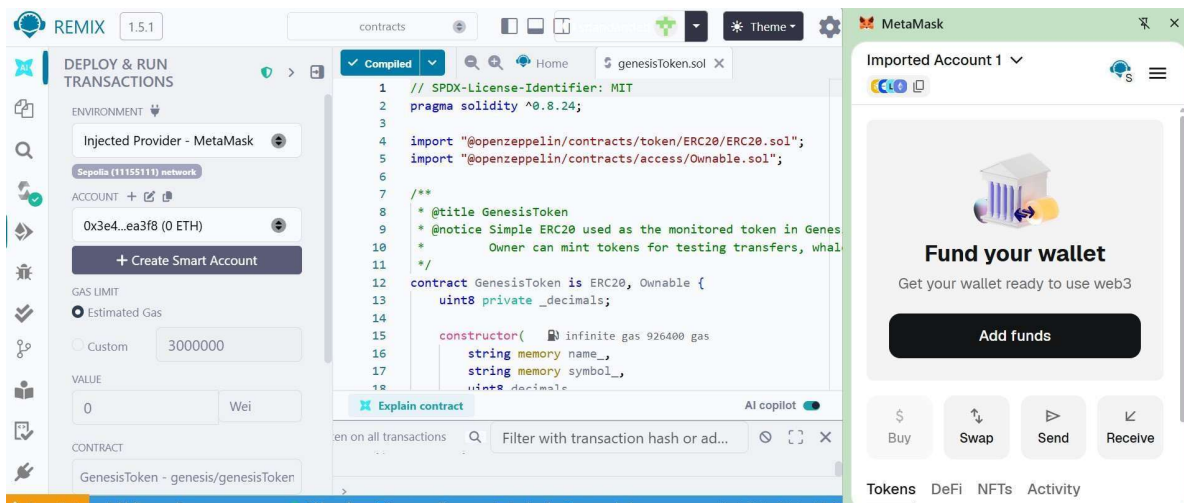
Copy the private key of any of the 10 accounts and paste it here



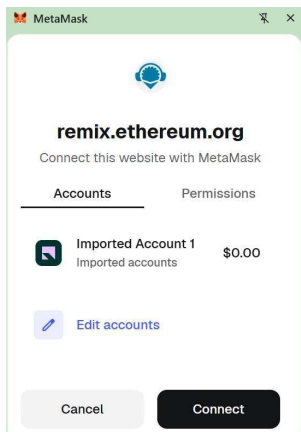
Account imported successfully balance is 100 ETH



Select in environment injected provider metamask



Connect with remix ide



GenesisToken.sol

```
// SPDX-License-Identifier: MIT
```

```
pragma solidity ^0.8.24;
```

```
import "@openzeppelin/contracts/token/ERC20/ERC20.sol";
```

```
import "@openzeppelin/contracts/access/Ownable.sol";
```

```
/**
```

```
 * @title GenesisToken
```

```
 * @notice Simple ERC20 used as the monitored token in Genesis demos.
```

```
 *      Owner can mint tokens for testing transfers, whale movements, etc.
```

```
 */
```

```
contract GenesisToken is ERC20, Ownable {
```

```
    uint8 private _decimals;
```

```
    constructor(
```

```
        string memory name_,
```

```
        string memory symbol_,
```

```
        uint8 decimals_,
```

```
        uint256 initialSupply
```

```
    ) ERC20(name_, symbol_) Ownable(msg.sender) {
```

```
        _decimals = decimals_;
```

```
        _mint(msg.sender, initialSupply * (10 ** decimals_));
```

```
    }
```

```

function decimals() public view override returns (uint8) {
    return _decimals;
}

function mint(address to, uint256 amount) external onlyOwner {
    _mint(to, amount);
}

function burn(uint256 amount) external {
    _burn(msg.sender, amount);
}
}

```

GenesisVault.sol

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.24;

import "@openzeppelin/contracts/token/ERC20/IERC20.sol";

import "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";

import "@openzeppelin/contracts/access/Ownable.sol";

import "@openzeppelin/contracts/utils/ReentrancyGuard.sol";

/**

* @title GenesisVault

* @notice On-chain vault that emits rich events for every deposit, withdrawal,

* and internal transfer. Genesis monitors these events in real time.

*

* KEY BLOCKCHAIN FEATURES SHOWCASED:

* 1. Event-driven architecture (Solidity `event`)

* 2. Inbound/outbound tracking on-chain

* 3. Reentrancy protection

* 4. Per-user balance accounting

* 5. Emergency pause (circuit-breaker pattern)

*/

contract GenesisVault is Ownable, ReentrancyGuard {

using SafeERC20 for IERC20;

// _____ State _____

IERC20 public immutable token;

mapping(address => uint256) public balances;

uint256 public totalDeposits;

uint256 public totalWithdrawals;

bool public paused;

// _____ Events (the core of Genesis monitoring) _____

/// @notice Emitted on every deposit — Genesis indexes this immediately.

event Deposit(

address indexed user,

```
uint256 amount,  
uint256 newBalance,  
uint256 timestamp  
);
```

```
/// @notice Emitted on every withdrawal — potential "outbound whale" signal.
```

```
event Withdrawal(  
    address indexed user,  
    uint256 amount,  
    uint256 remainingBalance,  
    uint256 timestamp  
);
```

```
/// @notice Vault-internal transfer between two users.
```

```
event InternalTransfer(  
    address indexed from,  
    address indexed to,  
    uint256 amount,  
    uint256 timestamp  
);
```

```
/// @notice Large movement detected on-chain — threshold exceeded.
```

```
event LargeMovement(  
    address indexed from,  
    address indexed to,  
    uint256 amount,  
    uint256 timestamp  
);
```

```
    address indexed user,
    string movementType, // "deposit" | "withdrawal"
    uint256 amount,
    uint256 thresholdUsed,
    uint256 timestamp
);
```

```
/// @notice Emergency pause toggled.
```

```
event EmergencyAction(
    string action, // "paused" | "unpaused"
    address indexed triggeredBy,
    uint256 timestamp
);
```

```
// _____ Modifiers _____
```

```
modifier whenNotPaused() {
    require(!paused, "Vault: paused");
    _;
}
```

```
// _____ Constructor _____
```

```
constructor(address tokenAddress) Ownable(msg.sender) {
    token = IERC20(tokenAddress);
}
```

```
// ----- Core functions -----

/**
 * @notice Deposit tokens into the vault.
 * @param amount Number of token units (with decimals).
 */

function deposit(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");
    token.safeTransferFrom(msg.sender, address(this), amount);
    balances[msg.sender] += amount;
    totalDeposits += amount;
    emit Deposit(msg.sender, amount, balances[msg.sender], block.timestamp);
    // On-chain large-movement detection (threshold hard-coded for demo;
    // the ThresholdEngine contract provides user-customisable thresholds).
    if (amount >= 100_000 * 1e6) {
        emit LargeMovement(msg.sender, "deposit", amount, 100_000 * 1e6, block.timestamp);
    }
}

/**
 * @notice Withdraw tokens from the vault.
 */

function withdraw(uint256 amount) external nonReentrant whenNotPaused {
    require(amount > 0, "Vault: zero amount");
    require(balances[msg.sender] >= amount, "Vault: insufficient balance");
}
```

```

balances[msg.sender] -= amount;

totalWithdrawals += amount;

token.safeTransfer(msg.sender, amount);

emit Withdrawal(msg.sender, amount, balances[msg.sender], block.timestamp);

if (amount >= 100_000 * 1e6) {

    emit LargeMovement(msg.sender, "withdrawal", amount, 100_000 * 1e6,
block.timestamp);

}

}

/**

* @notice Transfer balance to another vault user without touching the token contract.

*     Demonstrates internal accounting — useful for Layer-2 patterns.

*/

function internalTransfer(address to, uint256 amount) external nonReentrant whenNotPaused
{

    require(to != address(0) && to != msg.sender, "Vault: invalid recipient");

    require(balances[msg.sender] >= amount, "Vault: insufficient balance");

    balances[msg.sender] -= amount;

    balances[to] += amount;

    emit InternalTransfer(msg.sender, to, amount, block.timestamp);

}

// _____ Admin / Emergency _____

```

```

function pause() external onlyOwner {

    paused = true;

    emit EmergencyAction("paused", msg.sender, block.timestamp);

}

function unpause() external onlyOwner {

    paused = false;

    emit EmergencyAction("unpaused", msg.sender, block.timestamp);

}

// _____ View helpers _____

function getVaultStats() external view returns (

    uint256 _totalDeposits,

    uint256 _totalWithdrawals,

    uint256 _vaultBalance,

    bool _paused

) {

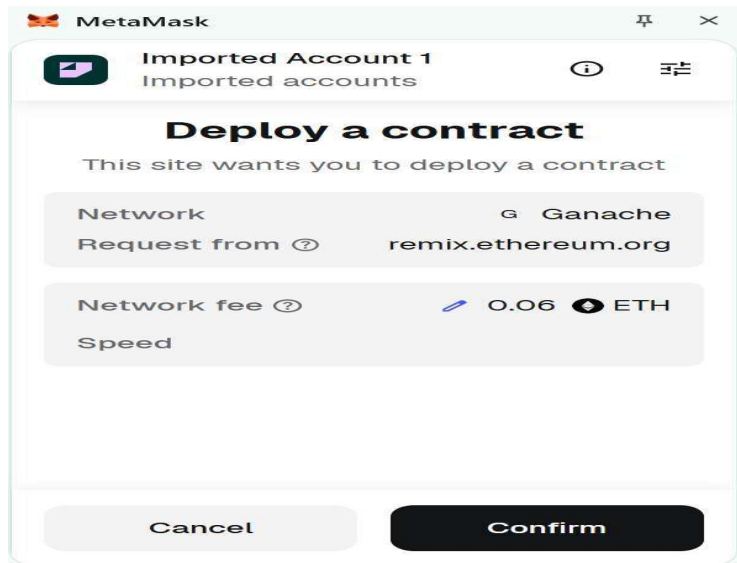
    return (totalDeposits, totalWithdrawals, token.balanceOf(address(this)), paused);

}

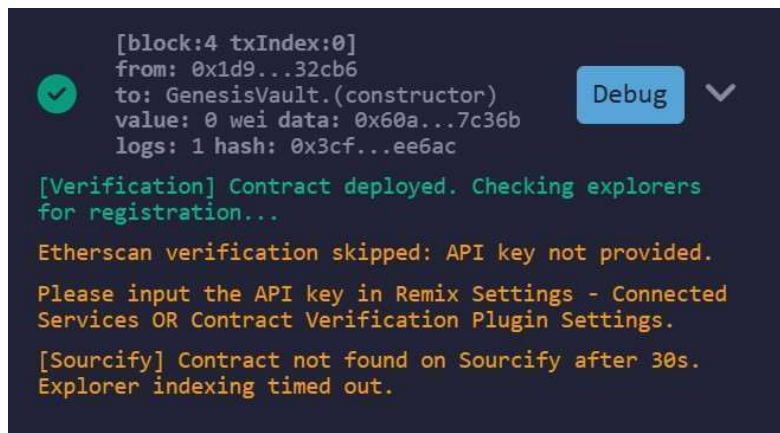
}

```

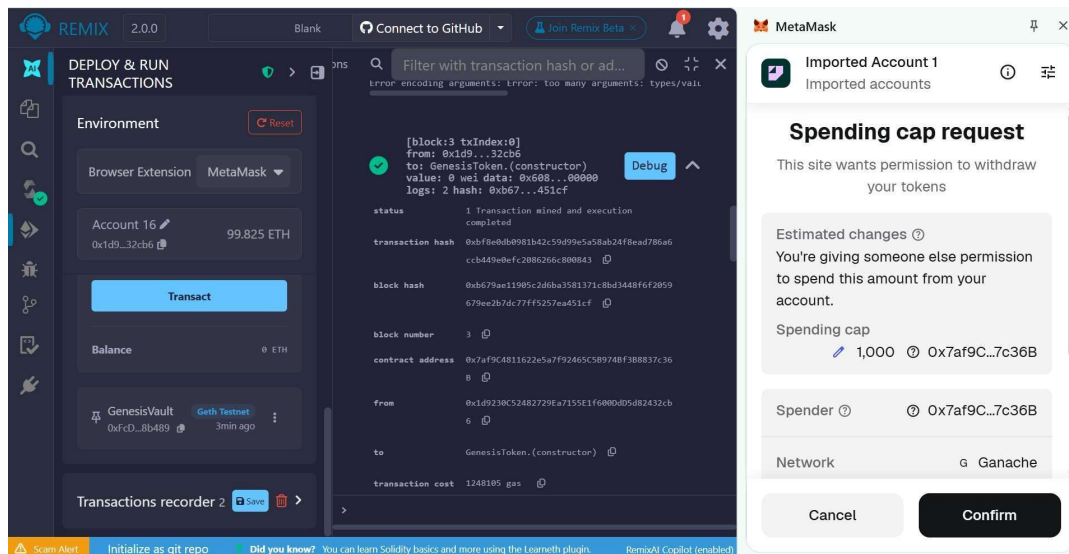
Contract Deployment of GenesisToken.sol



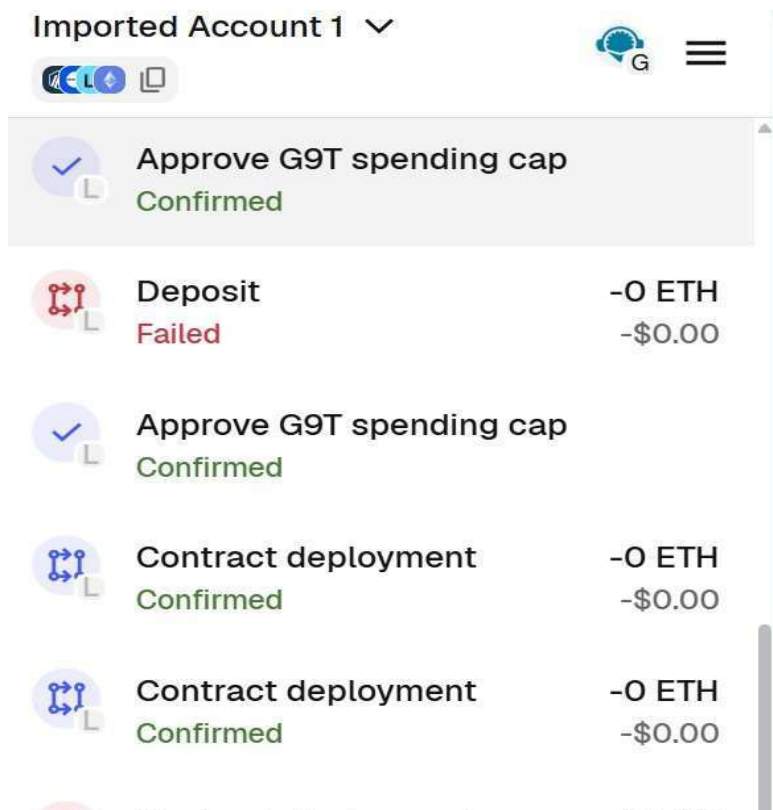
Also Deployed GenesisVault.sol



Give vault the permission to move tokens by using approve function in GenesisToken.sol



Deposit the tokens into the vault



Conclusion-

Ganache is a local Ethereum blockchain used for testing smart contracts. By connecting Ganache to MetaMask and then linking MetaMask with Remix IDE, developers can deploy and execute smart contracts in a simulated environment without using real cryptocurrency, enabling safe and efficient testing of blockchain applications.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 09

Aim: To implement a Private Ethereum Blockchain using Geth.

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO3:Implement smart contracts in Ethereum using different development frameworks.

EXPERIMENT 9

Name-Atharva Prabhu

Class-D20A

RollNo-45

Aim:To implement private Blockchain using Geth.

Theory:

A **blockchain** is a distributed and decentralized ledger that records transactions securely across multiple nodes.

Geth (Go Ethereum) is the official command-line client used to run an Ethereum node. It allows you to:

- Create a private blockchain
- Mine blocks
- Deploy smart contracts
- Perform transactions

Private Blockchain

A **private blockchain** is a restricted network where:

- Only authorized participants can join
- It is controlled by a single organization or group
- Faster and more efficient than public blockchains

Key Concepts

- **Genesis Block**

The first block of the blockchain. It defines:

- Chain configuration
- Initial accounts
- Difficulty level

- **Node**

A system running Geth that participates in the blockchain network.

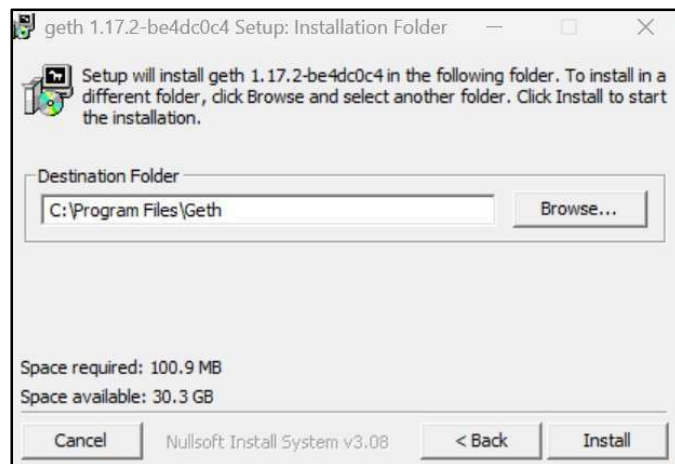
- **Mining**
The process of validating transactions and adding blocks to the chain.
- **Account**
Used to send/receive Ether in the network.

Steps to Implement Private Blockchain using Geth

1. Install Geth on your system (Windows/Linux).
2. Create a working directory for your private blockchain.
3. Create a genesis file to define the blockchain configuration (chain ID, difficulty, gas limit, etc.).
4. Initialize the blockchain using the genesis file.
5. Create a new account to use in the blockchain.
6. Start the Geth node with a custom network ID.
7. Open the Geth console to interact with the blockchain.
8. Start mining to generate blocks and Ether.
9. Check account balance to verify mining rewards.
10. Perform transactions between accounts.
11. View blockchain details like block number and block data.

Implementation:

Install Geth for windows from Ethereum website



Verify in command prompt if Geth is installed by using geth version

```
C:\Users\HOME\spandan>geth version
Geth
Version: 1.17.2-stable
Git Commit: be4dc0c4be2fe316dbdd0a73e48421f64978232f
Git Commit Date: 20260330
Architecture: amd64
Go Version: go1.25.7
Operating System: windows
GOPATH=
GOROOT=
```

Create project folder

```
C:\Users\HOME\spandan>mkdir private_blockchain

C:\Users\HOME\spandan>cd private_blockchain

C:\Users\HOME\spandan\private_blockchain>
```

Create a genesis.json file in notepad

```
C:\Users\HOME\spandan\private_blockchain>notepad genesis.json
```

```
{
  "config": {
    "chainId": 1234,
    "homesteadBlock": 0,
    "eip150Block": 0,
    "eip155Block": 0,
    "eip158Block": 0,
    "byzantiumBlock": 0,
    "constantinopleBlock": 0,
    "petersburgBlock": 0,
    "istanbulBlock": 0
  },
  "difficulty": "2000",
  "gasLimit": "2100000",
  "alloc": {}
}
```

Initialize the private blockchain using geth

```
C:\Users\HOME\spandan\private_blockchain>geth --datadir data init genesis.json
INFO [04-07|15:09:44.624] Maximum peer count          ETH=50 total=50
WARN [04-07|15:09:44.640] Sanitizing cache to Go's GC limits provided=4096 updated=2018
INFO [04-07|15:09:44.642] Set global gas cap          cap=50,000,000
INFO [04-07|15:09:44.642] Initializing the KZG library backend=gokzg
INFO [04-07|15:09:44.650] Using pebble as the backing database
INFO [04-07|15:09:44.650] Allocated cache and file handles database=C:\Users\HOME\spandan\private_blockchain\data\geth\chaindata
ata cache=1009.00MiB handles=8192
INFO [04-07|15:09:44.751] Opened ancient database      database=C:\Users\HOME\spandan\private_blockchain\data\geth\chaindata
ata\ancient\chain readonly=false
INFO [04-07|15:09:44.752] Opened Era store            datadir=C:\Users\HOME\spandan\private_blockchain\data\geth\chaindata
ta\ancient\chain\era
INFO [04-07|15:09:44.752] State schema set to default scheme=path
WARN [04-07|15:09:44.752] Head block is not reachable
INFO [04-07|15:09:44.752] Load database journal from disk
INFO [04-07|15:09:44.760] Opened ancient database      database=C:\Users\HOME\spandan\private_blockchain\data\geth\chaindata
ata\ancient\state readonly=false
INFO [04-07|15:09:44.760] State snapshot generator is not found
INFO [04-07|15:09:44.761] Starting snapshot generation root=56e81f..63b421 accounts=0 slots=0 storage=0.00B dangling=0 elapsed=545.4us
INFO [04-07|15:09:44.761] Initialized path database    trieache=16.00MiB statecache=16.00MiB buffer=64.00MiB state-hist
ry="last 90000 blocks" journal-dir=C:\Users\HOME\spandan\private_blockchain\data\geth\triedb
INFO [04-07|15:09:44.761] Resuming snapshot generation root=56e81f..63b421 accounts=0 slots=0 storage=0.00B dangling=0 elapsed=545.4us
INFO [04-07|15:09:44.761] Writing custom genesis block
INFO [04-07|15:09:44.761] Generated snapshot           accounts=0 slots=0 storage=0.00B dangling=0 elapsed=1.154ms
INFO [04-07|15:09:44.767] Successfully wrote genesis state database=chaindata hash=6f14b1..a2805e
```

Create an account

```
C:\Users\HOME\spandan\private_blockchain>geth --datadir data account new
INFO [04-07|15:12:54.245] Maximum peer count          ETH=50 total=50
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key: 0x0C0F4Cd470564897545A41A3C8243e60430ce94B
Path of the secret key file: data\keystore\UTC--2026-04-07T09-43-19.390875900Z--0c0f4cd470564897545a41a3c8243e60430ce94b

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!
```

Start Geth Node

```
C:\Users\HOME\spandan\private_blockchain>geth --datadir data --networkid 1234 console
INFO [04-07|16:50:01.532] Maximum peer count          ETH=50 LES=0 total=50
INFO [04-07|16:50:01.595] Set global gas cap          cap=25000000
INFO [04-07|16:50:01.603] Allocated trie memory caches clean=256.00MiB dirty=256.00MiB
INFO [04-07|16:50:01.611] Allocated cache and file handles database=C:\Users\HOME\spandan\private_blockchain\data\geth\chaindata
ata cache=512.00MiB handles=8192
INFO [04-07|16:50:01.665] Opened ancient database      database=C:\Users\HOME\spandan\private_blockchain\data\geth\chaindata\ancient
INFO [04-07|16:50:03.008] Initialised chain configuration config="{ChainID: 1234 Homestead: 0 DAO: <nil> DAOsupport: false EIP150: 0 EIP155: 0 EIP158: 0 Byzantium: 0 Constantinople: 0 Petersburg: 0 Istanbul: 0, Muir Glacier: <nil>, YOLO v2: <nil>, Engine: unknown}"
INFO [04-07|16:50:03.034] Disk storage enabled for ethash caches dir=C:\Users\HOME\spandan\private_blockchain\data\geth\ethash count=3
INFO [04-07|16:50:03.046] Disk storage enabled for ethash DAGs dir=C:\Users\HOME\AppData\Local\Ethash count=2
INFO [04-07|16:50:03.056] Initialising Ethereum protocol versions="[65 64 63]" network=1234 dbversion=<nil>
WARN [04-07|16:50:03.067] Upgrade blockchain database version from=<nil> to=8
INFO [04-07|16:50:03.077] Loaded most recent local header number=0 hash="6f14b1..a2805e" td=2000 age=57y1mo11h
INFO [04-07|16:50:03.091] Loaded most recent local full block number=0 hash="6f14b1..a2805e" td=2000 age=57y1mo11h
INFO [04-07|16:50:03.104] Loaded most recent local fast block number=0 hash="6f14b1..a2805e" td=2000 age=57y1mo11h
INFO [04-07|16:50:03.119] Regenerated local transaction journal transactions=0 accounts=0
INFO [04-07|16:50:03.141] Allocated fast sync bloom size=512.00MiB
INFO [04-07|16:50:03.148] Initialized fast sync bloom items=0 errorrate=0.000 elapsed=0s
INFO [04-07|16:50:03.149] Starting peer-to-peer node instance=Geth/v1.9.25-stable-e7072729/windows-amd64/go1.15.6
INFO [04-07|16:50:03.190] New local node record seq=1 id=eeee1fa666f7c09 ip=127.0.0.1 udp=30303 tcp=30303
INFO [04-07|16:50:03.203] Started P2P networking self=enode://a3e0ec411b9c6c658245a289eb62ae41f93a67c9839cf01d06840f25ea35bd73417385b2f6b7c7fc18b1f111b097e6ac0981ace7aee219532c88296e496b93b0127.0.0.1:30303
INFO [04-07|16:50:03.206] IPC endpoint opened url=\\.\pipe\geth.ipc
INFO [04-07|16:50:03.319] Etherbase automatically configured address=0xa0c392e16b0f0e576683d2e5d51faFad2AcA6e4f
Welcome to the Geth JavaScript console!
```

Check accounts

```
> eth.accounts
["0xa0c392e16b0f0e576683d2e5d51fafad2aca6e4f"]
```

Unlock the account

```
> personal.unlockAccount(eth.accounts[0])
Unlock account 0xa0c392e16b0f0e576683d2e5d51fafad2aca6e4f
Passphrase: INFO [04-07|16:56:23.593] Looking for peers      peercount=0 tried=66 static=0

true
```

Start

Mining

```
> miner.start(1)
INFO [04-07|16:57:49.057] Updated mining threads      threads=1
INFO [04-07|16:57:49.063] Transaction pool price threshold updated price=1000000000
null
> INFO [04-07|16:57:49.070] Commit new mining work      number=1 sealhash="cb5512..b7dc49" uncles=0 txs=0 gas=0 fees=0 elapsed=1.007ms
INFO [04-07|16:57:50.937] Generating DAG in progress    epoch=0 percentage=0 elapsed=1.106s
INFO [04-07|16:57:52.040] Generating DAG in progress    epoch=0 percentage=1 elapsed=2.210s
INFO [04-07|16:57:53.133] Generating DAG in progress    epoch=0 percentage=2 elapsed=3.302s
INFO [04-07|16:57:54.226] Generating DAG in progress    epoch=0 percentage=3 elapsed=4.396s
INFO [04-07|16:57:55.338] Generating DAG in progress    epoch=0 percentage=4 elapsed=5.507s
INFO [04-07|16:57:55.981] Looking for peers             peercount=0 tried=59 static=0
INFO [04-07|16:57:56.461] Generating DAG in progress    epoch=0 percentage=5 elapsed=6.631s
INFO [04-07|16:57:57.583] Generating DAG in progress    epoch=0 percentage=6 elapsed=7.753s
INFO [04-07|16:57:58.706] Generating DAG in progress    epoch=0 percentage=7 elapsed=8.876s
INFO [04-07|16:57:59.803] Generating DAG in progress    epoch=0 percentage=8 elapsed=9.973s
```

```
INFO [04-07|20:45:12.391] Commit new mining work      number=2 sealhash="4f33f6..a9b2e0" uncles=0 txs=0 gas=0 fees=0 elapsed=998µs
INFO [04-07|20:45:12.401] ⚡ mined potential block      number=1 hash="e781cd..21e1bb"
INFO [04-07|20:45:12.916] Successfully sealed new block number=2 sealhash="4f33f6..a9b2e0" hash="5590c8..45f96b" elapsed=524.571ms
INFO [04-07|20:45:12.916] Commit new mining work      number=3 sealhash="2fe669..f8004b" uncles=0 txs=0 gas=0 fees=0 elapsed=0s
INFO [04-07|20:45:12.926] ⚡ mined potential block      number=2 hash="5590c8..45f96b"
INFO [04-07|20:45:15.455] Successfully sealed new block number=3 sealhash="2fe669..f8004b" hash="c5becd..86cb9c" elapsed=2.539s
INFO [04-07|20:45:15.455] Commit new mining work      number=4 sealhash="b6f543..2d8eb5" uncles=0 txs=0 gas=0 fees=0 elapsed=0s
INFO [04-07|20:45:15.466] ⚡ mined potential block      number=3 hash="c5becd..86cb9c"
INFO [04-07|20:45:17.874] Successfully sealed new block number=4 sealhash="b6f543..2d8eb5" hash="4bbde3..c3cd16" elapsed=2.419s
```

Check account balance

```
=0 fees=0 elapsed=10.012ms
> eth.getBalance(eth.accounts[0])
2800000000000000000000000
> INFO [04-07|20:57:20.259] Looking for peers
```

Stop mining

```
> miner.stop()
null
> INFO [04-07|20:57:52.482] Looking for peers      peercount=0 tried=43 static=0
INFO [04-07|20:58:02.489] Looking for peers      peercount=0 tried=50 static=0
INFO [04-07|20:58:12.493] Looking for peers      peercount=0 tried=58 static=0
```

To Send transaction

Created another account

```

C:\Users\HOME\spandan\private_blockchain>geth --datadir data account new
INFO [04-07|21:07:14.350] Maximum peer count      ETH=50 LES=0 total=50
Your new account is locked with a password. Please give a password. Do not forget this password.
Password:
Repeat password:

Your new key was generated

Public address of the key:  0xc5Ab7B0f31c6304828eeC594b9eA37Af6D07f845
Path of the secret key file: data\keystore\UTC--2026-04-07T15-37-22.772205900Z--c5ab7b0f31c6304828eeC594b9eA37af6d07f845

- You can share your public address with anyone. Others need it to interact with you.
- You must NEVER share the secret key with anyone! The key controls access to your funds!
- You must BACKUP your key file! Without the key, it's impossible to access account funds!
- You must REMEMBER your password! Without the password, it's impossible to decrypt the key!

```

Now we have 2 accounts

```

> eth.accounts
["0xa0c392e16b0f0e576683d2e5d51fafad2aca6e4f", "0xc5ab7b0f31c6304828eeC594b9eA37af6d07f845"]
> INFO [04-07|21:09:10.220] Looking for peers      peercount=0 tried=22 static=0

```

Unlock the account 0 and send transaction to account 1

```

> personal.unlockAccount(eth.accounts[0])
Unlock account 0xa0c392e16b0f0e576683d2e5d51fafad2aca6e4f
Passphrase: INFO [04-07|21:16:11.421] Looking for peers      peercount=0 tried=57 static=0

true
> INFO [04-07|21:16:22.171] Looking for peers      peercount=0 tried=40 static=0
> eth.sendTransaction({
.....   from: eth.accounts[0],
.....   to: eth.accounts[1],
.....   value: web3.toWei(1, "ether")
..... })
INFO [04-07|21:16:31.327] Setting new local account      address=0xa0c392e16b0f0e576683d2e5d51FaFAd2AcA6e4f
INFO [04-07|21:16:31.337] Submitted transaction          fullhash=0x61629c85ab48b216143d2e288f5c6c30198fd6e81416315c516dcbd5d7fa6982
5d7fa6982 recipient=0xc5Ab7B0f31c6304828eeC594b9eA37Af6D07f845
"0x61629c85ab48b216143d2e288f5c6c30198fd6e81416315c516dcbd5d7fa6982"
> INFO [04-07|21:16:32.176] Looking for peers      peercount=0 tried=21 static=0
INFO [04-07|21:16:42.293] Looking for peers      peercount=0 tried=37 static=0
INFO [04-07|21:16:52.437] Looking for peers      peercount=0 tried=43 static=0
INFO [04-07|21:17:02.442] Looking for peers      peercount=0 tried=22 static=0

```

Start mining

```

> miner.start(1)
INFO [04-07|21:19:03.349] Updated mining threads      threads=1
INFO [04-07|21:19:03.355] Transaction pool price threshold updated price=1000000000
INFO [04-07|21:19:03.361null] Com
mit new> mining work      number=23 sealhash="dcf750..5b31fd" uncles=0 txs=0 gas=0 fees=0 elapsed=0s
INFO [04-07|21:19:03.375] Commit new mining work      number=23 sealhash="960b1a..081554" uncles=0 txs=1 gas=21000 fees=2
.1e-05 elapsed=13.991ms
INFO [04-07|21:19:03.790] Looking for peers      peercount=0 tried=32 static=0
INFO [04-07|21:19:14.580] Looking for peers      peercount=0 tried=36 static=0

```

Stop mining

```

> miner.stop()
null
> INFO [04-07|21:21:36.229] Looking for peers

```

Account Balances

```
> eth.getBalance(eth.accounts[0])  
11300000000000000000  
> INFO [04-07|21:24:18.456] Looking for peers peercount=0 tried=45 static=0  
> eth.getBalance(eth.accounts[1])  
1000000000000000000
```

Transaction was successful as balance of account 0 has decreased and balance of account 1 is 1 ether which was transferred to account 1.

To fetch the details of the latest block mined

[illegible]

Checking the admin node info

```

> admin.nodeInfo
{
  enode: "enode://a3e0ec411b9c6c658245a289eb62ae41f93a67c9839cf01d06840f25ea35bd73417385b2fe1b7c7fc18b1f111b097e6ac0981ace7aee219532c88296e496b93b049.43.24.168:30303",
  enr: "enr:-Je4QPY4FL-2hvTHUIROV8WfA5vt8dPiMOX_zviaV0cVVt5hUajzdpP7ib6n-Uh_Ug49_Gu5mJbhkBeILLiMjM6DJ4Q0g2V0aMfGhI3Zu0uAgmLkgnY0gmlwhDErGKiJc2VjcDI1NmsxoQ0j40x8G5xsZYJFoonrYq58-TpnyY0c8B8GhA8L6jW9c4N0Y3CCdL-DdWIRwgnZF",
  id: "eeeeelfa666f7c09061bfc3f965eab385282298806e745cd584c1b3ecd5f4261",
  ip: "49.43.24.168",
  listenAddr: "[::]:30303",
  name: "Geth/v1.9.25-stable-e7872729/windows-amd64/go1.15.6",
  ports: {
    discovery: 30303,
    listener: 30303
  },
  protocols: {
    eth: {
      config: {
        byzantiumBlock: 0,
        chainId: 1234,
        constantinopleBlock: 0,
        eip150Block: 0,
        eip150Hash: "0x0000000000000000000000000000000000000000000000000000000000000000",
        eip155Block: 0,
        eip158Block: 0,
        homesteadBlock: 0,
        istanbulBlock: 0,
        petersburgBlock: 0
      },
      difficulty: 7522001,
      genesis: "0x6f14b168d12c82428b52b485bddd66c77c8daf3c38dcc5c2678d03d260a2805e",
      head: "0xa2f12559d9f4bca9134f88ac1485816862f83f12e4e29d4a168bf0de50bf2cc6",
    }
  }
}

```

Conclusion:

A private blockchain was successfully implemented using Geth. The blockchain was initialized using a custom genesis block, and accounts were created for transactions. Mining was performed to generate Ether, and a transaction between accounts was successfully executed and verified. This demonstrates the working of a decentralized blockchain network in a controlled environment.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 10

Aim: Explore the Cryptocurrency Landscape

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

EXPERIMENT 10

Name-Atharva Prabhu

Class-D20A

Roll no-45

Aim:Explore the Cryptocurrency Landscape

Theory:

This experiment explores the **cryptocurrency landscape** by combining real-world market data analysis with blockchain technology research. The focus asset for this analysis is **Chainlink (LINK)**, and data was collected from **three reputable sources** as per the lab requirement:

1. **CoinGecko API** – Provided historical price data, market capitalization, and trading volumes in a structured JSON format, enabling direct processing in Python.
2. **CoinMarketCap** – Offered real-time and historical price charts, circulating supply, and market rank for visual cross-verification.
3. **Crypto.com** – Supplied quick-access market statistics, simplified price trends, and cryptocurrency news updates.

Cryptocurrency Landscape

1. **Definition** – Cryptocurrencies are decentralized digital assets built on blockchain networks, enabling secure, transparent, and immutable transactions without central authority.
2. **Market Diversity** – Thousands of cryptocurrencies exist, each serving unique roles:
 - **Store of Value** – Bitcoin, Litecoin.
 - **Utility Tokens** – Chainlink, Ethereum.
 - **Stablecoins** – USDT, USDC.
 - **Governance Tokens** – UNI, AAVE.
3. **Volatility** – Prices are highly sensitive to market sentiment, technological developments, and regulatory updates.
4. **Data Analysis Importance** – Tracking historical trends, market capitalization, and trading volume is essential for understanding asset performance.

5. **Advancements** – Growing use of **DeFi platforms**, **NFT ecosystems**, **layer-2 scaling**, and **cross-chain protocols**.

Data Collection Process

The program used the **CoinGecko API** to automate historical data fetching for Chainlink. The retrieved data was processed with **Pandas** to compute:

- **Daily Returns** – Percentage change between consecutive days to measure volatility.
- **Moving Averages (MA7, MA30)** – Short-term and long-term trend indicators.
- **Cumulative Returns** – Growth factor over time.
- **Rolling Volatility** – Short-term risk measurement based on daily return fluctuations.

Visualization & Dashboard

Using **Matplotlib**, a six-panel dashboard was created to display:

1. **Price Trend** – Daily price movement over the selected period.
2. **Price vs Volume** – Correlation between trading activity and price changes.
3. **Daily Returns Distribution** – Statistical spread of daily gains/losses.
4. **Moving Averages** – Technical analysis trend lines for price movement.
5. **Cumulative Returns** – Overall growth performance.
6. **Rolling Volatility** – Market risk trends over time.

Technological Analysis

Beyond market data, the study covered **advancements in blockchain technology**, including:

- **Consensus Mechanisms** – PoW (Proof of Work), PoS (Proof of Stake), and hybrid systems.
- **Scalability Solutions** – Layer-2 protocols, sharding, and sidechains to increase throughput.
- **Interoperability Protocols** – Chainlink's **CCIP**, Polkadot, and Cosmos enabling cross-chain data exchange.
- **Privacy Features** – Zero-knowledge proofs (zk-SNARKs) and ring signatures to ensure transaction confidentiality.

Chainlink's **roadmap** highlights staking implementation, advanced oracle features, and enhanced cross-chain capabilities, aligning with industry trends toward **decentralization, interoperability, and security**.

Market Trends & Adoption

- **Growing Merchant Acceptance** – More businesses accepting cryptocurrency payments.
- **Wallet Growth** – Increasing downloads and installations of crypto wallets.
- **Network Activity** – Higher transaction volumes and active addresses, signaling strong adoption.
- **Integration with Traditional Finance** – Partnerships between crypto platforms and fintech/payment providers.
- **Regulatory Developments** – Countries exploring frameworks for safe cryptocurrency adoption.

Utilizing Api for analysis and visualization :

Code:

```
import requests
import matplotlib.pyplot as plt
import pandas as pd

# ----- Fetch Data -----
def get_historical_data(coin_id, days='180', currency='usd'):
    url = f'https://api.coingecko.com/api/v3/coins/{coin_id}/market_chart'
    params = {'vs_currency': currency, 'days': days}
    response = requests.get(url, params=params)
    return response.json()

# ----- Dashboard Function -----
def chainlink_dashboard(days='180'):
    coin_id = 'chainlink'
```

```

data = get_historical_data(coin_id, days)

# Create DataFrame
prices_df = pd.DataFrame(data['prices'], columns=['timestamp', 'price'])
volumes_df = pd.DataFrame(data['total_volumes'], columns=['timestamp', 'volume'])

prices_df['timestamp'] = pd.to_datetime(prices_df['timestamp'], unit='ms')
volumes_df['timestamp'] = pd.to_datetime(volumes_df['timestamp'], unit='ms')

df = pd.merge(prices_df, volumes_df, on='timestamp')
df.set_index('timestamp', inplace=True)

# Calculate extra metrics
df['returns'] = df['price'].pct_change()
df['MA7'] = df['price'].rolling(window=7).mean()
df['MA30'] = df['price'].rolling(window=30).mean()
df['cumulative'] = (1 + df['returns']).cumprod()
df['volatility'] = df['returns'].rolling(window=7).std() * 100

# ----- Plot Dashboard -----
fig, axes = plt.subplots(3, 2, figsize=(16, 12))
fig.suptitle(f'{coin_id.capitalize()} Dashboard (Last {days} Days)', fontsize=16, fontweight='bold')

# 1. Price Trend
axes[0, 0].plot(df.index, df['price'], color='blue')
axes[0, 0].set_title('Price Trend')
axes[0, 0].set_ylabel('Price (USD)')
axes[0, 0].set_xlabel('Year-Month') # Add xlabel
axes[0, 0].grid(True)

```

2. Price vs Volume

```
axes[0, 1].plot(df.index, df['price'], color='blue', label='Price')
axes[0, 1].bar(df.index, df['volume'], color='orange', alpha=0.3, label='Volume')
axes[0, 1].set_title('Price vs Volume')
axes[0, 1].set_xlabel('Year-Month') # Add xlabel
axes[0, 1].set_ylabel('Price (USD) / Volume') # Add ylabel
axes[0, 1].legend()
axes[0, 1].grid(True)
```

3. Daily Returns Distribution

```
axes[1, 0].hist(df['returns'].dropna(), bins=50, alpha=0.7, color='purple')
axes[1, 0].set_title('Daily Returns Distribution')
axes[1, 0].set_xlabel('Daily Return')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].grid(True)
```

4. Moving Averages

```
axes[1, 1].plot(df.index, df['price'], color='lightgray', label='Price')
axes[1, 1].plot(df.index, df['MA7'], color='blue', label='7-day MA')
axes[1, 1].plot(df.index, df['MA30'], color='red', label='30-day MA')
axes[1, 1].set_title('Moving Averages')
axes[1, 1].set_xlabel('Year-Month')
axes[1, 1].set_ylabel('Price (USD)')
axes[1, 1].legend()
axes[1, 1].grid(True)
```

5. Cumulative Returns

```
axes[2, 0].plot(df.index, df['cumulative'], color='green')
axes[2, 0].set_title('Cumulative Returns')
axes[2, 0].set_ylabel('Growth Factor')
```

```
axes[2, 0].set_xlabel('Year-Month')
```

```
axes[2, 0].grid(True)
```

```
# 6. Volatility
```

```
axes[2, 1].plot(df.index, df['volatility'], color='orange')
```

```
axes[2, 1].set_title('Rolling Volatility (7-Day)')
```

```
axes[2, 1].set_ylabel('Volatility (%)')
```

```
axes[2, 1].set_xlabel('Year-Month')
```

```
axes[2, 1].grid(True)
```

```
plt.tight_layout(rect=[0, 0, 1, 0.96])
```

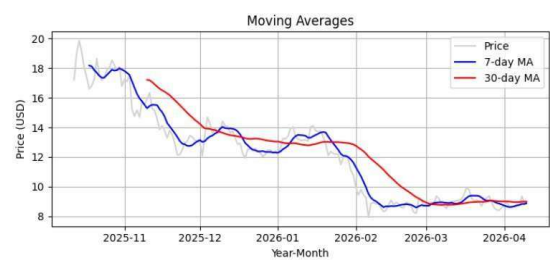
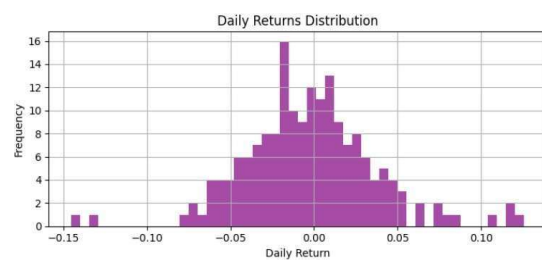
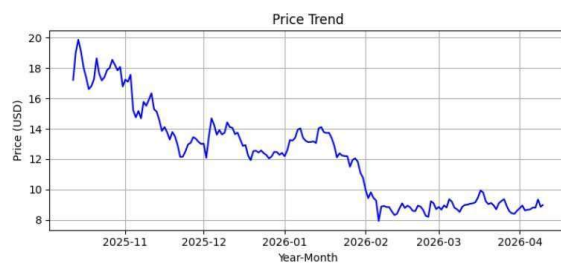
```
plt.subplots_adjust(hspace=0.5, wspace=0.3) # Adjust vertical and horizontal spacing
```

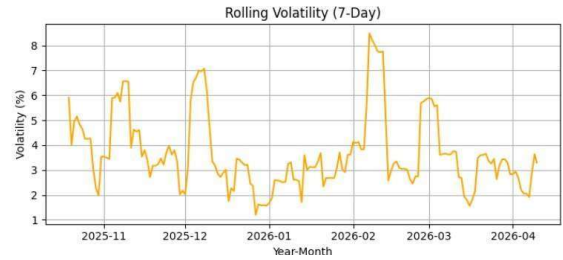
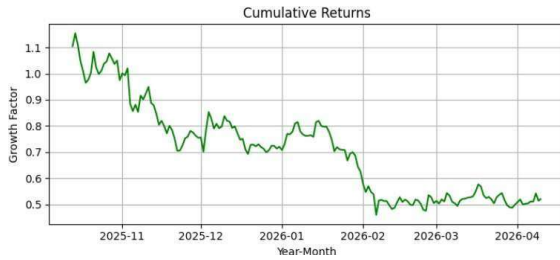
```
plt.show()
```

```
# ----- Run Dashboard -----
```

```
chainlink_dashboard('180')
```

Output:





Coingecko Website

Cryptocurrency Prices by Market Cap

The global cryptocurrency market cap today is \$2.53 Trillion, a ▲ 0.9% change in the last 24 hours. [Read more](#)

Category	Item	Price / Volume	Change (24h)
Market Summary	Market Cap	\$2,526,881,227,609	▲ 0.9%
	24h Trading Volume	\$87,100,760,179	
Trending	Zcash	\$363.04	▲ 11.1%
	Bitcoin	\$71,924.45	▲ 1.1%
	Siren	\$0.7399	▲ 29.8%
Top Gainers	RaveDAO	\$0.5855	▲ 92.0%
	Xphere	\$0.01634	▲ 63.7%
	Siren	\$0.7399	▲ 29.8%

Bitcoin BTC Price #1

\$72,037.64 ▲ 1.2% (24h)

1.0000 BTC ▲ 0.0%

24h Range: \$70,522.33 - \$72,495.25

Market Cap: \$1,441,734,490,650

Fully Diluted Valuation: \$1,441,734,490,650

24 Hour Trading Vol: \$35,707,938,832

Circulating Supply: 20,013,637

Total Supply: 20,013,637

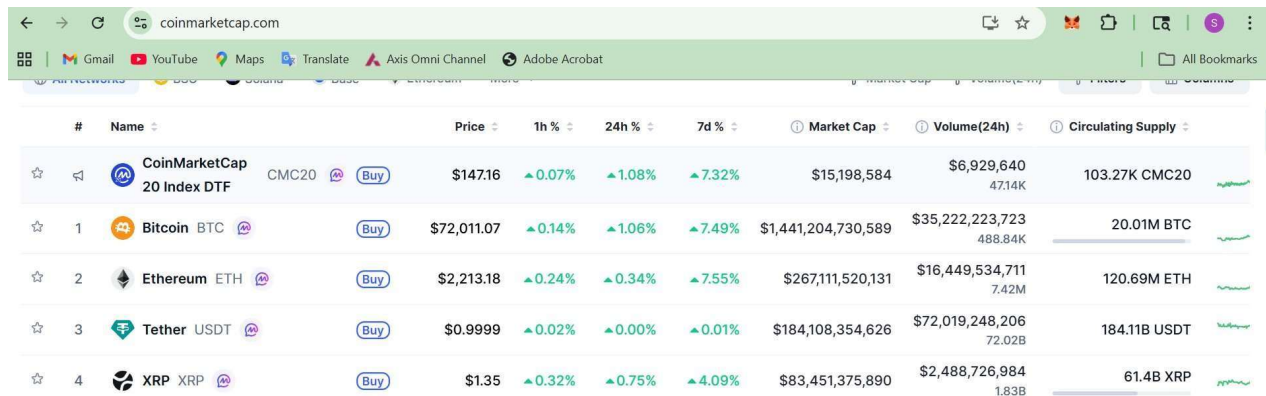
Max Supply: 21,000,000

Price Chart

Overview Markets Treasuries News Similar Coins Historical Data BTC Halving

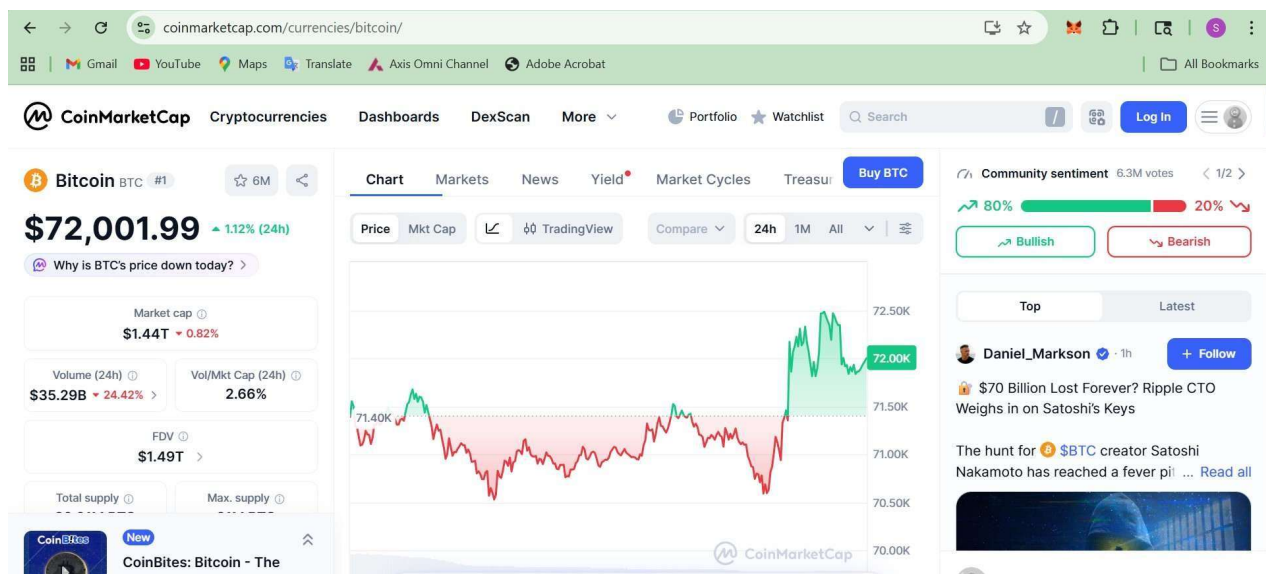
Price Compare 24H 7D 1M 3M YTD 1Y Max

CoinMarketCap website



This screenshot shows the top cryptocurrencies listed on the CoinMarketCap website. The table includes columns for rank, name, price, and percentage changes over 1h, 24h, and 7d. The top five cryptocurrencies are Bitcoin, Ethereum, Tether, XRP, and the CoinMarketCap 20 Index DTF.

#	Name	Price	1h %	24h %	7d %	Market Cap	Volume(24h)	Circulating Supply
	CoinMarketCap 20 Index DTF	\$147.16	▲0.07%	▲1.08%	▲7.32%	\$15,198,584	\$6,929,640	103.27K CMC20
1	Bitcoin BTC	\$72,011.07	▲0.14%	▲1.06%	▲7.49%	\$1,441,204,730,589	\$35,222,223,723	20.01M BTC
2	Ethereum ETH	\$2,213.18	▲0.24%	▲0.34%	▲7.55%	\$267,111,520,131	\$16,449,534,711	120.69M ETH
3	Tether USDT	\$0.9999	▲0.02%	▲0.00%	▲0.01%	\$184,108,354,626	\$72,019,248,206	184.11B USDT
4	XRP XRP	\$1.35	▲0.32%	▲0.75%	▲4.09%	\$83,451,375,890	\$2,488,726,984	61.4B XRP



Conclusion:

By combining automated market data processing, visual trend analysis, blockchain technology evaluation, and adoption metrics, this experiment presents a holistic view of the cryptocurrency ecosystem. It demonstrates how blockchain's technical foundations connect with real-world market behavior, providing insights into both current trends and future advancements in the digital asset industry.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801 : Blockchain & DLT Lab
Experiment 11

Aim: Implementation of Hyperledger Fabric Network and Asset Management using Chaincode

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO4:Develop applications in permissioned Hyperledger Fabric network.

EXPERIMENT 11

Name-Atharva Prabhu

Class-D20A

Rollno-45

Aim: To implement a Hyperledger Fabric network, deploy chaincode, and perform asset creation, querying, and transfer operations.

Theory:

Hyperledger Fabric is a **permissioned blockchain platform** developed by the Linux Foundation.

Unlike public blockchains, it is designed for **enterprise use**, where participants are known and trusted.

It provides:

- High scalability
- Privacy and confidentiality
- Modular architecture

Key Components of Fabric Network

a) Peers

- Nodes that maintain the ledger and execute transactions
- Two types:
 - **Endorsing peers** → simulate transactions
 - **Committing peers** → validate and commit transactions

b) Orderer

- Responsible for **ordering transactions** into blocks
- Ensures consistency across the network

c) Ledger

Each peer maintains a ledger with:

- **Blockchain** → immutable transaction log
- **World State** → current state of assets (stored in DB like CouchDB)

d) Channels

- Private communication layer between organizations
- Ensures **data isolation and confidentiality**

e) Membership Service Provider (MSP)

- Manages identities and authentication using digital certificates.

Chaincode

Chaincode is the **business logic** of the blockchain.

It defines:

- Rules for asset creation
- Validation of transactions
- Query and update operations

Chaincode is typically written in:

- Go
- JavaScript (Node.js)
- Java

Chaincode Lifecycle

Steps involved:

1. **Package** → Convert code into deployable format
2. **Install** → Install on peers
3. **Approve** → Organizations approve chaincode definition

4. **Commit** → Chaincode becomes active on channel

Transaction Flow in Fabric

1. Client sends transaction proposal
2. Endorsing peers simulate and sign results
3. Client sends endorsed transaction to orderer
4. Orderer creates block
5. Peers validate and commit to ledger

Procedure:

Prerequisite-Install WSL

Command:

wsl --install

```
spandan@LAPTOP-9JIMM8I3: ~
Ubuntu has been installed.
Launching Ubuntu...
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows user name.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: spandan
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

Welcome to Ubuntu 24.04.1 LTS (GNU/Linux 5.15.153.1-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Apr  9 14:40:23 UTC 2026

System load:  0.34               Processes:            47
Usage of /:   0.1% of 1006.85GB   Users logged in:     0
Memory usage: 14%               IPv4 address for eth0: 192.168.69.236
Swap usage:   0%

This message is shown once a day. To disable it please create the
/home/spandan/.hushlogin file.
spandan@LAPTOP-9JIMM8I3:~$
```

Step 1-Update the system

Command:

`sudo apt update`

`sudo apt upgrade -y`

```
spandan@LAPTOP-9JIMM8I3:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu noble InRelease
Get:2 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/universe amd64 Packages [15.0 MB]
Get:5 http://archive.ubuntu.com/ubuntu noble/universe Translation-en [5982 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 Components [3871 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 c-n-f Metadata [301 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Packages [269 kB]
Get:9 http://archive.ubuntu.com/ubuntu noble/multiverse Translation-en [118 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 Components [35.0 kB]
Get:11 http://archive.ubuntu.com/ubuntu noble/multiverse amd64 c-n-f Metadata [8328 B]
Get:12 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1881 kB]
Get:13 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [343 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
```

```
spandan@LAPTOP-9JIMM8I3:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
The following NEW packages will be installed:
  ethtool libllvm20 mesa-libgallium
The following packages have been kept back:
  libgl1-amber-dri libglapi-mesa
The following packages will be upgraded:
```

Explanation:

This step updates the system package index and upgrades all installed packages to their latest versions. It ensures that the system has the most recent security patches and compatible libraries required for Hyperledger Fabric. Running this step avoids dependency conflicts and installation failures later.

Step 2: Install Required Dependencies

Command:

```
sudo apt install jq curl git nodejs npm -y
```

```
node -v
```

```
npm -v
```

```
git --version
```

```
docker --version
```

```
docker ps
```

```
spandan@LAPTOP-9JIMM8I3:~$ sudo apt install jq curl git nodejs npm -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
curl is already the newest version (8.5.0-2ubuntu10.8).
curl set to manually installed.
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libwayland-server0 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
The following additional packages will be installed:
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ docker --version
```

```
Docker version 29.3.1, build c2be9cc
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ node -v
```

```
v18.19.1
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ npm -v
```

```
9.2.0
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME\$						

Explanation:

This step installs all necessary tools required to run Hyperledger Fabric. Docker is used to run blockchain components as containers, Git is used to clone repositories, and Node.js with npm is required for chaincode execution. Version checks confirm successful installation, while `docker ps` ensures Docker is running properly.

Step 3: Clone Fabric Samples

Command:

```
git clone https://github.com/hyperledger/fabric-samples
cd fabric-samples
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME$ git clone https://github.com/hyperledger/fabric-samples
Cloning into 'fabric-samples'...
remote: Enumerating objects: 15618, done.
remote: Counting objects: 100% (159/159), done.
remote: Compressing objects: 100% (101/101), done.
Receiving objects: 55% (8615/15618), 13.93 MiB | 887.00 KiB/s
```

Explanation:

This step downloads the official Hyperledger Fabric sample repository. It contains pre-built network configurations, scripts, and example chaincodes. Moving into this directory allows us to access all required files for setting up the blockchain network.

Step 4: Install Fabric Binaries and Images

Command:

```
curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ curl -sLO https://raw.githubusercontent.com/hyperledger/fabric/main/scripts/install-fabric.sh
chmod +x install-fabric.sh
./install-fabric.sh docker samples binary
./install-fabric.sh docker

Clone hyperledger/fabric-samples repo

==> Already in fabric-samples repo
fabric-samples v2.5.15 does not exist, defaulting to main. fabric-samples main branch is intended to work with recent versions of fabric.

Pull Hyperledger Fabric binaries

==> Downloading version 2.5.15 platform specific fabric binaries
==> Downloading: https://github.com/hyperledger/fabric/releases/download/v2.5.15/hyperledger-fabric-linux-amd64-2.5.15.tar.gz
==> Will unpack to: /mnt/c/Users/HOME/fabric-samples
```

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current	
			Dload	Upload	Total	Spent	Left	Speed
0	0	0	0	0	0	0	0	0
86	120M	86	104M	0	0	3094k	0	0:00:40 0:00:34 0:00:06 3290k

```

FABRIC_IMAGES: peer orderer ccenv baseos
====> Pulling fabric Images
====> ghcr.io/hyperledger/fabric-peer:2.5.15
2.5.15: Pulling from hyperledger/fabric-peer
b1cba2e842ca: Pull complete
10a7c8361220: Pull complete
d41d93850539: Pull complete
eb2bee6aa5bd: Pull complete
dbac02a8c9ad: Pull complete
f8c55d757df2: Pull complete
Digest: sha256:3e25adc31a757ee19dd8a24afdc9ac5aa46a86b3cb56fc93068b0e594706d6aa
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-peer:2.5.15
ghcr.io/hyperledger/fabric-peer:2.5.15
====> ghcr.io/hyperledger/fabric-orderer:2.5.15
2.5.15: Pulling from hyperledger/fabric-orderer
b1cba2e842ca: Already exists
94c191cca972: Pull complete
7f5f5563c22b: Pull complete
61c558a0dbbe: Pull complete
efe901203a90: Pull complete
e756cbd2164a: Pull complete
Digest: sha256:9972c209be47f45e377a24c88f2e3d21fae4dc224751c68bdd33f2e7a603ffa8
Status: Downloaded newer image for ghcr.io/hyperledger/fabric-orderer:2.5.15
ghcr.io/hyperledger/fabric-orderer:2.5.15

```

Explanation:

This step downloads and executes a script that installs Hyperledger Fabric binaries and required Docker images. It pulls essential components like peer, orderer, and certificate authority images. These components are necessary to create and manage the blockchain network.

Step 5: Verify Docker Images

Command:

`docker images | grep fabric`

```

spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ docker images | grep fabric
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
ghcr.io/hyperledger/fabric-baseos:2.5.15      1aad8b995da9      156MB      0B
ghcr.io/hyperledger/fabric-ca:1.5.17          0548296e36d4      244MB      0B
ghcr.io/hyperledger/fabric-ccenv:2.5.15       f80b289a3828      671MB      0B
ghcr.io/hyperledger/fabric-orderer:2.5.15     4fb4e60fb623      118MB      0B
ghcr.io/hyperledger/fabric-peer:2.5.15        d5150827e5b1      154MB      0B
hyperledger/fabric-baseos:2.5                 1aad8b995da9      156MB      0B
hyperledger/fabric-baseos:2.5.15              1aad8b995da9      156MB      0B
hyperledger/fabric-baseos:latest              1aad8b995da9      156MB      0B
hyperledger/fabric-ca:1.5                     0548296e36d4      244MB      0B
hyperledger/fabric-ca:1.5.17                  0548296e36d4      244MB      0B
hyperledger/fabric-ca:latest                  0548296e36d4      244MB      0B
hyperledger/fabric-ccenv:2.5                  f80b289a3828      671MB      0B
hyperledger/fabric-ccenv:2.5.15               f80b289a3828      671MB      0B
hyperledger/fabric-ccenv:latest               f80b289a3828      671MB      0B
hyperledger/fabric-orderer:2.5                4fb4e60fb623      118MB      0B
hyperledger/fabric-orderer:2.5.15             4fb4e60fb623      118MB      0B
hyperledger/fabric-orderer:latest             4fb4e60fb623      118MB      0B
hyperledger/fabric-peer:2.5                   d5150827e5b1      154MB      0B
hyperledger/fabric-peer:2.5.15                d5150827e5b1      154MB      0B
hyperledger/fabric-peer:latest                d5150827e5b1      154MB      0B

```

Explanation:

This command checks whether all required Hyperledger Fabric Docker images are successfully downloaded. It ensures that components such as peer and orderer are available before starting the network. Missing images may cause deployment errors.

Step 6: Set Environment Variables

Command:

```
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
peer version
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ export PATH=$PATH:/mnt/c/Users/HOME/fabric-samples/bin
export FABRIC_CFG_PATH=/mnt/c/Users/HOME/fabric-samples/config
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ peer version
peer:
  Version: v2.5.15
  Commit SHA: 83c7930
  Go version: go1.26.0
  OS/Arch: linux/amd64
  Chaincode:
    Base Docker Label: org.hyperledger.fabric
    Docker Namespace: hyperledger
```

Explanation:

This step sets environment variables required to run Fabric CLI commands. The PATH variable allows access to Fabric binaries, while FABRIC_CFG_PATH points to configuration files. Running the peer version verifies that the setup is successful.

Step 7: Start Network and Create Channel

Command:

```
cd ~/fabric-samples/test-network
./network.sh up createChannel
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples$ cd test-network
./network.sh up createChannel
Using docker and docker-compose
Creating channel 'mychannel'.
If network is not up, starting nodes with CLI timeout of '5' tries and CLI delay of '3' seconds and using d
to from 'cryptogen'
```

```

LOCAL_VERSION=v2.5.15
DOCKER_IMAGE_VERSION=v2.5.15
/mnt/c/Users/HOME/fabric-samples/test-network/./bin/cryptogen
Generating certificates using cryptogen tool
Creating Org1 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org1.yaml --output=organizations
org1.example.com
+ res=0
Creating Org2 Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-org2.yaml --output=organizations
org2.example.com
+ res=0
Creating Orderer Org Identities
+ cryptogen generate --config=./organizations/cryptogen/crypto-config-orderer.yaml --output=organizations
+ res=0
Generating CCP files for Org1 and Org2
[+] up 7/7
✓Network fabric_test Created 2.7ss
✓Volume compose_orderer.example.com Created 2.6ss
✓Volume compose_peer0.org1.example.com Created 0.2ss
✓Volume compose_peer0.org2.example.com Created 0.7ss
✓Container orderer.example.com Started 67.8s
✓Container peer0.org2.example.com Started 66.6s
✓Container peer0.org1.example.com Started 68.5s
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS
MES
246fe6a8d014 hyperledger/fabric-peer:latest "peer node start" About a minute ago Up 44 seconds 0.0.0
0.9851->9851/tcp 0.1.9851->9851/tcp 0.0.0.0.9851->9851/tcp 0.1.9851->9851/tcp

```

Explanation:

This step initializes the blockchain network by starting Docker containers for peers and orderers. It also creates a communication channel where transactions will occur. Certificates and configurations are automatically generated during this process.

Step 8: Deploy Chaincode

Command:

```

./network.sh deployCC \
  -ccn basic \
  -ccp ../asset-transfer-basic/chaincode-javascript \
  -ccl javascript

```

```

spandan@LAPTOP-9JIM8I3: /mnt/c/Users/HOME/fabric-samples/test-network$ ./network.sh deployCC -ccn basic -ccp ../asset-transfer-ba
sic/chaincode-javascript -ccl javascript
Using docker and docker-compose
deploying chaincode on channel 'mychannel'
executing with the following
- CHANNEL_NAME: mychannel
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
- CC_SEQUENCE: auto
- CC_END_POLICY: NA
- CC_COLL_CONFIG: NA
- CC_INIT_FCN: NA
- DELAY: 3
- MAX_RETRY: 5
- VERBOSE: false
executing with the following
- CC_NAME: basic
- CC_SRC_PATH: ../asset-transfer-basic/chaincode-javascript
- CC_SRC_LANGUAGE: javascript
- CC_VERSION: 1.0
+ '[' false = true ']'
+ peer lifecycle chaincode package basic.tar.gz --path ../asset-transfer-basic/chaincode-javascript --lang node --label basic_1.0
+ res=0
Chaincode is packaged
Installing chaincode on peer0.org1...

```

Explanation:

This step deploys the smart contract (chaincode) onto the blockchain network. The chaincode defines the business logic for asset operations such as creation, transfer, and querying. Once deployed, it is ready to interact with the ledger.

Step 9: Set Peer Environment

Command:

```
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export
CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export
CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

```
spandan@LAPTOP-9JIMM8I3: /mnt/c/Users/HOME/fabric-samples/test-network$ export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export PATH=$PATH:$HOME/fabric-samples/bin
export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"
export CORE_PEER_TLS_ROOTCERT_FILE=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt
export CORE_PEER_MSPCONFIGPATH=$HOME/fabric-samples/test-network/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp
export CORE_PEER_ADDRESS=localhost:7051
```

Explanation:

This step configures the identity and communication settings of the peer node. It specifies organization details, TLS certificates, and network addresses. These settings are required for secure interaction with the blockchain network.

Step 10: Initialize Ledger

Command:

peer chaincode invoke ... InitLedger

```
spandan@LAPTOP-9JIMM8I3: /mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function": "InitLedger", "Args": []}'
2026-04-09 17:50:03.612 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200
```

Explanation:

This step initializes the blockchain ledger with default sample assets. It writes initial data to the ledger, which can later be queried or modified. This operation is a transaction that gets recorded on the blockchain.

Step 11: Query All Assets**Command:**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'  
[{"AppraisedValue":500,"Color":"blue","ID":"asset2","Owner":"Tom","Size":10}]
```

Explanation:

This step retrieves all assets stored in the ledger. It performs a read-only operation and displays current data without modifying the blockchain. It helps verify that the ledger has been initialized correctly.

Step 12: Read Specific Asset**Command:**

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset5"]}'
```

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset2"]}'  
{"AppraisedValue":500,"Color":"blue","ID":"asset2","Owner":"Tom","Size":10}
```

Explanation:

This step retrieves detailed information of a specific asset using its ID. It demonstrates how individual records can be accessed from the ledger. This is also a read-only operation.

Step 13: Create New Asset

Command:

peer chaincode invoke ... CreateAsset

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
-c '{"function":"CreateAsset","Args":["asset1","car","blue","Tom","500"]}'
2026-04-09 18:08:34.702 UTC 0001 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset1","Color":"car","Size":null,"Owner":"Tom","AppraisedValue":500}
```

Explanation:

This step creates a new asset on the blockchain with specified attributes such as ID, color, size, owner, and value. The owner name is set to Manav, which confirms that the transaction was performed by the user. This operation updates the ledger.

Step 14: Transfer Asset

Command:

peer chaincode invoke ... TransferAsset

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls --cafile $ORDERER_CA \
-C mychannel -n basic \
--peerAddresses localhost:7051 \
--tlsRootCertFiles $CORE_PEER_TLS_ROOTCERT_FILE \
--peerAddresses localhost:9051 \
--tlsRootCertFiles $PEER0_ORG2_CA \
--waitForEvent \
-c '{"function":"TransferAsset","Args":["asset2","Jerry"]}'
2026-04-09 18:23:54.916 UTC 0001 INFO [chaincodeCmd] ClientWait -> txid [9729dfdb66a176d035db287b9b104d582d2ff8822ee56b79d14499c4abeb7860] committed with status (VALID) at localhost:9051
2026-04-09 18:23:54.933 UTC 0002 INFO [chaincodeCmd] ClientWait -> txid [9729dfdb66a176d035db287b9b104d582d2ff8822ee56b79d14499c4abeb7860] committed with status (VALID) at localhost:7051
2026-04-09 18:23:54.934 UTC 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery -> Chaincode invoke successful. result: status:200 payload:{"ID":"asset2","Color":"Jerry"}
```

Explanation:

This step transfers ownership of an asset from one user to another. It updates the owner field in the ledger. This demonstrates how blockchain ensures secure and traceable asset transfers.

Step 15: Stop Network

Command:

`./network.sh down`

```
spandan@LAPTOP-9JIMM8I3:/mnt/c/Users/HOME/fabric-samples/test-network$ ./network.sh down
Using docker and docker-compose
Stopping network
[+] down 7/7
✓Container peer0.org2.example.com      Removed
✓Container orderer.example.com         Removed
✓Container peer0.org1.example.com      Removed
✓Volume compose_orderer.example.com   Removed
✓Volume compose_peer0.org2.example.com Removed
✓Network fabric_test                  Removed
✓Volume compose_peer0.org1.example.com Removed
```

Explanation:

This step stops all running Docker containers and shuts down the Hyperledger Fabric network. It frees system resources and safely ends the experiment.

Conclusion:

This experiment demonstrated how blockchain networks work using Hyperledger Fabric and how transactions are securely executed. The experiment also helped in understanding real-time asset management using smart contracts.



**VIVEKANAND EDUCATION SOCIETY'S
INSTITUTE OF TECHNOLOGY
Department of Information Technology
Academic Year: 2025-26**

ITC801: Blockchain and DLT Lab
Experiment 12

Aim: Mini Project

Roll No.	45
Name	Atharva Prabhu
Class	D20A
Subject	ITC801: Blockchain and DLT Lab
CO Mapped	CO1:Describe the basic concept of Blockchain and Distributed Ledger Technology. CO2:Interpret the knowledge of the Bitcoin network, nodes, keys, wallets and transactions CO3:Implement smart contracts in Ethereum using different development frameworks. CO4:Develop applications in permissioned Hyperledger Fabric network. CO5:Interpret different Crypto assets and Crypto currencies CO6:Analyze the use of Blockchain with AI, IoT and Cyber Security using case studies.

Chain Pulse

- **PROBLEM STATEMENT** :Real-Time On-Chain Event Indexer & Notification System
- **TEAM MEMBERS:**

MEMBER 1- Aryan Dangat 19

MEMBER 2 - Spandan Deb 20

MEMBER 3 - Sanket More 35

MEMBER 4 - Atharva Prabhu 45

○



PROPOSED SOLUTION



PROBLEM STATEMENT: RAW DATA, MISSED SIGNALS

Although publicly available, monitoring meaningful on-chain events in real time is fragmented and non-scalable. Teams rely on block explorers or basic listeners that expose raw transactions without context or reliability, resulting in noisy alerts, missed signals, and slow decisions—highlighting the need for a system that delivers timely, reliable, and actionable insights

PROPOSED SOLUTION: THE GAME-CHANGER

- **Unified Intelligence:** Query any contract or event from one rule-based interface.
- **Signal-First:** Filters blockchain noise into context-aware, actionable alerts.
- **Reorg-Safe:** Finality-aware architecture ensures accuracy in volatile conditions.
- **Instant Delivery:** Real-time webhooks, Telegram, or email notifications.
- **Lean Scaling:** Selective indexing slashes costs while maintaining performance.

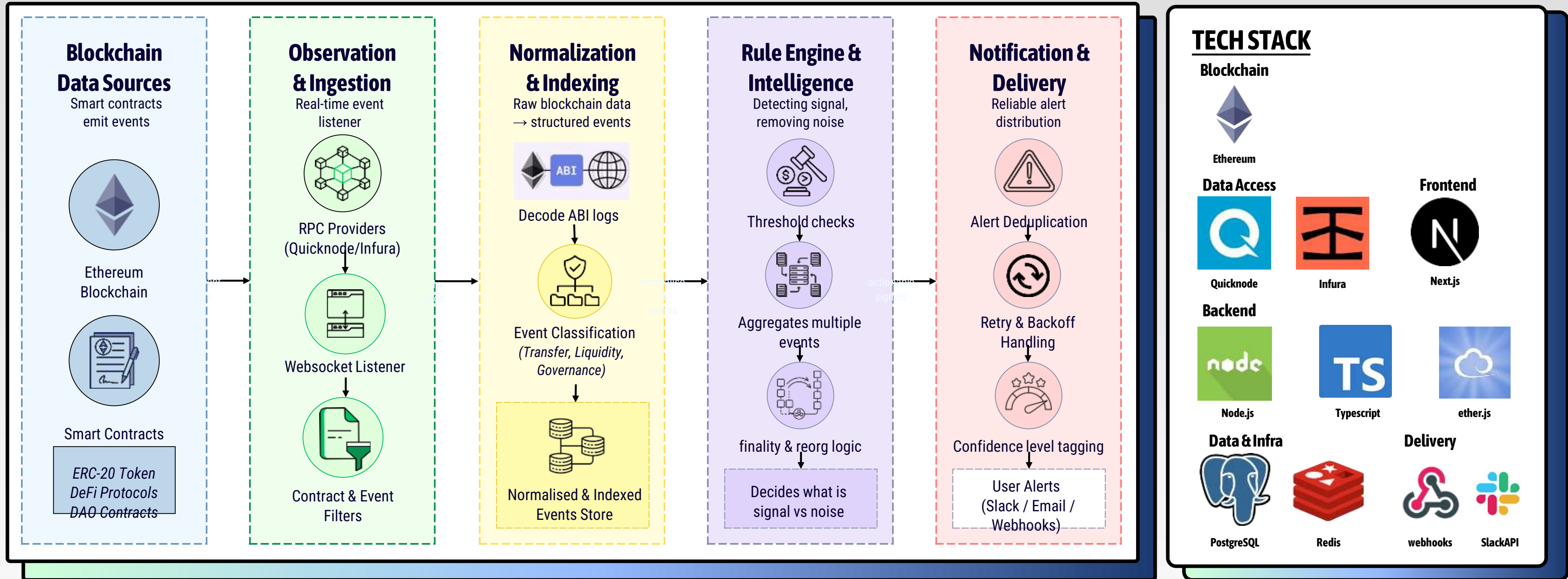
UNIQUE VALUE PROPOSITION

- **Finality-Aware Alerts**, not instant but unreliable notifications
- **Reorg-Safe by Design**, avoiding false positives
- **Index What Matters**, not the entire chain
- **Insights over Events**, optimized for human decisions
- **Built for Real-World Web3**, not demos

PROBLEM TO SOLUTION: Why Existing Monitoring Fails — And How We Fix It

<u>Problem</u>	<u>Existing System (The "Why it Fails")</u>	<u>Our Solution</u>
On-Chain Noise	Surfacing raw events without context or prioritization.	Signal-first event filtering.
Data Reliability	Vulnerable to reorgs and confirmation uncertainty.	Finality-aware processing.
Alert Fatigue	Manual filtering of high-volume, low-value data.	Smart event aggregation.
Lack of Context	No explainability; difficult to verify data sources.	Deterministic, auditable rules.
Operational Delay	Manual, time-consuming verification/monitoring.	Automated real-time alerts.

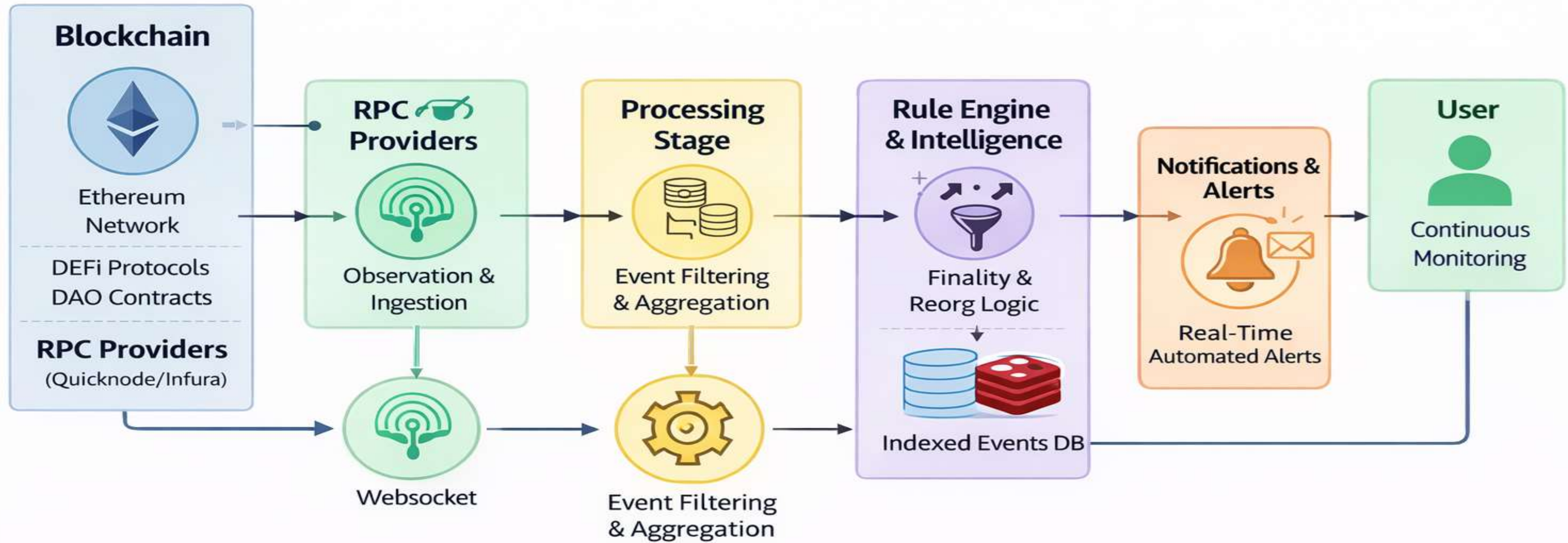
TECHNICAL APPROACH



- **Ethereum** → Most mature smart-contract ecosystem; trusted on-chain event source
- **QuickNode + Infura** → Low-latency streams with high availability & redundancy
- **Node.js + TypeScript** → Fast, safe, event-driven backend for real-time processing
- **ethers.js** → Reliable Ethereum contract interaction & ABI decoding

- **PostgreSQL** → ACID-safe, query-efficient storage for indexed on-chain data
- **Redis** → Scalable, fault-tolerant async event handling
- **Webhooks + Slack API** → Instant delivery to apps and ops teams

SYSTEM ARCHITECTURE



IMPACT AND BENEFITS



Real-world use case:

• DeFi Lending Platform User Alert System

This system can be used by a DeFi lending platform to notify users in real time when their collateral value drops, liquidation risk increases, or loans are repaid or liquidated, based on live on-chain events. It helps users take timely action and prevents unexpected financial losses.

IMPACTS

- **Reduced Signal Blindness in Web3:** Converts overwhelming on-chain activity into meaningful, prioritized signals for users and teams.
- **Faster, Confident Decision-Making:** Enables real-time responses to critical events like large transfers, protocol risks, and governance actions.
- **More Reliable Web3 Infrastructure:** Finality-aware and reorg-safe alerts reduce false signals and increase trust in blockchain monitoring systems.



Efficiency

Users react to token moves, unlocks, and governance events instantly.
Eliminates manual dashboard refreshes and polling.



Feasibility

Uses public chain data and standard Web3 tooling.
Easy to extend to new chains and contracts by adding listeners, not redesigning systems.



Security

Leveraging immutable on-chain logs with strictly read-only access.
Zero asset custody ensures no write-risk or vulnerability to user funds.



Scalability

Horizontal scaling via stateless listeners and workload sharding (by chain/contract) ensures high performance as event volume grows.



Cost Effectiveness

Selective indexing and finality-aware processing slash RPC calls, compute, and storage, driving down monitoring costs.



Reliability

Reorg-safe and finality-weighted alerts ensure users act on reliable information, minimizing false positives and operational risk.

COMPARISON WITH EXISTING TOOLS



Why Chain Pulse is Better

Feature	Block Explorers	The Graph	Traditional Listeners
Real-time Alerts	 Manual Only	 No	 Custom Dev
Context & Decoding	 Raw Logs	 Limited	 JSON payload
Reorg Handling	 None	 Partial	 Manual
Noise Filtering	 No	 Basic	 Custom
Setup Effort	 None	 Complex	 High
Best For:	 Viewing Data	 Querying	 Developers



Bottom Line: Chain Pulse – Real-time, Actionable.

SECURITY CONSIDERATIONS



Built with Security First Mindset



Your Funds Are Never at Risk. We Only Read Public Data.

RESEARCH AND REFERENCES



Future Enhancements

- **Multi-Chain Support:** Extend beyond Ethereum to Polygon, Arbitrum, Solana
- **ML Signal Ranking:** Anomaly detection + priority scoring for critical alerts
- **Natural Language Rules:** Users define alerts like “Notify if >\$1M whale move”
- **Enterprise Integrations:** PagerDuty, Discord, SIEM, governance dashboards
- **Scalable Deployment:** Distributed workers, sharded indexing, high-throughput pipelines
- **Security Expansion:** Stronger audit trails, role-based access, proof anchoring

Features	Comparative Analysis		Target Audience	
	Traditional Tools	Our System		
Meaningful Signal Detection	✗	✓		
Finality-Aware Alerts	✗	✓		
Reorg Handling	✗	✓	 DeFi Protocol Teams	
Aggregated Insights	✗	✓		
Real-Time Notifications	!	✓		
Selective Indexing	✗	✓		
Production Reliability	!	✓	 DAO & Governance Operators	

Our product aims to become the trusted real-time intelligence layer for on-chain activity.

TOOLS & REFERENCES

- **Ethereum Event Logs + ABI Decoding** (ethers.js)
- **QuickNode / Infura RPC Providers** for low-latency ingestion
- **Node.js + TypeScript Backend** with PostgreSQL + Redis
- **Verified Alerts & Finality Concepts** (Ethereum Documentation)
- **Delivery Channels:** Telegram, Slack, Email, Webhooks

RESEARCH AND REFERENCES



Future Enhancements

- **Multi-Chain Support:** Extend beyond Ethereum to Polygon, Arbitrum, Solana
- **ML Signal Ranking:** Anomaly detection + priority scoring for critical alerts
- **Natural Language Rules:** Users define alerts like “Notify if >\$1M whale move”
- **Enterprise Integrations:** PagerDuty, Discord, SIEM, governance dashboards
- **Scalable Deployment:** Distributed workers, sharded indexing, high-throughput pipelines
- **Security Expansion:** Stronger audit trails, role-based access, proof anchoring

REAL-WORLD USE CASES

- **DeFi exploit & liquidity drain early-warning**
- **DAO treasury monitoring & governance oversight**
- **Whale tracking for exchanges and institutions**
- **Smart contract upgrade + admin risk detection**

TOOLS & REFERENCES

- **Ethereum Event Logs + ABI Decoding (ethers.js)**
- **QuickNode / Infura RPC Providers for low-latency ingestion**
- **Node.js + TypeScript Backend with PostgreSQL + Redis**
- **Verified Alerts & Finality Concepts (Ethereum Documentation)**
- **Delivery Channels: Telegram, Slack, Email, Webhooks**

Our product aims to become the trusted real-time intelligence layer for on-chain activity.